

Weekly Research Candidate - 2026-07-20

Candidate - awaiting-eval - not stable guidance

Status and scope

This is a CANDIDATE. It is not stable guidance and carries no authority. Nothing in this report has been applied to `Runtime/stable/`, `Core/`, the platform bootstraps, global pointers, sibling repositories or Git history. The run ends at `awaiting-eval`; promotion is a separate, human-approved step.

Run id `2026-07-20-weekly-research`. Objective: triage the **47** `pending-review` sources carried over from the Claude-v1 migration, recheck the **9 due active** registered sources, detect drift, then perform bounded discovery - producing an atomic claim ledger where every claim carries an independent verifier verdict. Result: **286 atomic claims** (282 accepted, 4 rejected), **77 proposed source-registry changes**, **100 drift observations** and **84 open questions**.

Method

Ten topic buckets covered all 56 due sources, so no due source was silently dropped. Each bucket ran a two-stage pipeline:

1. **Research** - fetch every assigned registered source first, emit a disposition for each (promote-active / update / retire / keep-pending with tier and recheck cadence), and only then perform bounded discovery for material newer than 2026-07-12.
2. **Independent verification** - a separate agent re-opened the citations and issued exactly one verdict per claim id. Enforced rules: a `version_sensitive` claim requires a current **tier-1** primary source or it is `unsupported` regardless of plausibility; a citation that does not directly support the statement is `unsupported` even if the statement seems true; tier 4-5 material is a lead, never sufficient authority for a technical mechanic or security policy.

The orchestration hard-failed on duplicate claim ids, a missing verdict, or an empty bucket, so the runbook's blocking conditions could not be skipped silently. 21 agents completed, 0 errors, 0 empty results.

Claim coverage

Verdict	Count
confirmed	251
corrected	31
unsupported	4

Accepted confidence: **high 271**, medium 11. **242 of 282** accepted claims are version-sensitive and therefore tier-1 backed.

Bucket	Accepted	Rejected
claude-loops-scheduling	27	0
claude-hooks-permissions	30	0
claude-context-memory	32	0
claude-agents-skills	31	0
codex-config-hooks	24	0
codex-exec-automation	42	2
codex-subagents-sandbox	26	0
security-sandboxing-injection	16	1
evals-judging	25	1
drift-discovery	29	0

Source changes

Proposed: **52 add** (24 promoted from `pending-review`, 28 newly discovered), **17 update**, **8 retire**. These are proposals in `source-registry.patch.json`; `Research/sources/registry.json` is untouched.

Retire (with cause)

- `claude-v1-code-claude-com-docs-en-agent-loop`** - Returns HTTP 404. The page now lives at <https://code.claude.com/docs/en/agent-sdk/agent-loop> (confirmed via the docs index at code.claude.com/docs/llms.txt and fetched successfully). Retire this URL and register the `/agent-sdk/` path in its place; the content itself is high value and supplied the `max_turns/max_budget_usd` claims.
 <<https://code.claude.com/docs/en/agent-loop>>
- `claude-v1-kunalganglani-com-blog-loop-engineering-agent-loops`** - Reachable but wholly derivative of code.claude.com documentation with no independent measurement or novel mechanic. Every technical element it names (skills in `.claude/skills/`, subagent context isolation, and the `PostToolUseFailure`, `Stop`, and `TaskCompleted` hooks) is documented upstream at tier 1, where it is more precise. Adds citation risk without adding information.
 <<https://kunalganglani.com/blog/loop-engineering-agent-loops>>
- `claude-v1-explainx-ai-blog-anthropic-engineer-loops-prompts-ai-coding-harness-engi`** - Reachable (June 2026) but built around uncited third-hand statistics attributed to Anthropic engineers (8x daily code output, over 80% of merged production code, 76% success rate, a '14%

Claude.md tax'). None traces to a primary source. Retire to avoid laundering unverifiable numbers into the repo; its underlying observe-plan-act-reflect framing is already covered at tier 1 and tier 3.

<<https://explainx.ai/blog/anthropic-engineer-loops-prompts-ai-coding-harness-engineering-2026>>

- **claude-v1-tembo-io-blog-claude-code-multi-agent-orchestration** - Fully subsumed by the tier-1 agent-teams doc and now materially stale: it states agent teams need 'Claude Code v2.1.32 or later' and documents 'Shift+Down to cycle through teammates', whereas current docs describe setup-free spawning from v2.1.178 and up/down arrow keys in the agent panel. It offers no quantitative measurements, and every qualitative claim it makes (3-5 teammates, three focused be
<<https://tembo.io/blog/claude-code-multi-agent-orchestration>>
- **claude-v1-developers-openai-com-codex-hooks** - Resolves, but only via HTTP 308 redirect to <https://learn.chatgpt.com/docs/hooks> -- which is already a separately registered assigned source. This entry is therefore a duplicate of the canonical URL and should be dropped from the registry to avoid double-fetching the same page each cycle. Retiring loses no content.
<<https://developers.openai.com/codex/hooks>>
- **claude-v1-simonwillison-net-2026-mar-16-codex-subagents** - Reachable, dated 2026-03-16, and it did corroborate the `~/codex/agents/` path and the three built-in agent names. But it is a static one-off post with no recurring content, and everything in it is now covered more precisely by tier-1 docs. It also carries author-acknowledged uncertainty (the worker vs default distinction). No ongoing recheck value.
<<https://simonwillison.net/2026/Mar/16/codex-subagents/>>
- **claude-v1-github-com-openai-codex-issues-16068** - Reachable but CLOSED as a duplicate of #16033. It will not receive further updates, so it is dead as a tracking source. Its root-cause narrative about `fill_to_context_window` is issue-author analysis, not maintainer-confirmed. Replace with #16033 as the live tracker.
<<https://github.com/openai/codex/issues/16068>>
- **claude-v1-amux-io-guides-ai-agent-sandboxing** - HTTP 403 Forbidden on fetch 2026-07-20; no response body retrieved. Cause is ambiguous between bot/UA filtering, geo-blocking, and removal, so unreachability alone is not proof the content is gone. Retiring rather than keeping pending because the bucket is already well covered by tier 1-2 primaries (Claude Code official sandboxing docs, NVIDIA AI Red Team guidance) that supply the same material wi
<<https://amux.io/guides/ai-agent-sandboxing/>>

Promote to active

- **claude-v1-code-claude-com-docs-en-goal** (tier 1, registry, recheck 2026-07-27) - Primary and only authoritative reference for the `/goal` evaluator: version floor, evaluator model, 4,000-char condition limit, resume semantics, and the `disableAllHooks/allowManagedHooksOnly` unavailability rules. Densest tier-1 sou

- **claude-v1-code-claude-com-docs-en-scheduled-tasks** (tier 1, registry, recheck 2026-07-27) - Authoritative for /loop modes, loop.md precedence and 25,000-byte truncation, ScheduleWakeup stop:true termination, CronCreate/CronList/CronDelete, the 50-task cap, 7-day expiry, jitter rules, and CLAUDE_CODE_DISABLE_CRON. Carries
- **claude-v1-code-claude-com-docs-en-routines** (tier 1, registry, recheck 2026-07-27) - Authoritative for cloud scheduling: 1-hour minimum interval, /schedule and /routines aliases, the <routine-fire-payload> untrusted-data wrapper, the experimental-cc-routine-2026-04-01 beta header, and the claude/-prefix branch-pus
- **claude-v1-anthropic-com-engineering-effective-harnesses-for-long-running-agents** (tier 3, registry, recheck 2026-08-19) - Vendor production engineering report (November 2025). Durable, citable source for the one-shotting and premature-completion failure modes and for checkpoint-artifact design. Stable prose, low churn, so a 30-day cadence is right. N
- **claude-v1-anthropic-com-engineering-harness-design-long-running-apps** (tier 3, registry, recheck 2026-08-19) - Vendor production engineering report (March 2026). Supplies the context-reset-versus-compaction distinction, the generator/evaluator/planner structure, and concrete cost envelopes for multi-hour harness runs. Retains value as the
- **claude-v1-code-claude-com-docs-en-hooks** (tier 1, registry, recheck 2026-07-27) - Assigned source, reachable and complete. The single most authoritative reference for this bucket: it carries the full hook event list, the exit-code-2-behavior-per-event table, the three-way matcher evaluation rules, all five hand
- **claude-v1-code-claude-com-docs-en-hooks-guide** (tier 1, registry, recheck 2026-07-27) - Assigned source, reachable and complete. Non-redundant with the reference despite the overlap: the guide is where the two load-bearing policy statements for this bucket actually live - the 'Hooks and permission modes' section stat
- **claude-v1-anthropic-com-engineering-claude-code-auto-mode** (tier 1, registry, recheck 2026-07-27) - Assigned source, reachable. First-party Anthropic engineering post, hence tier 1 on authority, though it is announcement-style prose rather than a reference and in practice changes far less often than the docs pages - the 7-day ca
- **wr20260720-code-claude-com-docs-en-permission-modes** (tier 1, discovered, recheck 2026-07-27) - Newly discovered during this pass and the strongest single source for the auto-mode half of the focus. Contains the complete default block-list and allow-list with per-version annotations, the four-step classifier decision order,
- **wr20260720-code-claude-com-docs-en-auto-mode-config** (tier 1, discovered, recheck 2026-07-27) - Newly discovered. The configuration reference for auto mode block-lists: the four autoMode.* keys, the '\$defaults' splice semantics and the danger of omitting it, the four-tier internal precedence, the settings-scope exclusion of
- **wr20260720-raw-githubusercontent-com-anthropics-claude-code-main-changelog-md** (tier 1, discovered, recheck 2026-07-27) - Newly discovered. Necessary for drift detection in a bucket where nearly every claim is version-sensitive - it is the only source that establishes the current release floor (v2.1.215 at time of access). Significant caveat for futu

- **claude-v1-code-claude-com-docs-en-context-window** (tier 1, registry, recheck 2026-07-27) - Highest-yield source in this bucket. Official Claude Code docs page carrying the authoritative 'What survives compaction' table (skill body caps of 5,000/25,000 tokens, path-scoped rule loss, auto memory re-injection), the v2.1.19
- **wr20260720-code-claude-com-docs-en-memory** (tier 1, discovered, recheck 2026-07-27) - NEW — discovered this pass. The primary reference for CLAUDE.md/MEMORY.md persistence: hierarchy and load order, the 200-line/25KB auto memory cap, ~/.claude/projects/<project>/memory/ layout, autoMemoryDirectory, claudeMdExcludes
- **wr20260720-code-claude-com-docs-en-env-vars** (tier 1, discovered, recheck 2026-07-27) - NEW — discovered this pass. Sole authoritative source for the compaction-threshold controls CLAUDE_AUTOCOMPACT_PCT_OVERRIDE and CLAUDE_CODE_AUTO_COMPACT_WINDOW, plus CLAUDE_CODE_DISABLE_AUTO_MEMORY and CLAUDE_CODE_DISABLE_1M_CONTE
- **wr20260720-code-claude-com-docs-en-model-config** (tier 1, discovered, recheck 2026-07-27) - NEW — discovered this pass. Carries the only published concrete auto-compaction token threshold (Sonnet 5 at about 967K by default), the extended-context model list, [1m] suffix semantics including per-variable behavior on Bedrock
- **wr20260720-code-claude-com-docs-en-how-claude-code-works** (tier 1, discovered, recheck 2026-07-27) - NEW — discovered this pass. Documents the two-stage context-fill behavior (older tool outputs cleared first, then conversation summarized), the auto-compaction thrashing guard, and the Compact Instructions CLAUDE.md section. Fills
- **wr20260720-code-claude-com-docs-en-changelog** (tier 1, discovered, recheck 2026-07-27) - NEW — discovered this pass. Required for drift detection in this bucket: it is the only way to date memory/context changes against a weekly baseline. Confirmed current release v2.1.214 (2026-07-18). Note it disagrees with the memo
- **wr20260720-platform-claude-com-docs-en-build-with-claude-context-editing** (tier 1, discovered, recheck 2026-07-27) - NEW — discovered this pass. Authoritative for API-side context management, which is a different mechanism from Claude Code compaction and is routinely conflated with it: clear_tool_uses_20250919 and clear_thinking_20251015 strateg
- **wr20260720-claude-com-blog-claude-managed-agents-memory** (tier 1, discovered, recheck 2026-07-27) - NEW — discovered this pass. Official announcement of Managed Agents memory (public beta, 2026-04-23), which is the t1 corroboration that makes the assigned t5 usewire source redundant for factual purposes. Relevant to this bucket
- **wr20260720-code-claude-com-docs-en-costs** (tier 1, discovered, recheck 2026-07-27) - Newly surfaced; sole first-party quantified cost claim for agent teams (~7x tokens vs a standard session when teammates run in plan mode) plus the CLAUDE.md-under-200-lines and move-instructions-to-skills guidance. Directly suppor
- **claude-v1-code-claude-com-docs-en-agent-teams** (tier 1, registry, recheck 2026-07-27) - Authoritative and by far the densest source in this bucket: enable flag, teammate-vs-subagent comparison table, permission/trust boundary, on-disk layout, hook events, sizing guidance, and an explicit limitations list. Carries inl

- **wr20260720-code-claude-com-docs-en-sub-agents** (tier 1, discovered, recheck 2026-07-27) - Newly surfaced in this pass and essential to the bucket: complete frontmatter field table, scope precedence, model resolution order, foreground/background defaults, worktree isolation, nested subagents, and the plugin security car
- **wr20260720-code-claude-com-docs-en-skills** (tier 1, discovered, recheck 2026-07-27) - Newly surfaced; the only source found carrying hard, checkable skill-authoring constraints (1,536-character description+when_to_use cap, single-message persistence with no re-read, context: fork, disable-model-invocation semantics)
- **wr20260720-research-google-blog-towards-a-science-of-scaling-agent-systems-when-and-why-ag** (tier 2, discovered, recheck 2026-08-19) - Newly discovered and the best quantitative evidence found for the helps-vs-harms question: 180 configurations, 5 architectures, 4 benchmarks, 3 model families, with 17.2x vs 4.4x error amplification and a +80.9% / -39-70% split by
- **claude-v1-learn-chatgpt-com-docs-hooks** (tier 1, registry, recheck 2026-07-27) - Canonical official reference for the entire Codex hooks surface: all ten event names, scope split, handler types, TOML/JSON schema, trust model, exit-code semantics, and the /hooks TUI command. Sole or primary source for 11 claims
- **claude-v1-learn-chatgpt-com-docs-config-file-config-reference** (tier 1, registry, recheck 2026-07-27) - Authoritative key-by-key config.toml reference. Uniquely carries the project-scoped config trust gate and the explicit ignored-key list, which is the load-bearing text for any Codex threat model. Also independently corroborates th
- **claude-v1-learn-chatgpt-com-docs-agent-configuration-agents-md** (tier 1, registry, recheck 2026-07-27) - Only source found that states the project_doc_max_bytes default (32 KiB), the root-downward discovery direction, the AGENTS.override.md filename, and the skip-empty/stop-at-limit truncation behavior. Note it describes the byte lim
- **wr20260720-learn-chatgpt-com-docs-changelog** (tier 1, discovered, recheck 2026-07-27) - Newly discovered, not previously assigned. Dated per-version entries make it the fastest way to detect drift in this bucket week over week, and it supplies the version anchor (0.144.6, 2026-07-18). Recommend adding to the registry
- **wr20260720-github-com-openai-codex-releases** (tier 1, discovered, recheck 2026-07-27) - Newly discovered, not previously assigned. Gives exact release tags and timestamps including the prerelease line, which the docs changelog does not surface. Useful as the authoritative version anchor for all version_sensitive clai
- **claude-v1-learn-chatgpt-com-docs-non-interactive-mode** (tier 1, registry, recheck 2026-07-27) - Primary authoritative reference for codex exec: exact flags (--json, --output-schema, --ephemeral, -o, --ignore-user-config, --ignore-rules, --skip-git-repo-check), JSONL event taxonomy, stdin semantics, auth via CODEX_API_KEY, an
- **claude-v1-learn-chatgpt-com-docs-automations** (tier 1, registry, recheck 2026-07-27) - Authoritative on the review queue (Scheduled view as inbox, unread indicator, mark-all-as-read, All/Active/Paused filters), worktree vs local execution, RRULE scheduling, standalone vs in-chat models, and the approval_policy = "ne

- **claude-v1-developers-openai-com-blog-run-long-horizon-tasks-with-codex** (tier 1, registry, recheck 2026-07-27) - Official vendor engineering post and the only assigned source covering long-horizon guidance. Supplies the four-file project-memory pattern, per-milestone acceptance criteria plus validation commands, and the ~25h / ~13M token / ~
- **wr20260720-github-com-openai-codex-pull-25021** (tier 1, discovered, recheck 2026-07-27) - NEWLY DISCOVERED. Authoritative provenance for session archiving: exact commands, thread-name-or-UUID resolution, thread/archive RPCs, resume/fork rejection behaviour, files touched, merged 2026-05-30. This is the tier-1 anchor th
- **wr20260720-learn-chatgpt-com-docs-build-skills** (tier 1, discovered, recheck 2026-07-27) - NEWLY DISCOVERED and now the authoritative surface for reusable instructions, since custom prompts are deprecated. Supplies skill discovery paths (.agents/skills scanned cwd-to-repo-root, \$HOME/.agents/skills, /etc/codex/skills),
- **wr20260720-github-com-openai-codex-action** (tier 1, discovered, recheck 2026-07-27) - NEWLY DISCOVERED. Official GitHub Action wrapping codex exec; authoritative on inputs, the Responses API proxy, and the four safety strategies including the Windows unsafe caveat. Directly relevant to headless automation guidance.
- **wr20260720-learn-chatgpt-com-docs-agent-configuration-subagents** (tier 1, discovered, recheck 2026-07-27) - Canonical post-migration home of the subagents documentation. Sole authoritative source for agents.max_threads/max_depth defaults, the custom agent TOML field list, built-in agent names, and the inheritance rule. Primary reference
- **claude-v1-github-com-openai-codex-blob-main-codex-rs-linux-sandbox-readme-md** (tier 1, registry, recheck 2026-07-27) - Durable, citable, in-repo primary source and the only place documenting the bubblewrap-first backend, exact bwrap flags, PR_SET_NO_NEW_PRIVS + seccomp behaviour, the codex-resources/bwrap fallback, features.use_legacy_landlock, an
- **claude-v1-learn-chatgpt-com-docs-agent-approvals-security** (tier 1, registry, recheck 2026-07-27) - Reachable tier-1 primary. Authoritative for approval_policy values including the granular table form, the three sandbox_mode values, per-platform enforcement (Seatbelt / bwrap+seccomp / Windows native / WSL2), the protected .git/.
- **claude-v1-learn-chatgpt-com-docs-extend-mcp** (tier 1, registry, recheck 2026-07-27) - Reachable tier-1 primary for the Codex MCP surface: [mcp_servers.<name>] table, stdio vs streamable-HTTP key sets, startup_timeout_sec/tool_timeout_sec defaults, enabled_tools/disabled_tools, default_tools_approval_mode values, co
- **claude-v1-sophos-com-en-us-blog-inside-the-lethal-trifecta-blast-radius-reduction** (tier 3, registry, recheck 2026-08-19) - Highest-value source in the assigned set. CISO-authored production engineering report organised as seven named control patterns (agent sandboxing, credential isolation, sealed tool endpoints, egress restriction, EDR, human-gated a
- **claude-v1-arxiv-org-abs-2606-29920** (tier 2, registry, recheck 2026-08-19) - Reachable; submitted 2026-06-29. Directly on-bucket: the first meta-evaluation benchmark for LLM-as-a-Judge reliability in rubric verification for agentic scenarios, including an agentic-coding split over serialized trajectories.

- **wr20260720-arxiv-org-abs-2607-05904** (tier 2, discovered, recheck 2026-08-19) - Newly discovered; submitted 2026-07-07. Highest-value find this pass: gives a clean quantified separation between judge score and true accuracy under self-play (0.72 to 0.94 pass rate at 0.20 true accuracy), shows ensembling fails
- **wr20260720-arxiv-org-abs-2607-01211** (tier 2, discovered, recheck 2026-08-19) - Newly discovered; v1 2026-07-01, v2 2026-07-16 (the revision falls inside the discovery window). Direct audit of coding-agent benchmark validity with hard replay numbers and evidence of benchmark over-optimization. Core reference
- **wr20260720-arxiv-org-abs-2606-19544** (tier 2, discovered, recheck 2026-08-19) - Newly discovered; submitted 2026-06-17. Largest systematic judge evaluation located (21 judges, 9 providers, ~541k judgments) and the source of the reliability-versus-validity distinction plus the kappa deflation figure. Anchors t
- **wr20260720-arxiv-org-abs-2605-02964** (tier 2, discovered, recheck 2026-08-19) - Newly discovered; submitted 2026-05-03. Primary reward-hacking benchmark for tool-using agents with per-model exploit rates, CoT-monitor coverage (72 percent), and a measured environment-hardening effect. Directly serves the rewar
- **wr20260720-arxiv-org-abs-2606-04923** (tier 2, discovered, recheck 2026-08-19) - Newly discovered; submitted 2026-06-03. Connects LLM-as-a-judge bias directly to reward hacking via the CHERRL controlled-injection framework, which is a transferable red-team method for validating that a harness can detect judge-
- **wr20260720-github-com-sourcegraph-codescalebench** (tier 1, discovered, recheck 2026-07-27) - Newly discovered official repo, Apache 2.0. Source of the most directly actionable artifact this pass: the null / golden / adversarial calibration triad gating verifier admission, plus documentation that scoring is deterministic r
- **wr20260720-sourcegraph-com-blog-codescalebench-testing-coding-agents-on-large-codebases-an** (tier 3, discovered, recheck 2026-08-19) - Newly discovered; published 2026-03-03. The substantive eval-methodology post that the assigned agentic-coding blog only gestures at: task counts, suite split, verifier design, pinned mirrors, and measured retrieval deltas. Vendor
- **wr20260720-github-com-thu-keg-ruverbench** (tier 1, discovered, recheck 2026-07-27) - Newly discovered official repo for the assigned RuVerBench paper; MIT code with separately licensed data. Supplies the case/rubric-point decomposition absent from the abstract and ships regenerable results tables. Repo is young (3
- **openai-codex-docs** (tier 1, registry, recheck 2026-07-27) - Now the canonical host for OpenAI API, ChatGPT, Codex and Apps SDK documentation; developers.openai.com paths 308-redirect here. Rich, citable primary reference covering Codex CLI config, approvals and security. Caveat for the reg
- **anthropic-claude-code-docs** (tier 1, registry, recheck 2026-07-27) - Durable, high-quality primary reference. Confirmed install surfaces, the Homebrew stable-vs-latest channel split, and the surface matrix. Efficiency note for future passes: the page header advertises <https://code.claude.com/docs/>
- **simon-willison** (tier 5, registry, recheck 2026-07-27) - Best discovery surface in this pass by a wide margin: 17 dated entries inside the window, and it independently surfaced the Codex destructive-rm incident that corroborates the tier-1 Codex CLI 0.144.5 changelog entry, plus the Fab

Update in place

- **claude-v1-anthropic-com-engineering-effective-context-engineering-for-ai-agents** (tier 3, next check 2026-08-19) - Reachable and still the canonical vendor framing (context engineering definition, context rot, compaction, structured note-taking, sub-agent architectures, just-in-time retrieval). Retier from t1 to t3: it is a vendor engineering
- **wr20260720-claude-com-blog-context-management** (tier 1, next check 2026-07-27) - NEW — discovered this pass; note www.anthropic.com/news/context-management now 308-redirects here, so any registry entry on the old anthropic.com host should be rewritten to the claude.com URL. Sole primary source for the 29% / 39
- **claude-v1-claude-com-blog-lessons-from-building-claude-code-how-we-use-skills** (tier 3, next check 2026-08-19) - Retier from the registered t5 to t3: this is Anthropic's own production engineering report on internal Skills practice, not practitioner/social commentary. Content is durable authoring guidance (trigger-not-summary descriptions, G
- **claude-v1-cognition-com-blog-dont-build-multi-agents** (tier 3, next check 2026-08-19) - Retier from t5 to t3: a production engineering report from an agent company, and the canonical statement of the anti-multi-agent position. Content is stable and roughly 13 months old, so 30-day cadence is right. Keep it as the del
- **claude-v1-developers-openai-com-codex-config-advanced** (tier 1, next check 2026-07-27) - Resolves via HTTP 308 to <https://learn.chatgpt.com/docs/config-file/config-advanced> (note the added /config-file/ path segment). Content is high-value and not registered under any other URL, so update the entry's URL to the canoni
- **claude-v1-learn-chatgpt-com-docs-developer-commands** (tier 1, next check 2026-07-27) - Reachable and useful, but MIS-SCOPED in the registry: it is a built-in slash-command and CLI-subcommand reference, not a spec for user-authored developer commands (no directory, file format, frontmatter or precedence for custom co
- **claude-v1-learn-chatgpt-com-docs-custom-prompts** (tier 1, next check 2026-07-27) - Still reachable and precise on syntax, but the feature it documents is now explicitly DEPRECATED in favour of skills. Retain as the authoritative deprecation record and migration reference for legacy ~/.codex/prompts setups; do no
- **claude-v1-developers-openai-com-codex-cli-features** (tier 1, next check 2026-07-27) - Returns 308 Permanent Redirect to a different host: <https://learn.chatgpt.com/docs/codex/cli>. Content is alive and valuable; the registered URL is stale. Update the registry entry to the learn.chatgpt.com URL. This is part of a si
- **claude-v1-developers-openai-com-codex-subagents** (tier 1, next check 2026-07-27) - Reachable but returns HTTP 308 to <https://learn.chatgpt.com/docs/agent-configuration/subagents>. Content is high-value tier-1 primary documentation for the [agents] table, .codex/agents/*.toml schema, and sandbox/approval inheritan
- **claude-v1-helptestsecurity-com-2026-06-11-owasp-prompt-injection-ai-security-failur** (tier 5, next check 2026-07-27) - Reachable and accurate secondary reporting on OWASP 'State of Agent AI Security and Governance' v2.01, but it is trade-press coverage of a standards document, not the document. Registered tier t5 is correct and should stay. Reco

- `claude-v1-northflank-com-blog-best-code-execution-sandbox-for-ai-agents` (tier 5, next check 2026-07-27) - Retier from t3 to t5. This is not a production engineering report but a vendor-authored competitive comparison in which the authoring vendor leads on cold start, session limits, BYOC and price simultaneously. It also carries an in
- `claude-v1-modal-com-resources-best-code-execution-sandboxes-ai-agents` (tier 5, next check 2026-07-27) - Retier from t3 to t5, same rationale as the Northflank entry: vendor-authored comparison (dated May 2026) in which Modal leads on GPU breadth, concurrency and compliance. Value is that it independently corroborates the isolation-m
- `claude-v1-sourcegraph-com-blog-agentic-coding` (tier 3, next check 2026-08-19) - Reachable; published 2026-05-21. Legitimate tier-3 production engineering writing, but for the evals-judging bucket it is nearly content-free: one pointer to CodeScaleBench and one vendor adoption statistic (84 percent LOC increas
- `openai-codex-changelog` (tier 1, next check 2026-07-27) - Content is live and high-yield (five Codex CLI releases in the 2026-07-13..2026-07-18 window) but the URL now returns 308 Permanent Redirect to <https://learn.chatgpt.com/docs/changelog>. Retarget the registered URL to the redirect
- `openai-codex-blog` (tier 1, next check 2026-07-27) - Fetched successfully but yielded nothing dated in the 2026-07-12 window, and the rendered dates were unreliable (most-recent entries parsed as June 2024, which conflicts with the site's own recency). Given sibling paths on this ho
- `anthropic-claude-code-changelog` (tier 1, next check 2026-07-27) - The highest-yield source in this pass, but the registered blob URL is the wrong one to fetch: the GitHub HTML view returns only page chrome and navigation, no changelog body (the file is ~450 KB / 5112 lines). Retarget to <https://>
- `anthropic-engineering` (tier 1, next check 2026-07-27) - Reachable and authoritative but currently low-yield: newest post is 2026-04-23, nothing in the drift window, and three consecutive weekly passes would likely find it unchanged. Keep tier 1 and the 7-day cadence since postmortems
|

Accepted claims

All 282 accepted claims, grouped by bucket. Statements are abridged here for reviewability -

`claims.json` is the authoritative machine-readable ledger, carrying each claim's full statement, candidate guidance, every supporting source, the verifier's reason and an `expires_at` recheck horizon.

claude-loops-scheduling (27)

- `claude-loops-scheduling-01` (confirmed/high) `version-sensitive` - The `/goal` command requires Claude Code v2.1.139 or later, sets one completion condition per session, and clears itself automatically once the condition is met.

Applies to: Claude Code CLI v2.1.139+ (interactive, headless -p, desktop, Remote Control) - [Keep Claude working toward a goal](#)

- **claude-loops-scheduling-02** (confirmed/high) **version-sensitive** - /goal is implemented as a wrapper around a session-scoped prompt-based Stop hook: after each turn the condition plus the conversation are sent to the configured small fast model (default Haiku), which returns a yes/no de ...

Applies to: Claude Code CLI v2.1.139+ - [Keep Claude working toward a goal](#)

- **claude-loops-scheduling-03** (confirmed/high) **version-sensitive** - The /goal evaluator does not call tools; it can only judge what Claude has already surfaced in the conversation transcript.

Applies to: Claude Code CLI v2.1.139+ - [Keep Claude working toward a goal](#)

- **claude-loops-scheduling-04** (confirmed/high) **version-sensitive** - A /goal condition can be up to 4,000 characters, and there is no separate turn-limit flag: bounding a goal's runtime is done by writing a turn or time clause into the condition text itself (for example 'or stop after 20 ...

Applies to: Claude Code CLI v2.1.139+ - [Keep Claude working toward a goal](#)

- **claude-loops-scheduling-05** (confirmed/high) **version-sensitive** - /goal is unavailable in untrusted workspaces, when disableAllHooks is set at any settings level, or when allowManagedHooksOnly is set in managed settings; the command reports the reason rather than silently doing nothing ...

Applies to: Claude Code CLI v2.1.139+; enterprise/managed-settings environments - [Keep Claude working toward a goal](#)

- **claude-loops-scheduling-06** (confirmed/high) **version-sensitive** - An active goal is restored when a session is resumed with --resume or --continue, but the turn count, timer, and token-spend baseline all reset on resume; an already-achieved or cleared goal is not restored.

Applies to: Claude Code CLI v2.1.139+ - [Keep Claude working toward a goal](#)

- **claude-loops-scheduling-07** (confirmed/high) **version-sensitive** - /loop behavior is determined by what is supplied: interval plus prompt runs on a fixed cron schedule; prompt only runs at a Claude-chosen interval between one minute and one hour picked after each iteration; interval onl ...

Applies to: Claude Code CLI v2.1.x (bundled /loop skill); different on Amazon Bedrock, Claude Platform on AWS, Google Cloud Agent PI ... - [Run prompts on a schedule](#)

- **claude-loops-scheduling-08** (confirmed/high) **version-sensitive** - A loop.md file replaces the built-in maintenance prompt for bare /loop; Claude reads .claude/loop.md (project scope, takes precedence) then ~/.claude/loop.md (user scope), edits take effect on the next iteration, and con ...

Applies to: Claude Code CLI v2.1.x; not read on Amazon Bedrock, Claude Platform on AWS, Google Cloud Agent Platform, or Microsoft Fo ... - [Run prompts on a schedule](#)

- **claude-loops-scheduling-09** (confirmed/high) **version-sensitive** - In self-paced /loop mode Claude ends the loop by calling the ScheduleWakeup tool with stop: true, which cancels the pending wakeup immediately; if an iteration ends without either rescheduling or stopping, Claude Code sc ...

Applies to: Claude Code CLI v2.1.202+ for the stop: true path; earlier versions self-terminate only by omitting a reschedule - [Run prompts on a schedule](#)

- **claude-loops-scheduling-10** (confirmed/high) **version-sensitive** - Session-scoped scheduling uses three tools, CronCreate, CronList, and CronDelete, accepting standard 5-field cron expressions interpreted in local time; each task has an 8-character ID and a session can hold at most 50 s ...

Applies to: Claude Code CLI v2.1.x, in-session (not Routines or Desktop scheduled tasks) - [Run prompts on a schedule](#)

- **claude-loops-scheduling-11** (confirmed/high) **version-sensitive** - Recurring session-scoped scheduled tasks automatically expire 7 days after creation: the task fires one final time and then deletes itself, and this expiry also applies to dynamically scheduled (self-paced) loops.

Applies to: Claude Code CLI v2.1.x, in-session scheduling only - [Run prompts on a schedule](#)

- **claude-loops-scheduling-12** (confirmed/high) **version-sensitive** - The scheduler applies deterministic jitter derived from the task ID: recurring tasks fire up to 30 minutes after the scheduled time (or up to half the interval for sub-hourly tasks), and one-shot tasks scheduled for the ...

Applies to: Claude Code CLI v2.1.x, in-session scheduling - [Run prompts on a schedule](#)

- **claude-loops-scheduling-13** (confirmed/high) **version-sensitive** - Setting CLAUDE_CODE_DISABLE_CRON=1 in the environment disables the scheduler entirely: the cron tools and /loop become unavailable and already-scheduled tasks stop firing.

Applies to: Claude Code CLI v2.1.x - [Run prompts on a schedule](#)

- **claude-loops-scheduling-14** (confirmed/high) **version-sensitive** - As of Claude Code v2.1.196, a scheduled fire only executes skills that Claude is allowed to invoke on its own; built-in commands such as /permissions, /model, or /clear, skills marked disable-model-invocation: true, skill ...

Applies to: Claude Code CLI v2.1.196+ - [Run prompts on a schedule](#)

- **claude-loops-scheduling-15** (confirmed/high) **version-sensitive** - Claude Code v2.1.215 (July 19, 2026) changed the /verify and /code-review skills so Claude no longer invokes them on its own; they must be invoked explicitly by the user.

Applies to: Claude Code CLI v2.1.215+ - [Claude Code changelog](#)

- **claude-loops-scheduling-16** (confirmed/high) **version-sensitive** - Routines run on Anthropic-managed cloud infrastructure with a minimum schedule interval of one hour; cron expressions that run more frequently are rejected. Routines are in research preview and are created from claude.ai ...

Applies to: Claude Code Routines research preview, July 2026; Pro/Max/Team/Enterprise plans with Claude Code on the web enabled - [Automate work with routines](#)

- **claude-loops-scheduling-17** (confirmed/high) **version-sensitive** - Text sent to a routine's /fire endpoint (or supplied via Run now) does not reach the routine as a bare message: it arrives

wrapped in a <routine-fire-payload> block labeled as untrusted data, and the routine's saved prom
...

Applies to: Claude Code Routines research preview, July 2026;

api.anthropic.com/v1/claude_code/routines/{id}/fire - [Automate work with routines](#)

- **claude-loops-scheduling-18** (confirmed/high) **version-sensitive** - Routines run autonomously as full Claude Code cloud sessions with no permission-mode picker and no approval prompts during a run; by default Claude can only push to branches prefixed with claude/, unless 'Allow unrestric ...

Applies to: Claude Code Routines research preview, July 2026 - [Automate work with routines](#)

- **claude-loops-scheduling-19** (confirmed/high) **version-sensitive** - The Agent SDK caps the loop with max_turns / maxTurns (counting tool-use round trips only) and max_budget_usd / maxBudgetUsd; both default to no limit, and hitting either returns a ResultMessage with subtype error_max_tu ...

Applies to: Claude Agent SDK (Python and TypeScript), July 2026 - [How the agent loop works](#)

- **claude-loops-scheduling-20** (confirmed/high) **version-sensitive** - A single-shot query() that ends on an error result yields the final result message and then raises/throws (the Claude Code process also exits nonzero), whereas a streaming-input session stays alive and can keep receiving ...

Applies to: Claude Agent SDK (Python and TypeScript), July 2026 - [How the agent loop works](#)

- **claude-loops-scheduling-21** (confirmed/high) **version-sensitive** - The Monitor tool streams background script output or WebSocket events back into a running session instead of polling, uses the same permission rules as Bash for commands, and is unavailable on Amazon Bedrock, Google Clou ...

Applies to: Claude Code CLI v2.1.x; WebSocket source v2.1.195+ - [Tools reference](#)

- **claude-loops-scheduling-22** (corrected/high) **version-sensitive** - Claude Code v2.1.212 (July 17, 2026) added two per-session runaway-loop ceilings: a cap on subagent spawns per session (default 200, override with CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION, reset by /clear) explicitly to sto ...

Applies to: Claude Code CLI v2.1.x (July 2026 changelog); exact introducing version uncertain, see open questions - [Claude Code changelog](#)

- **claude-loops-scheduling-23** (confirmed/high) **version-sensitive** - In claude -p runs, background Bash tasks are terminated about five seconds after the final result and stdin closes, while background subagents and workflows are waited for; from v2.1.182 that wait is capped at ten minute ...

Applies to: Claude Code CLI v2.1.182+ in non-interactive (-p) mode - [Run Claude Code programmatically](#)

- **claude-loops-scheduling-24** (confirmed/high) **version-sensitive** - Claude Code documents three distinct ways to keep a session running between prompts, differing in what starts the next turn

and what stops it: /goal (next turn starts when the previous finishes; stops when a model confir ...

Applies to: Claude Code CLI v2.1.139+ for /goal; v2.1.x for /loop and Stop hooks - [Keep Claude working toward a goal](#)

- **claude-loops-scheduling-25** (corrected/medium) **version-sensitive** - Anthropic's harness-design engineering write-up (March 24, 2026) distinguishes context resets - 'clearing the context window entirely and starting a fresh agent, combined with a structured handoff that carries the previo ...

Applies to: Vendor-agnostic harness design; model-version specifics unresolved, see open questions - [Harness design for long-running application development](#)

- **claude-loops-scheduling-26** (confirmed/high) - Anthropic's long-running-agent harness guidance identifies two named failure modes, attempting to complete a project inside a single context window and prematurely declaring completion, and addresses them with an initial ...

Applies to: Vendor-agnostic long-running agent harness design; published November 2025 - [Effective harnesses for long-running agents](#)

- **claude-loops-scheduling-27** (confirmed/high) - The Ralph technique is a bare shell loop of the form 'while ;; do cat PROMPT.md | claude-code ; done' that runs one discrete task per iteration, carries state through specification and plan files (notably a fix_plan.md T ...

Applies to: Vendor-agnostic practitioner pattern; original post dated July 2025, predates /goal and /loop - [Ralph](#)

claude-hooks-permissions (30)

- **claude-hooks-permissions-01** (confirmed/high) **version-sensitive** - A Claude Code hook that exits with code 2 signals a blocking error: Claude Code ignores the hook's stdout (including any JSON it printed) and feeds the hook's stderr text back to Claude as the error message.

Applies to: Claude Code v2.1.x CLI, command-type hooks (verified against docs at CHANGELOG v2.1.215) - [Hooks reference](#) - [Claude Code Docs](#)

- **claude-hooks-permissions-02** (confirmed/high) **version-sensitive** - Exit code 1, and any non-zero exit code other than 2, is treated as a NON-blocking error for hook events: the action proceeds, the transcript shows a `<hook name> hook error` notice followed by the first line of stderr, ...

Applies to: Claude Code v2.1.x CLI, command-type hooks - [Hooks reference](#) - [Claude Code Docs](#)

- **claude-hooks-permissions-03** (confirmed/high) **version-sensitive** - Exit code 2 actually blocks on these hook events: PreToolUse, PermissionRequest, UserPromptSubmit, UserPromptExpansion, Stop, SubagentStop, TeammateIdle, TaskCreated, TaskCompleted, ConfigChange, PostToolBatch, PreCompac ...

Applies to: Claude Code v2.1.x CLI - [Hooks reference](#) - [Claude Code Docs](#)

- **claude-hooks-permissions-04** (confirmed/high) **version-sensitive** - Exit code 2 has no blocking effect on these hook events: StopFailure, PostToolUse, PostToolUseFailure,

PermissionDenied, Notification, SubagentStart, SessionStart, Setup, SessionEnd, CwdChanged, FileChanged, PostCompact, ...

Applies to: Claude Code v2.1.x CLI - [Hooks reference](#) - [Claude Code Docs](#)

- **claude-hooks-permissions-05** (confirmed/high) `version-sensitive` - PreToolUse hooks fire before any permission-mode check, in every permission mode including `dontAsk`; a hook returning `permissionDecision: "deny"` blocks the tool even in `bypassPermissions` mode or under `--dangerously` ...

Applies to: Claude Code v2.1.x CLI, PreToolUse hooks - [Automate actions with hooks](#) - [Claude Code Docs](#)

- **claude-hooks-permissions-06** (confirmed/high) `version-sensitive` - The converse of hook-deny precedence does not hold: a PreToolUse hook returning `permissionDecision: "allow"` skips the interactive prompt but does not override deny rules from any settings scope including managed settin ...

Applies to: Claude Code v2.1.x CLI, PreToolUse hooks - [Automate actions with hooks](#) - [Claude Code Docs](#)

- **claude-hooks-permissions-07** (confirmed/high) `version-sensitive` - PreToolUse structured output uses `hookSpecificOutput.permissionDecision` with values `allow`, `deny`, and `ask`, plus `permissionDecisionReason` for the explanation; a fourth value `defer` exists only in non-interactive ...

Applies to: Claude Code v2.1.x CLI, PreToolUse hooks; `defer` is headless/-p only - [Automate actions with hooks](#) - [Claude Code Docs](#)

- **claude-hooks-permissions-08** (confirmed/high) `version-sensitive` - All hooks matching an event run in parallel and every one runs to completion - one hook returning deny does not stop sibling hooks from executing or from causing side effects. For PreToolUse permission decisions the most ...

Applies to: Claude Code v2.1.x CLI, all hook events - [Automate actions with hooks](#) - [Claude Code Docs](#)

- **claude-hooks-permissions-09** (confirmed/high) `version-sensitive` - When multiple PreToolUse hooks return `updatedInput` to rewrite a tool's arguments, the last one to finish takes effect, and because hooks run in parallel that order is non-deterministic.

Applies to: Claude Code v2.1.x CLI, PreToolUse hooks - [Automate actions with hooks](#) - [Claude Code Docs](#)

- **claude-hooks-permissions-10** (corrected/high) `version-sensitive` - For most hook events a `matcher` is evaluated three different ways: `"*"`, `"`, or an omitted matcher matches everything; a value containing only letters, digits, underscore, hyphen, spaces, comma and pipe is treated as ...

Applies to: Claude Code v2.1.x CLI, hook matcher field - [Hooks reference](#) - [Claude Code Docs](#)

- **claude-hooks-permissions-11** (confirmed/high) **version-sensitive** - Comma and pipe are interchangeable list separators in tool-name matchers (so `Edit|Write` and `Edit, Write` are equivalent) as of Claude Code v2.1.191; hyphens inside an exact-match set require v2.1.195 or later.

Applies to: Claude Code v2.1.191+ (comma separators), v2.1.195+ (hyphens in exact-match sets) - [Hooks reference - Claude Code Docs](#)

- **claude-hooks-permissions-12** (confirmed/high) **version-sensitive** - These hook events accept no matcher and always fire on every occurrence: `UserPromptSubmit`, `PostToolBatch`, `Stop`, `TeammateIdle`, `TaskCreated`, `TaskCompleted`, `WorktreeCreate`, `WorktreeRemove`, `CwdChanged`, and `MessageDisplay`.

Applies to: Claude Code v2.1.x CLI - [Hooks reference - Claude Code Docs](#)

- **claude-hooks-permissions-13** (confirmed/high) **version-sensitive** - MCP tools are matched by the name pattern `mcp__<server>__<tool>`, and matching all tools from one server requires an explicit regex such as `mcp__memory__.*` - the trailing `.*` is required. Plugin-bundled MCP servers u ...

Applies to: Claude Code v2.1.x CLI, tool-name matcher events (`PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `PermissionRequest`, `Permis ...` - [Hooks reference - Claude Code Docs](#)

- **claude-hooks-permissions-14** (confirmed/high) **version-sensitive** - The per-handler `if` field filters using permission-rule syntax (e.g. `"Bash(git *)"`, `"Edit(*.ts)"`) and works only on the tool events `PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `PermissionRequest` and `PermissionDenied` ...

Applies to: Claude Code v2.1.x CLI, hook handler `if` field - [Automate actions with hooks - Claude Code Docs](#)

- **claude-hooks-permissions-15** (confirmed/high) **version-sensitive** - Default hook timeouts are 600 seconds (10 minutes) for `command`, `http` and `mcp_tool` handlers, 30 seconds for `prompt` handlers, and 60 seconds for `agent` handlers; `UserPromptSubmit` lowers the `command/http/mcp_tool` t ...

Applies to: Claude Code v2.1.x CLI, all hook handler types - [Hooks reference - Claude Code Docs](#)

- **claude-hooks-permissions-16** (confirmed/high) **version-sensitive** - Claude Code overrides a `Stop` hook after it blocks eight times in a row without progress; hook scripts are expected to read the `stop_hook_active` field from their JSON input and exit 0 when it is true, and the cap can be ...

Applies to: Claude Code v2.1.x CLI, `Stop` and `SubagentStop` hooks - [Automate actions with hooks - Claude Code Docs](#)

- **claude-hooks-permissions-17** (confirmed/high) **version-sensitive** - `PermissionRequest` hooks do not fire in non-interactive mode with the `-p` flag; the documented substitute for automated permission decisions in headless runs is a `PreToolUse` hook.

Applies to: Claude Code v2.1.x CLI, headless/`-p` sessions - [Automate actions with hooks - Claude Code Docs](#)

- claude-hooks-permissions-18** (corrected/high) `version-sensitive` - In auto mode each action goes through a fixed decision order where the first matching step wins: (1) actions matching allow, ask or deny rules resolve immediately — except that writes to protected paths route to the clas ...
Applies to: Claude Code v2.1.x CLI, `auto` permission mode - [Choose a permission mode - Claude Code Docs](#)
- claude-hooks-permissions-19** (confirmed/high) `version-sensitive` - `permissions.deny` rules block an action before the auto mode classifier is consulted and cannot be overridden by classifier configuration or by user intent stated in conversation; the `autoMode` prose rule lists are a s ...
Applies to: Claude Code v2.1.x CLI, `auto` permission mode with managed or user settings - [Configure auto mode - Claude Code Docs](#)
- claude-hooks-permissions-20** (confirmed/high) `version-sensitive` - Entering auto mode drops broad allow rules that grant arbitrary code execution - blanket `Bash(*)` or `PowerShell(*)`, wildcarded interpreters such as `Bash(python*)`, package-manager run commands, and `Agent` allow rule ...
Applies to: Claude Code v2.1.x CLI, `auto` permission mode - [Choose a permission mode - Claude Code Docs](#)
- claude-hooks-permissions-21** (confirmed/high) `version-sensitive` - The auto mode classifier is reasoning-blind by design: it sees user messages, tool calls, and CLAUDE.md content, while Claude's own messages and all tool results are stripped, so hostile content arriving in a file or web ...
Applies to: Claude Code v2.1.x CLI, `auto` permission mode classifier - [Choose a permission mode - Claude Code Docs](#)
- claude-hooks-permissions-22** (confirmed/high) `version-sensitive` - If the auto mode classifier blocks an action 3 times in a row or 20 times total, auto mode pauses and Claude Code resumes prompting; these thresholds are not configurable. Any allowed action resets the consecutive counte ...
Applies to: Claude Code v2.1.x CLI, `auto` permission mode - [Choose a permission mode - Claude Code Docs](#)
- claude-hooks-permissions-23** (confirmed/high) `version-sensitive` - Auto mode block-lists are customised through four array-of-prose settings keys: `autoMode.environment`, `autoMode.allow`, `autoMode.soft_deny` and `autoMode.hard_deny`. Including the literal string `"$defaults"` in an ar ...
Applies to: Claude Code v2.1.x CLI, `autoMode` settings block - [Configure auto mode - Claude Code Docs](#)
- claude-hooks-permissions-24** (confirmed/high) `version-sensitive` - Inside the classifier, rule precedence works in four tiers: `hard_deny` blocks unconditionally and neither user intent nor `allow` exceptions apply; `soft_deny` blocks next; `allow` entries then override matching `soft_d ...
Applies to: Claude Code v2.1.x CLI, `autoMode` settings across user and managed scopes - [Configure auto mode - Claude Code Docs](#)

- **claude-hooks-permissions-25** (confirmed/high) **version-sensitive** - The classifier does not read the **autoMode** block from **.claude/settings.json** or **.claude/settings.local.json** - only from **~/.claude/settings.json**, managed settings, or inline JSON via the **--settings** flag or Agent S ...

Applies to: Claude Code v2.1.207+ (project-local exclusion), **autoMode** settings block - [Configure auto mode - Claude Code Docs](#)

- **claude-hooks-permissions-26** (confirmed/high) **version-sensitive** - Claude Code v2.1.142 and later ignore **permissions.defaultMode: "auto"** when it is set in **.claude/settings.json** or **.claude/settings.local.json**, so a repository cannot grant itself auto mode; the setting must live in ...

Applies to: Claude Code v2.1.142+, **permissions.defaultMode** setting - [Choose a permission mode - Claude Code Docs](#)

- **claude-hooks-permissions-27** (confirmed/high) **version-sensitive** - Four CLI subcommands inspect auto mode configuration: **claude auto-mode defaults** prints the built-in environment/allow/soft_deny/hard_deny rules as JSON, **claude auto-mode config** prints the effective configuration with ...

Applies to: Claude Code v2.1.x CLI; **reset** requires v2.1.212+, **--label** requires v2.1.208+ - [Configure auto mode - Claude Code Docs](#)

- **claude-hooks-permissions-28** (confirmed/high) **version-sensitive** - The **PermissionDenied** hook event fires when a tool call is denied by the auto mode classifier. It is not block-capable, and a handler can return **hookSpecificOutput** with **hookEventName: "PermissionDenied"** and **retry: ...**

Applies to: Claude Code v2.1.x CLI, **auto** permission mode with **PermissionDenied** hooks - [Hooks reference - Claude Code Docs](#)

- **claude-hooks-permissions-29** (confirmed/high) **version-sensitive** - Writes to protected paths are never auto-approved in any mode except **bypassPermissions**; they are prompted in **default**, **acceptEdits** and **plan**, routed to the classifier in **auto**, denied in **dontAsk**, and allowed in ...

Applies to: Claude Code v2.1.x CLI, all permission modes - [Choose a permission mode - Claude Code Docs](#)

- **claude-hooks-permissions-30** (confirmed/high) **version-sensitive** - **autoMode.classifyAllShell: true** suspends every Bash and PowerShell allow rule while auto mode is active so the classifier evaluates every shell command regardless of the allow list; the key requires Claude Code v2.1.19 ...

Applies to: Claude Code v2.1.193+, **autoMode.classifyAllShell** setting, **auto** permission mode only - [Configure auto mode - Claude Code Docs](#)

claude-context-memory (32)

- **claude-context-memory-01** (confirmed/high) **version-sensitive** - Claude Code loads only the first 200 lines of **MEMORY.md**, or the first 25KB, whichever comes first, at the start of every

conversation; content beyond that threshold is not loaded at session start.

Applies to: Claude Code v2.1.x auto memory (docs verified at v2.1.214) - [How Claude remembers your project - Claude Code Docs](#)

- **claude-context-memory-02** (confirmed/high) **version-sensitive** - Auto memory is stored per project at `~/.claude/projects/<project>/memory/` with a `MEMORY.md` entryptpoint plus optional topic files; the `<project>` path is derived from the git repository, so all worktrees and subdirectories ...

Applies to: Claude Code v2.1.x auto memory (docs verified at v2.1.214) - [How Claude remembers your project - Claude Code Docs](#)

- **claude-context-memory-03** (confirmed/high) **version-sensitive** - The auto memory directory can be relocated with the `settings.json` key `autoMemoryDirectory`, whose value must be an absolute path or start with `~/`; when set in a project's `.claude/settings.json` or `.claude/settings.local.js` ...

Applies to: Claude Code v2.1.x `settings.json` (docs verified at v2.1.214) - [How Claude remembers your project - Claude Code Docs](#)

- **claude-context-memory-04** (confirmed/high) **version-sensitive** - `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1` disables auto memory; setting it to 0 forces auto memory on even when `--bare` mode or `autoMemoryEnabled: false` would otherwise disable it.

Applies to: Claude Code v2.1.x environment variables (docs verified at v2.1.214) - [Environment variables - Claude Code Docs](#)

- **claude-context-memory-05** (confirmed/high) **version-sensitive** - `CLAUDE_AUTOCOMPACT_PCT_OVERRIDE` sets the percentage (1-100) of the auto-compaction window at which auto-compaction triggers; it can only lower the threshold (values above the default have no effect) and it applies to bot ...

Applies to: Claude Code v2.1.x environment variables (docs verified at v2.1.214) - [Environment variables - Claude Code Docs](#)

- **claude-context-memory-06** (confirmed/high) **version-sensitive** - `CLAUDE_CODE_AUTO_COMPACT_WINDOW` sets the context capacity in tokens used for auto-compaction calculations, defaults to the model's context window (200K standard, 1M extended-context, with Sonnet 5 having its own default) ...

Applies to: Claude Code v2.1.x environment variables (docs verified at v2.1.214) - [Environment variables - Claude Code Docs](#)

- **claude-context-memory-07** (confirmed/high) **version-sensitive** - On the Anthropic API, Sonnet 5 always runs with the 1M token context window (no 200K variant and no [1m] suffix to select) and its sessions auto-compact before the window fills, at about 967K tokens by default.

Applies to: Claude Code v2.1.x with Sonnet 5 on the Anthropic API (docs verified at v2.1.214) - [Model configuration - Claude Code Docs](#)

- **claude-context-memory-08** (corrected/high) **version-sensitive** - Auto-compaction timing is not uniform across models: Claude Code compacts proactively when `CLAUDE_CODE_AUTO_COMPACT_WINDOW` is set, in cloud sessions, on Sonnet 4.6 and Opus 4.6 without extended context (which compact at ...
Applies to: Claude Code v2.1.x, model-dependent (docs verified at v2.1.214) - [Environment variables](#) - [Claude Code Docs](#)
- **claude-context-memory-09** (confirmed/high) **version-sensitive** - After compaction, the project-root `CLAUDE.md`, unscoped rules, and auto memory are re-injected from disk, while the system prompt and output style are unchanged because they are not part of message history.
Applies to: Claude Code v2.1.x compaction (docs verified at v2.1.214) - [Explore the context window](#) - [Claude Code Docs](#)
- **claude-context-memory-10** (confirmed/high) **version-sensitive** - Rules carrying paths: frontmatter and nested `CLAUDE.md` files in subdirectories are lost at compaction and only reload the next time Claude reads a matching file in that path or subdirectory; removing the paths: frontmatt ...
Applies to: Claude Code v2.1.x compaction and `.claude/rules/` (docs verified at v2.1.214) - [Explore the context window](#) - [Claude Code Docs](#)
- **claude-context-memory-11** (confirmed/high) **version-sensitive** - Invoked skill bodies are re-injected after compaction but capped at 5,000 tokens per skill and 25,000 tokens total with the oldest invoked skills dropped first, and truncation keeps the start of the file.
Applies to: Claude Code v2.1.x skills and compaction (docs verified at v2.1.214) - [Explore the context window](#) - [Claude Code Docs](#)
- **claude-context-memory-12** (confirmed/high) **version-sensitive** - The startup skill-description listing is the one piece of startup content that is not re-injected after `/compact`; only skills that were actually invoked are preserved.
Applies to: Claude Code v2.1.x skills and compaction (docs verified at v2.1.214) - [Explore the context window](#) - [Claude Code Docs](#)
- **claude-context-memory-13** (confirmed/high) **version-sensitive** - As of Claude Code v2.1.198, the compaction summarization request inherits the session's extended thinking configuration, reasoning with thinking enabled when the session has it enabled and staying off otherwise; thinking ...
Applies to: Claude Code v2.1.198 and later - [Explore the context window](#) - [Claude Code Docs](#)
- **claude-context-memory-14** (confirmed/high) **version-sensitive** - When context fills up, Claude Code clears older tool outputs first and only then summarizes the conversation if still needed; requests and key code snippets are preserved while detailed instructions from early in the con ...
Applies to: Claude Code v2.1.x context management (docs verified at v2.1.214) - [How Claude Code works](#) - [Claude Code Docs](#)

- **claude-context-memory-15** (confirmed/high) **version-sensitive** - If a single file or tool output is so large that context refills immediately after each summary, Claude Code stops auto-compacting after a few attempts and shows an error instead of looping.

Applies to: Claude Code v2.1.x auto-compaction (docs verified at v2.1.214) - [How Claude Code works](#) - [Claude Code Docs](#)

- **claude-context-memory-16** (confirmed/high) **version-sensitive** - What compaction preserves can be steered two ways: a 'Compact Instructions' section added to CLAUDE.md, or running `/compact` with a focus argument such as `/compact focus on the API changes`; in a fresh session `/compact p ...`

Applies to: Claude Code v2.1.x `/compact` command and CLAUDE.md (docs verified at v2.1.214) - [How Claude Code works](#) - [Claude Code Docs](#)

- **claude-context-memory-17** (corrected/high) **version-sensitive** - Fable 5, Sonnet 5, Opus 4.6 and later, and Sonnet 4.6 support a 1 million token context window. For Opus and Sonnet 4.6 it is selected via the `[1m]` suffix on aliases or full model names (for example `/model opus[1m]` or `/m ...`

Applies to: Claude Code v2.1.x model configuration (docs verified at v2.1.214) - [Model configuration](#) - [Claude Code Docs](#)

- **claude-context-memory-18** (confirmed/high) **version-sensitive** - CLAUDE.md files are loaded in full regardless of length with no truncation limit (unlike MEMORY.md), but the documented target is under 200 lines per file because longer files consume more context and reduce adherence.

Applies to: Claude Code v2.1.x CLAUDE.md (docs verified at v2.1.214) - [How Claude remembers your project](#) - [Claude Code Docs](#)

- **claude-context-memory-19** (confirmed/high) **version-sensitive** - CLAUDE.md supports `@path/to/import` syntax whose files are expanded and loaded into context at launch, recursively to a maximum depth of four hops; import parsing skips Markdown code spans and fenced code blocks, so a bac ...

Applies to: Claude Code v2.1.x CLAUDE.md imports (docs verified at v2.1.214) - [How Claude remembers your project](#) - [Claude Code Docs](#)

- **claude-context-memory-20** (confirmed/high) **version-sensitive** - CLAUDE.md content is delivered as a user message after the system prompt, not as part of the system prompt itself, so there is no guarantee of strict compliance; for instructions that must run at a fixed lifecycle point ...

Applies to: Claude Code v2.1.x CLAUDE.md (docs verified at v2.1.214) - [How Claude remembers your project](#) - [Claude Code Docs](#)

- **claude-context-memory-21** (confirmed/high) **version-sensitive** - Claude Code discovers CLAUDE.md and CLAUDE.local.md by walking up the directory tree from the working directory and concatenates all discovered files rather than overriding, ordered from filesystem root down to the worki ...

Applies to: Claude Code v2.1.x CLAUDE.md resolution (docs verified at v2.1.214) - [How Claude remembers your project](#) - [Claude Code Docs](#)

- **claude-context-memory-22** (confirmed/high) **version-sensitive** - Block-level HTML comments in CLAUDE.md files are stripped before the content is injected into Claude's context, while comments inside code blocks are preserved and comments remain visible when the file is opened with the ...

Applies to: Claude Code v2.1.x CLAUDE.md (docs verified at v2.1.214) - [How Claude remembers your project - Claude Code Docs](#)

- **claude-context-memory-23** (corrected/medium) **version-sensitive** - As of Claude Code v2.1.210, after Claude writes to MEMORY.md the file is measured against the 200-line and 25KB read limits: a near-limit file triggers a reminder to shorten it, and an over-limit file still writes success ...

Applies to: Claude Code v2.1.210 / v2.1.211 and later - [How Claude remembers your project - Claude Code Docs](#)

- **claude-context-memory-24** (confirmed/high) **version-sensitive** - MCP tool schemas are deferred by default so only tool names consume context until Claude loads a specific schema via tool search; ENABLE_TOOL_SEARCH=auto loads schemas upfront when they fit within 10% of the context window ...

Applies to: Claude Code v2.1.x MCP tool search (docs verified at v2.1.214) - [Explore the context window - Claude Code Docs](#)

- **claude-context-memory-25** (corrected/high) **version-sensitive** - Subagents run in a separate context window that loads CLAUDE.md and the same MCP and skill setup but not the main conversation's history or auto memory; the built-in Explore and Plan agents are an exception and skip CLAU ...

Applies to: Claude Code v2.1.x subagents (docs verified at v2.1.214) - [Explore the context window - Claude Code Docs](#)

- **claude-context-memory-26** (confirmed/high) **version-sensitive** - The claudeMdExcludes setting skips specific CLAUDE.md files by absolute-path glob, can be configured at user, project, local, or managed-policy settings layers with arrays merging across layers, but managed-policy CLAUDE ...

Applies to: Claude Code v2.1.x settings.json (docs verified at v2.1.214) - [How Claude remembers your project - Claude Code Docs](#)

- **claude-context-memory-27** (confirmed/high) **version-sensitive** - On the Claude Developer Platform, context editing is enabled with the beta header context-management-2025-06-27 and offers the clear_tool_uses_20250919 strategy (defaults: trigger at 100,000 input tokens, keep the 3 most ...

Applies to: Claude Developer Platform API, context-management-2025-06-27 beta (not Claude Code CLI) - [Context editing - Claude Platform Docs](#)

- **claude-context-memory-28** (confirmed/high) - Anthropic reports that on an internal agentic-search evaluation, combining the memory tool with context editing improved performance by 39%

over baseline and context editing alone delivered a 29% improvement, and that in ...

Applies to: Claude Developer Platform context editing + memory tool, vendor internal evaluation (published 2025-09-29) - [Managing context on the Claude Developer Platform](#)

- **claude-context-memory-29** (confirmed/medium) - Anthropic's context engineering guidance defines context engineering as 'curating and maintaining the optimal set of tokens during LLM inference' and asserts context rot: as the number of tokens in the context window inc ...

Applies to: Vendor-agnostic agent design guidance (Anthropic engineering, published 2025-09-29) - [Effective context engineering for AI agents - Anthropic Engineering](#)

- **claude-context-memory-30** (confirmed/high) **version-sensitive** - Anthropic launched memory for Claude Managed Agents in public beta on 2026-04-23, storing memories as files on a mounted filesystem that developers can export, edit and manage via API or the Claude Console, with per-sess ...

Applies to: Claude Managed Agents on the Claude Platform, public beta as of 2026-04-23 - [Built-in memory for Claude Managed Agents](#)

- **claude-context-memory-31** (confirmed/high) **version-sensitive** - Claude Code exposes /context for a live per-category breakdown of actual context usage including which CLAUDE.md and auto memory files loaded, /memory to browse and edit memory file locations and toggle auto memory, and ...

Applies to: Claude Code v2.1.x commands and hooks (docs verified at v2.1.214) - [Explore the context window - Claude Code Docs](#)

- **claude-context-memory-32** (confirmed/high) **version-sensitive** - Anthropic documents that agent teams use approximately 7x more tokens than standard sessions when teammates run in plan mode, because each teammate maintains its own context window as a separate Claude instance, and agen ...

Applies to: Claude Code v2.1.x agent teams, experimental (docs verified at v2.1.214) - [Manage costs effectively - Claude Code Docs](#)

claude-agents-skills (31)

- **claude-agents-skills-01** (confirmed/high) **version-sensitive** - Agent teams in Claude Code are experimental and disabled by default; they are enabled only by setting the environment variable CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS to 1, either in the shell environment or in the "env" bl ...

Applies to: Claude Code v2.1.178+ (doc describes behavior as of v2.1.178); CLI, all platforms - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-02** (confirmed/high) **version-sensitive** - Anthropic's docs draw the subagent/agent-team line on communication topology: subagents each get their own context window but report results back to the calling agent only and never message each other, while agent-team t ...

Applies to: Claude Code v2.1.178+ (agent teams experimental); vendor-specific - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-03** (confirmed/high) **version-sensitive** - Anthropic's cost documentation states that agent teams use approximately 7x more tokens than standard sessions when teammates run in plan mode, because each teammate maintains its own context window and runs as a separat ...

Applies to: Claude Code v2.1.x with CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1; figure is stated for teammates running in plan mode - [Manage costs effectively - Claude Code Docs](#)

- **claude-agents-skills-04** (confirmed/high) - Anthropic's agent-teams docs state there is no hard limit on teammate count but recommend starting with 3-5 teammates and 5-6 tasks per teammate, giving 15 independent tasks / 3 teammates as the worked example, and asser ...

Applies to: Claude Code v2.1.178+ agent teams; guidance, not an enforced limit - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-05** (confirmed/high) - Anthropic's own docs name the counter-indications for agent teams: "For sequential tasks, same-file edits, or work with many dependencies, a single session or subagents are more effective," and separately warn that "Two ...

Applies to: Claude Code v2.1.178+ agent teams; vendor-agnostic in spirit - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-06** (confirmed/high) **version-sensitive** - A spawned teammate loads the same project context as a regular session (CLAUDE.md, MCP servers, and skills) plus the lead's spawn prompt, but the lead's conversation history does not carry over. Teammates read CLAUDE.md ...

Applies to: Claude Code v2.1.178+ agent teams - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-07** (confirmed/high) **version-sensitive** - Anthropic documents an explicit agent-to-agent trust boundary for agent teams: teammates start with the lead's permission settings and inherit --dangerously-skip-permissions if the lead used it; a teammate cannot approve ...

Applies to: Claude Code v2.1.178+ agent teams; security-relevant - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-08** (confirmed/high) **version-sensitive** - Agent team state is stored locally under a session-derived name of the form "session-" plus the first eight characters of the session ID: team config at ~/.claude/teams/{team-name}/config.json, task list at ~/.claude/tas ...

Applies to: Claude Code v2.1.178+ agent teams - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-09** (confirmed/high) **version-sensitive** - Agent team display mode is controlled by the teammateMode setting in ~/.claude/settings.json or the experimental --

teammate-mode CLI flag, with accepted values "in-process", "auto", "tmux", and (as of v2.1.186) "iterm2". ...

Applies to: Claude Code v2.1.179+ for the in-process default; v2.1.186+ for "iterm2" - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-10** (confirmed/high) **version-sensitive** - Three hook events exist for enforcing quality gates on agent teams: TeammateIdle (runs when a teammate is about to go idle), TaskCreated (runs when a task is being created), and TaskCompleted (runs when a task is being m ...

Applies to: Claude Code v2.1.178+ agent teams with hooks configured in settings.json - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-11** (confirmed/high) **version-sensitive** - Documented agent-team limitations include: /resume and /rewind do not restore in-process teammates; exactly one team per session with no additional named teams and no sharing across sessions; no nested teams (teammates c ...

Applies to: Claude Code v2.1.178+ agent teams (experimental) - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-12** (confirmed/high) **version-sensitive** - A teammate can be spawned from an existing subagent definition of any scope (project, user, plugin, or CLI-defined). On that path the teammate honors the definition's tools allowlist and model, and the definition's body ...

Applies to: Claude Code v2.1.178+ agent teams - [Orchestrate teams of Claude Code sessions - Claude Code Docs](#)

- **claude-agents-skills-13** (confirmed/high) **version-sensitive** - Claude Code v2.1.212 (released 2026-07-17) added a per-session cap on subagent spawns, defaulting to 200 and overridable with the CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION environment variable, to stop runaway delegation loo ...

Applies to: Claude Code v2.1.212+ - [Claude Code changelog - Claude Code Docs](#)

- **claude-agents-skills-14** (confirmed/high) **version-sensitive** - As of Claude Code v2.1.198 subagents run in the background by default, with Claude running one in the foreground only when it needs the result before continuing; before v2.1.198 Claude chose per task. The frontmatter fie ...

Applies to: Claude Code v2.1.198+ for the default change; env vars vendor-specific - [Create custom subagents - Claude Code Docs](#)

- **claude-agents-skills-15** (confirmed/high) **version-sensitive** - Subagents are Markdown files with YAML frontmatter stored in .claude/agents/ or ~/.claude/agents/. Only name and description are required. The full optional field set is: tools, disallowedTools, model, permissionMode, ma ...

Applies to: Claude Code v2.1.x (field set current as of v2.1.212-era docs) - [Create custom subagents - Claude Code Docs](#)

- **claude-agents-skills-16** (confirmed/high) **version-sensitive** - The subagent frontmatter field isolation: worktree runs the subagent in a temporary git worktree, giving it an isolated copy of the

repository branched by default from the repository's default branch rather than the pare ...

Applies to: Claude Code v2.1.x; v2.1.210+ for the containment fix - [Create custom subagents - Claude Code Docs](#)

- **claude-agents-skills-17** (confirmed/high) **version-sensitive** - Subagent name collisions resolve by scope priority, with project .claude/agents/ (priority 3) outranking user ~/.claude/agents/ (priority 4), and managed subagents deployed by administrators in the managed settings direc ...

Applies to: Claude Code v2.1.178+ for nearest-wins nested resolution - [Create custom subagents - Claude Code Docs](#)

- **claude-agents-skills-18** (confirmed/high) **version-sensitive** - For security reasons plugin subagents do not support the hooks, mcpServers, or permissionMode frontmatter fields; these fields are ignored when an agent is loaded from a plugin. To use them the agent file must be copied ...

Applies to: Claude Code v2.1.x; security-relevant - [Create custom subagents - Claude Code Docs](#)

- **claude-agents-skills-19** (confirmed/high) **version-sensitive** - As of Claude Code v2.1.172 a subagent can spawn its own subagents, so a delegated task that splits into parallel subtasks keeps its intermediate output out of the main conversation; only the top-level subagent's summary ...

Applies to: Claude Code v2.1.172+ - [Create custom subagents - Claude Code Docs](#)

- **claude-agents-skills-20** (confirmed/high) - Anthropic's subagent docs warn that subagent fan-out has a context cost at the join point: "When subagents complete, their results return to your main conversation. Running many subagents that each return detailed result ...

Applies to: Claude Code v2.1.x; vendor-specific but architecture-general - [Create custom subagents - Claude Code Docs](#)

- **claude-agents-skills-21** (confirmed/high) **version-sensitive** - In a Claude Code skill's SKILL.md frontmatter, the combined description and when_to_use text is truncated at 1,536 characters in the skill listing to reduce context usage. The docs instruct authors to put the key use cas ...

Applies to: Claude Code v2.1.x skills; also relevant to the agentskills.io Agent Skills standard that Claude Code follows - [Extend Claude with skills - Claude Code Docs](#)

- **claude-agents-skills-22** (confirmed/high) **version-sensitive** - When a skill is invoked, the rendered SKILL.md content enters the conversation as a single message and stays there for the rest of the session; Claude Code does not re-read the skill file on later turns. The persistence ...

Applies to: Claude Code v2.1.x skills - [Extend Claude with skills - Claude Code Docs](#)

- **claude-agents-skills-23** (confirmed/high) **version-sensitive** - Adding context: fork to a skill's frontmatter runs that skill in its own subagent context, with the skill content becoming the prompt that drives the subagent; the forked skill has no access to the conversation history. ...

Applies to: Claude Code v2.1.x; context: fork and subagent skills: are Claude Code extensions to the Agent Skills standard - [Extend Claude with skills - Claude Code Docs](#)

- **claude-agents-skills-24** (confirmed/high) **version-sensitive** - Setting `disable-model-invocation: true` in a skill's frontmatter removes the skill's description from Claude's context entirely so only the user can invoke it with `/name`; it also prevents the skill from being preloaded in ...
Applies to: Claude Code v2.1.196+ for the scheduled-task behavior; field itself available earlier - [Extend Claude with skills - Claude Code Docs](#)
- **claude-agents-skills-25** (confirmed/high) - Anthropic's cost documentation recommends moving specialized instructions out of `CLAUDE.md` into skills because `CLAUDE.md` is loaded into context at session start whereas skills load on demand only when invoked, and advise ...
Applies to: Claude Code v2.1.x; vendor-specific - [Manage costs effectively - Claude Code Docs](#)
- **claude-agents-skills-26** (confirmed/high) - Anthropic's engineering write-up on internal Skills use states that skills are folders of instructions, scripts, and resources rather than merely markdown documents, that "The description field is not a summary, it's a d ...
Applies to: Vendor-agnostic authoring guidance; reflects Anthropic internal practice, no version dependency - [Lessons from building Claude Code: how we use Skills](#)
- **claude-agents-skills-27** (confirmed/high) - Cognition's "Don't Build Multi-Agents" argues against multi-agent architectures on two stated principles — "Share context, and share full agent traces, not just individual messages" and "Actions carry implicit decisions, ...
Applies to: Vendor-agnostic architectural argument; published mid-2025, predates Claude Code agent teams - [Don't Build Multi-Agents - Walden Yan, Cognition](#)
- **claude-agents-skills-28** (confirmed/high) - A Google Research study evaluated 180 agent configurations across 5 canonical architectures (single-agent, independent, centralized, decentralized, hybrid), 4 benchmarks (Finance-Agent, BrowseComp-Plus, PlanCraft, Workbe ...
Applies to: Vendor-agnostic; published 2026-01-28; note that none of the four benchmarks is a software-engineering benchmark - [Towards a science of scaling agent systems: When and why agent systems work - Google Research](#)
- **claude-agents-skills-29** (confirmed/high) - The same Google Research study reports a task-shape dependence: on the parallelizable Finance-Agent benchmark, centralized multi-agent coordination improved results by +80.9%, while on the sequential PlanCraft benchmark ...
Applies to: Vendor-agnostic; non-coding benchmarks; published 2026-01-28 - [Towards a science of scaling agent systems: When and why agent systems work - Google Research](#)
- **claude-agents-skills-30** (confirmed/medium) - The Google Research study fits a predictive model ($R^2 = 0.513$) that selects the optimal coordination architecture from measurable task properties — tool count, decomposability, sequential dependencies, and tool density ...
Applies to: Vendor-agnostic; published 2026-01-28; $R^2 = 0.513$ indicates roughly half of variance unexplained - [Towards a science of scaling agent systems: When and why agent systems work - Google Research](#)

- **claude-agents-skills-31** (confirmed/high) **version-sensitive** - Claude Code v2.1.215 changed skill auto-invocation so that Claude no longer runs the `/verify` and `/code-review` skills on its own; they must be invoked explicitly with `/verify` or `/code-review`.

Applies to: Claude Code v2.1.215+ - [anthropics/claude-code CHANGELOG.md](#)

codex-config-hooks (24)

- **codex-config-hooks-01** (confirmed/high) **version-sensitive** - The Codex documentation host has moved: URLs under `developers.openai.com/codex/` now return `HTTP 308 Permanent Redirect to learn.chatgpt.com/docs/`. Specifically `developers.openai.com/codex/hooks` -> `learn.chatgpt.com/doc` ...

Applies to: Codex documentation site, observed 2026-07-20 - [developers.openai.com/codex/hooks](#) (308 redirect observed live)

- **codex-config-hooks-02** (confirmed/high) **version-sensitive** - Codex hooks define exactly ten lifecycle events: `SessionStart`, `SubagentStart`, `PreToolUse`, `PermissionRequest`, `PostToolUse`, `PreCompact`, `PostCompact`, `UserPromptSubmit`, `SubagentStop`, and `Stop`.

Applies to: Codex CLI, docs current as of 2026-07-20 (latest stable 0.144.6) - [Codex Hooks](#)

- **codex-config-hooks-03** (confirmed/high) **version-sensitive** - Codex hook events split by scope: `SessionStart` and `SubagentStart` run at thread or subagent-start scope, while `PreToolUse`, `PermissionRequest`, `PostToolUse`, `PreCompact`, `PostCompact`, `UserPromptSubmit`, `SubagentStop`, and `Stop` ...

Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Hooks](#)

- **codex-config-hooks-04** (confirmed/high) **version-sensitive** - Only handler type "command" actually executes in Codex hooks; the documented types "prompt" and "agent" are parsed but skipped. The docs state: "Only type: \"command\" handlers run today. prompt and agent handlers are pa ..."

Applies to: Codex CLI, docs current as of 2026-07-20 (latest stable 0.144.6) - [Codex Hooks](#)

- **codex-config-hooks-05** (confirmed/high) **version-sensitive** - If the hook **timeout** field is omitted, Codex applies a default timeout of 600 seconds.

Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Hooks](#)

- **codex-config-hooks-06** (corrected/high) **version-sensitive** - A Codex hook signals a decision by exiting with code 2 and writing a reason to `stderr`. This path is documented for `PreToolUse`, `PostToolUse`, `UserPromptSubmit`, `SubagentStop`, and `Stop`, but the meaning of the `stderr` text dif ...

Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Hooks](#)

- **codex-config-hooks-07** (confirmed/high) **version-sensitive** - Codex records hook trust against the hook's current hash: "Codex records trust against the hook's current hash, so new or changed hooks are marked for review and skipped until trusted."

Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Hooks](#)

- **codex-config-hooks-08** (confirmed/high) **version-sensitive** - The CLI flag `--dangerously-bypass-hook-trust` runs enabled hooks without requiring persisted hook trust for that invocation.
Applies to: Codex CLI, docs current as of 2026-07-20 (latest stable 0.144.6) - [Codex Hooks](#)
- **codex-config-hooks-09** (confirmed/high) **version-sensitive** - Hooks originating from system, MDM, cloud, or requirements.toml sources are marked as managed, are trusted by policy, and cannot be disabled from the user hook browser.
Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Hooks](#)
- **codex-config-hooks-10** (confirmed/high) **version-sensitive** - Codex provides a `/hooks` TUI command: "Use `/hooks` in the CLI to inspect hook sources, review new or changed hooks, trust hooks, or disable individual non-managed hooks."
Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Hooks](#)
- **codex-config-hooks-11** (confirmed/high) **version-sensitive** - Codex discovers hooks in two interchangeable forms next to active config layers: a `hooks.json` file, or inline `[hooks]` tables inside `config.toml`. The Configuration Reference describes the `[hooks]` table as "Lifecycle hooks ..."
Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Hooks](#)
- **codex-config-hooks-12** (confirmed/high) **version-sensitive** - Codex hook definitions in TOML use the fields: `matcher`, `type`, `command`, `command_windows`, `timeout`, and `statusMessage`, arranged as `[[hooks.EventName]]` with a `matcher`, containing `[[hooks.EventName.hooks]]` handler entries.
Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Hooks](#)
- **codex-config-hooks-13** (confirmed/high) **version-sensitive** - Codex limits each model-visible hook-output message to roughly 2,500 tokens; output exceeding that is saved to disk and the model is shown a preview.
Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Hooks](#)
- **codex-config-hooks-14** (confirmed/high) **version-sensitive** - The config key `project_doc_max_bytes` controls the byte limit applied when building project instructions from `AGENTS.md`, and defaults to 32 KiB.
Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex AGENTS.md](#)
- **codex-config-hooks-15** (confirmed/high) **version-sensitive** - Codex `AGENTS.md` discovery starts at global scope (`~/.codex/AGENTS.override.md` if present, otherwise `~/.codex/AGENTS.md`), then walks from the project root -- typically the Git root -- down to the current working directory ...
Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex AGENTS.md](#)
- **codex-config-hooks-16** (confirmed/high) **version-sensitive** - Codex concatenates discovered `AGENTS.md` files from root downward separated by blank lines, and files closer to the current directory override earlier guidance because they appear later in the combined prompt.
Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex AGENTS.md](#)

- **codex-config-hooks-17** (confirmed/high) **version-sensitive** - When accumulating AGENTS.md content, Codex skips empty files and stops adding files once the combined size reaches the limit defined by project_doc_max_bytes.

Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex AGENTS.md](#)

- **codex-config-hooks-18** (confirmed/high) **version-sensitive** - The config key project_doc_fallback_filenames (array of strings) specifies additional filenames Codex tries when AGENTS.md is missing.

Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Configuration Reference](#)

- **codex-config-hooks-19** (confirmed/high) **version-sensitive** - The config key project_root_markers (array of strings) sets which filenames or directories mark a project root when searching parent directories, defaulting to [".git"].

Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Advanced Configuration](#)

- **codex-config-hooks-20** (confirmed/high) **version-sensitive** - Codex loads project-scoped config files (.codex/config.toml inside a project) only when the user has trusted the project. User-level configuration lives at ~/.codex/config.toml.

Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Configuration Reference](#)

- **codex-config-hooks-21** (confirmed/high) **version-sensitive** - Project-scoped Codex config cannot override machine-local provider, auth, host-owned app request metadata, notification, profile-selection, or telemetry-routing keys. Codex specifically ignores openai_base_url, chatgpt_b ...

Applies to: Codex CLI, docs current as of 2026-07-20 - [Codex Configuration Reference](#)

- **codex-config-hooks-22** (confirmed/high) **version-sensitive** - As of Codex 0.134.0, profile loading changed: legacy [profiles.name] tables and the profile = "name" selector moved to separate ~/.codex/profile-name.config.toml files.

Applies to: Codex CLI 0.134.0 and later - [Codex Advanced Configuration](#)

- **codex-config-hooks-23** (confirmed/high) **version-sensitive** - The latest stable Codex CLI release as of 2026-07-20 is 0.144.6, published 2026-07-18; a 0.145.0 prerelease line is active, with 0.145.0-alpha.24 published 2026-07-18.

Applies to: Codex CLI, observed 2026-07-20 - [openai/codex Releases](#)

- **codex-config-hooks-24** (confirmed/medium) **version-sensitive** - An open issue on the official openai/codex repository (#20204, opened 2026-04-29) reports that PreToolUse hook events are emitted by only four tool categories -- shell/Bash, unified_exec, apply_patch, and mcp -- while ot ...

Applies to: Codex CLI; report filed against a pre-0.144 build, current status against 0.144.6 unconfirmed - [openai/codex issue #20204: Inconsistent PreToolUse hook coverage across tool handlers \(most tools never emit hook events\)](#)

codex-exec-automation (42)

- **codex-exec-automation-01** (confirmed/high) **version-sensitive** - `codex exec` is the Codex CLI non-interactive entry point; it streams progress to stderr and prints the final agent message to stdout, so stdout can be piped without capturing progress noise.

Applies to: Codex CLI 0.144.x (behaviour documented without version qualifier) - [Non-interactive mode | ChatGPT Learn](#)

- **codex-exec-automation-02** (confirmed/high) **version-sensitive** - `codex exec --json` emits a JSON Lines event stream on stdout whose documented event types include `thread.started`, `turn.started`, `turn.completed`, `turn.failed`, `item.*` and `error`.

Applies to: Codex CLI 0.144.x - [Non-interactive mode | ChatGPT Learn](#)

- **codex-exec-automation-03** (corrected/high) **version-sensitive** - The `turn.completed` JSONL event carries a `usage` object with four fields: `input_tokens`, `cached_input_tokens`, `output_tokens` and `reasoning_output_tokens`. The documented sample event is `{"type":"turn.completed"," ...`

Applies to: Codex CLI 0.144.x - [Non-interactive mode | ChatGPT Learn](#)

- **codex-exec-automation-04** (confirmed/high) **version-sensitive** - `codex exec --output-schema <path>` requests a final response conforming to a supplied JSON Schema.

Applies to: Codex CLI 0.144.x - [Non-interactive mode | ChatGPT Learn](#)

- **codex-exec-automation-05** (corrected/high) **version-sensitive** - `codex exec -o <path>` (long form `--output-last-message <path>`) writes the final agent message to a file — and still prints that message to stdout as well. It filters the transcript down to the final message; it does n ...

Applies to: Codex CLI 0.144.x - [Non-interactive mode | ChatGPT Learn](#)

- **codex-exec-automation-06** (confirmed/high) **version-sensitive** - `codex exec --ephemeral` runs a task without persisting session files to disk.

Applies to: Codex CLI 0.144.x - [Non-interactive mode | ChatGPT Learn](#)

- **codex-exec-automation-07** (corrected/high) **version-sensitive** - Headless runs can be continued with `codex exec resume --last "<task>"` (the tier-1 example is a two-stage pipeline: `codex exec "review the change for race conditions"` then `codex exec resume --last "fix the race condi ...`

Applies to: Codex CLI 0.144.x - [Non-interactive mode | ChatGPT Learn](#)

- **codex-exec-automation-08** (confirmed/high) **version-sensitive** - `codex exec` requires the working directory to be a Git repository by default, and `--skip-git-repo-check` overrides that requirement.

Applies to: Codex CLI 0.144.x - [Non-interactive mode | ChatGPT Learn](#)

- **codex-exec-automation-09** (confirmed/high) **version-sensitive** - In `codex exec`, when stdin is piped AND a prompt argument is supplied, the prompt argument is treated as the instruction and

the piped content as context; `codex exec -` reads the entire prompt from stdin.

Applies to: Codex CLI 0.144.x - [Non-interactive mode](#) | [ChatGPT Learn](#)

- **codex-exec-automation-10** (corrected/high) `version-sensitive` - `CODEX_API_KEY` can supply authentication for a single headless run (e.g. `CODEX_API_KEY=<key> codex exec --json "<task>"`), and it is accepted only by `codex exec`. The documented warning is scoped to CI jobs rather than ...

Applies to: Codex CLI 0.144.x - [Non-interactive mode](#) | [ChatGPT Learn](#)

- **codex-exec-automation-11** (confirmed/high) `version-sensitive` - `codex exec` supports `--ignore-user-config` to skip `$CODEX_HOME/config.toml` and `--ignore-rules` to skip user and project rule files.

Applies to: Codex CLI 0.144.x - [Non-interactive mode](#) | [ChatGPT Learn](#)

- **codex-exec-automation-12** (confirmed/high) `version-sensitive` - `--full-auto` is a deprecated compatibility flag on `codex exec` that prints a warning.

Applies to: Codex CLI 0.144.x - [Non-interactive mode](#) | [ChatGPT Learn](#)

- **codex-exec-automation-13** (confirmed/high) `version-sensitive` - The Codex sandbox setting accepts `read-only`, `workspace-write` and `danger-full-access`, exposed as `--sandbox` / `-s`.

Applies to: Codex CLI 0.144.x - [Developer commands](#) | [ChatGPT Learn](#)

- **codex-exec-automation-14** (confirmed/high) `version-sensitive` - An MCP server configured with `required = true` causes `codex exec` to exit with an error if that server fails to initialize.

Applies to: Codex CLI 0.144.x - [Non-interactive mode](#) | [ChatGPT Learn](#)

- **codex-exec-automation-16** (confirmed/high) `version-sensitive` - Codex scheduled tasks (Automations) run with `approval_policy = "never"` when organization policy allows it.

Applies to: Codex desktop/web app, July 2026 - [Scheduled tasks](#) | [ChatGPT Learn](#)

- **codex-exec-automation-17** (confirmed/high) `version-sensitive` - The Codex "Scheduled" view acts as the review-queue inbox: runs with findings appear there, an unread indicator marks runs needing attention, and the view supports "Mark all as read" plus filtering by All / Active / Paus ...

Applies to: Codex desktop/web app, July 2026 - [Scheduled tasks](#) | [ChatGPT Learn](#)

- **codex-exec-automation-18** (confirmed/high) `version-sensitive` - For Git repositories, Codex scheduled tasks execute either in dedicated background worktrees or locally in the main checkout; frequent schedules on worktree mode accumulate many worktrees over time.

Applies to: Codex desktop/web app, July 2026 - [Scheduled tasks](#) | [ChatGPT Learn](#)

- **codex-exec-automation-19** (confirmed/high) - Codex docs advise archiving scheduled runs that are no longer needed and avoiding pinning runs unless their worktrees are meant to be retained.

Applies to: Codex desktop/web app, July 2026 - [Scheduled tasks](#) | [ChatGPT Learn](#)

- **codex-exec-automation-20** (confirmed/high) **version-sensitive** - Codex scheduled tasks support custom recurrence via RFC 5545 rules, e.g.

`RRULE:FREQ=MONTHLY;BYMONTHDAY=1;BYHOUR=9;BYMINUTE=0`.

Applies to: Codex desktop/web app, July 2026 - [Scheduled tasks | ChatGPT Learn](#)

- **codex-exec-automation-21** (confirmed/high) **version-sensitive** - Codex scheduled tasks come in two deployment models: standalone tasks that start a new chat for each scheduled run, and in-chat tasks that return to an existing chat and preserve context.

Applies to: Codex desktop/web app, July 2026 - [Scheduled tasks | ChatGPT Learn](#)

- **codex-exec-automation-22** (confirmed/high) **version-sensitive** - The Codex CLI has no scheduling UI; scheduled tasks must be created and managed from ChatGPT web or the desktop app.

Applies to: Codex CLI 0.144.x - [Scheduled tasks | ChatGPT Learn](#)

- **codex-exec-automation-23** (confirmed/medium) **version-sensitive** - Codex CLI provides `codex archive <thread>` and `codex unarchive <thread>` commands that resolve either UUIDs or exact thread names and call `thread/archive` / `thread/unarchive` RPCs.

Applies to: Codex CLI $\geq 0.136.0$ - [Add thread archive CLI commands by etraut-openai · Pull Request #25021 · openai/codex](#)

- **codex-exec-automation-24** (confirmed/high) **version-sensitive** - `codex resume <thread id>` and `codex fork <thread id>` reject archived sessions and emit an error instructing the user to run `codex unarchive <thread id>` first.

Applies to: Codex CLI $\geq 0.136.0$ - [Add thread archive CLI commands by etraut-openai · Pull Request #25021 · openai/codex](#)

- **codex-exec-automation-25** (confirmed/high) **version-sensitive** - The thread archive CLI commands were added in openai/codex PR #25021, merged 30 May 2026, touching `codex-rs/cli/src/main.rs`, `codex-rs/tui/src/session_archive_commands.rs` and `codex-rs/app-server/src/request_processor ...`

Applies to: openai/codex repository, May 2026 - [Add thread archive CLI commands by etraut-openai · Pull Request #25021 · openai/codex](#)

- **codex-exec-automation-26** (confirmed/high) **version-sensitive** - Codex custom prompts are deprecated in favour of skills for reusable instructions.

Applies to: Codex CLI 0.144.x - [Custom prompts | ChatGPT Learn](#)

- **codex-exec-automation-27** (confirmed/high) **version-sensitive** - Codex custom prompts are top-level Markdown files in `~/.codex/prompts/` (subdirectories are ignored), invoked as `/prompts:<filename>` without the `.md` extension.

Applies to: Codex CLI 0.144.x (deprecated feature) - [Custom prompts | ChatGPT Learn](#)

- **codex-exec-automation-28** (confirmed/high) **version-sensitive** - Custom prompt files use YAML front matter with `description` and `argument-hint`, and support `$1` - `$9` positional

placeholders, named uppercase placeholders supplied as `KEY=value`, `$ARGUMENTS` for all arguments, and `...

Applies to: Codex CLI 0.144.x (deprecated feature) - [Custom prompts | ChatGPT Learn](#)

- **codex-exec-automation-29** (confirmed/high) `version-sensitive` - Codex skills are discovered in `.agents/skills` folders scanned from the current working directory upward to the repository root, plus `$HOME/.agents/skills` for user scope and `/etc/codex/skills` for system scope.

Applies to: Codex CLI 0.144.x - [Build skills | ChatGPT Learn](#)

- **codex-exec-automation-30** (confirmed/high) `version-sensitive` - A Codex skill requires a `SKILL.md` file with YAML front matter containing at minimum `name` and `description`, and may include optional `scripts/`, `references/`, `assets/` directories and an `agents/openai.yaml` file.

Applies to: Codex CLI 0.144.x - [Build skills | ChatGPT Learn](#)

- **codex-exec-automation-31** (confirmed/high) `version-sensitive` - Codex skills expose `policy.allow_implicit_invocation` in `agents/openai.yaml`, defaulting to `true`, which controls whether Codex may select the skill autonomously.

Applies to: Codex CLI 0.144.x - [Build skills | ChatGPT Learn](#)

- **codex-exec-automation-32** (corrected/high) `version-sensitive` - When Codex builds the initial skills list, that list is budgeted to at most 2% of the model's context window, or 8,000 characters when the context window is unknown — the 8,000-character figure is a fallback for the unknown ...

Applies to: Codex CLI 0.144.x - [Build skills | ChatGPT Learn](#)

- **codex-exec-automation-33** (confirmed/medium) `version-sensitive` - Codex CLI configuration precedence is: command-line `-c key=value` overrides highest, then `--profile` layers, then base `~/.codex/config.toml` defaults.

Applies to: Codex CLI 0.144.x - [Developer commands | ChatGPT Learn](#)

- **codex-exec-automation-34** (confirmed/high) - The Codex long-horizon guidance recommends four durable project-memory Markdown files: `Prompt.md` (goals, non-goals, hard constraints), `Plan.md` (milestones sized for one loop plus acceptance criteria and validation commands ...

Applies to: vendor-agnostic pattern, published by OpenAI 2026-02-23 - [Run long-horizon tasks with Codex](#)

- **codex-exec-automation-35** (confirmed/high) `version-sensitive` - OpenAI reports a Codex long-horizon demonstration run lasting approximately 25 hours uninterrupted, consuming about 13M tokens and producing about 30k lines of code, using GPT-5.3-Codex at "Extra High" reasoning.

Applies to: GPT-5.3-Codex, reported 2026-02-23 - [Run long-horizon tasks with Codex](#)

- **codex-exec-automation-36** (confirmed/high) - The long-horizon run executed verification commands after each milestone and repaired failures before continuing, with quality commands spanning lint, typecheck, tests, build and export validation.

Applies to: vendor-agnostic pattern, published by OpenAI 2026-02-23 - [Run long-horizon tasks with Codex](#)

- **codex-exec-automation-37** (confirmed/high) **version-sensitive** - Codex added native planning surfaced through the `/plan` slash command, which breaks larger tasks into reviewable steps before execution.

Applies to: Codex CLI 0.144.x - [Run long-horizon tasks with Codex](#)

- **codex-exec-automation-38** (confirmed/high) **version-sensitive** - The official GitHub Action is `openai/codex-action@v1`; it requires an `openai-api-key` input, exposes `prompt` / `prompt-file`, `permission-profile`, `safety-strategy`, `responses-api-endpoint` and `codex-version` inputs, ...

Applies to: openai/codex-action v1, July 2026 - [openai/codex-action](#)

- **codex-exec-automation-39** (confirmed/high) **version-sensitive** - `openai/codex-action` supports four safety strategies — `drop-sudo` (default, revokes the default user's sudo membership on Linux and macOS runners), `unprivileged-user`, `read-only`, and `unsafe` (required on Windows).

Applies to: openai/codex-action v1, July 2026 - [openai/codex-action](#)

- **codex-exec-automation-40** (confirmed/high) **version-sensitive** - Codex CLI 0.144.6, released 18 July 2026, refreshed bundled instructions for GPT-5.6 Sol, Terra and Luna and corrected their context windows to 272,000 tokens.

Applies to: Codex CLI 0.144.6 - [Codex changelog](#) | [ChatGPT Learn](#)

- **codex-exec-automation-41** (confirmed/high) **version-sensitive** - Codex CLI 0.144.5, released 16 July 2026, improved dangerous-command detection including more forced `rm` forms and provides clearer rejection reasons when commands are denied.

Applies to: Codex CLI 0.144.5 - [Codex changelog](#) | [ChatGPT Learn](#)

- **codex-exec-automation-42** (confirmed/high) **version-sensitive** - Codex CLI 0.144.0, released 9 July 2026, added a `writes` app-approval mode that allows declared read-only actions while prompting for writes.

Applies to: Codex CLI >= 0.144.0 - [Codex changelog](#) | [ChatGPT Learn](#)

- **codex-exec-automation-44** (confirmed/high) **version-sensitive** - The page registered as the Codex "developer commands" reference documents built-in slash commands (`/model`, `/permissions`, `/plan`, `/review`, `/status`, `/init`, `/goal`, `/feedback`) and CLI subcommands, and does not ...

Applies to: learn.chatgpt.com Codex docs as of 2026-07-20 - [Developer commands](#) | [ChatGPT Learn](#)

codex-subagents-sandbox (26)

- **codex-subagents-sandbox-01** (confirmed/high) **version-sensitive** - Codex subagent concurrency is capped by `agents.max_threads` in config.toml, which defaults to 6 open agent

threads.

Applies to: Codex CLI 0.144.x (config.toml `[agents]` table) - [Configuration Reference | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-02** (confirmed/high) `version-sensitive` - Codex subagent nesting is capped by `agents.max_depth`, which defaults to 1: the root thread (depth 0) may spawn direct children, but those children cannot spawn deeper descendants.

Applies to: Codex CLI 0.144.x (config.toml `[agents]` table) - [Subagents | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-03** (confirmed/high) `version-sensitive` - The documented `[agents]` config surface consists of `agents.max_threads`, `agents.max_depth`, `agents.job_max_runtime_seconds`, `agents.interrupt_message`, plus per-agent keys `agents.<name>.config_file`, ``agents.<name> ...`

Applies to: Codex CLI 0.144.x config reference - [Configuration Reference | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-04** (confirmed/high) `version-sensitive` - Custom Codex agents are defined as TOML files in `~/.codex/agents/` for user-global scope and `.codex/agents/` for project scope.

Applies to: Codex CLI 0.144.x - [Subagents | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-05** (confirmed/high) `version-sensitive` - A custom Codex agent TOML file requires the fields `name`, `description`, and `developer_instructions`.

Applies to: Codex CLI 0.144.x (`.codex/agents/*.toml` schema) - [Subagents | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-06** (confirmed/high) `version-sensitive` - A custom Codex agent TOML file may optionally override `model`, `model_reasoning_effort`, `sandbox_mode`, `mcp_servers`, `skills.config`, and `nickname_candidates` relative to the parent session.

Applies to: Codex CLI 0.144.x (`.codex/agents/*.toml` schema) - [Subagents | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-07** (confirmed/high) `version-sensitive` - Codex subagents inherit the parent turn's permission mode and sandbox policy, including runtime overrides made during the session such as `/permissions` changes and approval choices.

Applies to: Codex CLI 0.144.x - [Subagents | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-08** (corrected/high) `version-sensitive` - Codex ships three built-in agents: `default` (general-purpose fallback agent), `worker` (execution-focused agent for implementation and fixes), and `explorer` (read-heavy codebase exploration agent).

Applies to: Codex CLI 0.144.x - [Subagents | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-09** (confirmed/high) `version-sensitive` - The Codex Linux sandbox (crate `codex-linux-sandbox`) uses bubblewrap (`bwrap`) as its primary backend, preferring the first

`bwrap` found on PATH outside the current working directory and falling back to a bundled binar ...

Applies to: Codex CLI 0.144.x on Linux - [codex-rs/linux-sandbox/README.md](#) — [openai/codex](#) (raw)

- **codex-subagents-sandbox-10** (confirmed/high) `version-sensitive` - Under bubblewrap the Codex Linux sandbox mounts the filesystem read-only by default via `--ro-bind / /`, grants writable roots via `--bind <root> <root>`, and re-applies `.git` and `.codex` subpaths as read-only via `--r ...`

Applies to: Codex CLI 0.144.x on Linux - [codex-rs/linux-sandbox/README.md](#) — [openai/codex](#) (raw)

- **codex-subagents-sandbox-11** (corrected/high) `version-sensitive` - The Codex Linux sandbox applies `--unshare-user` and `--unshare-pid`, and additionally applies `--unshare-net` when network is restricted AND no proxy routing is in use, plus `PR_SET_NO_NEW_PRIVS` and an in-process secco ...

Applies to: Codex CLI 0.144.x on Linux - [codex-rs/linux-sandbox/README.md](#) — [openai/codex](#) (raw)

- **codex-subagents-sandbox-12** (confirmed/high) `version-sensitive` - Landlock plus mount protections remain available in Codex only as a legacy backend, forced via `features.use_legacy_landlock = true` (or `-c use_legacy_landlock=true` on the CLI).

Applies to: Codex CLI 0.144.x on Linux - [codex-rs/linux-sandbox/README.md](#) — [openai/codex](#) (raw)

- **codex-subagents-sandbox-13** (confirmed/high) `version-sensitive` - WSL1 is unsupported by the Codex sandbox; Codex rejects sandboxed commands on WSL1 before invoking bubblewrap.

Applies to: Codex CLI 0.144.x on WSL1 - [codex-rs/linux-sandbox/README.md](#) — [openai/codex](#) (raw)

- **codex-subagents-sandbox-14** (corrected/high) `version-sensitive` - Codex `sandbox_mode` accepts exactly three values: `read-only`, `workspace-write`, and `danger-full-access`. Under `workspace-write` the agent runs with network access turned off by default and write permissions limited ...

Applies to: Codex CLI 0.144.x - [Agent approvals & security | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-15** (confirmed/high) `version-sensitive` - Codex `approval_policy` accepts `untrusted`, `on-request`, `never`, or a granular table form `approval_policy = { granular = { ... } }` whose documented sub-keys include `sandbox_approval`, `rules`, `mcp_elicitations`, ` ...

Applies to: Codex CLI 0.144.x - [Configuration Reference | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-16** (confirmed/high) `version-sensitive` - Within a Codex writable root, `<writable_root>/.git`, `<writable_root>/.agents`, and `<writable_root>/.codex` are held read-only.

Applies to: Codex CLI 0.144.x - [Agent approvals & security | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-17** (corrected/high) `version-sensitive` - Codex network access when enabled is governed by the `features.network_proxy` `domains` policy, which is unset by default — meaning no external destinations are allowed until `allow` rules are added. Deny rules always wi ...

Applies to: Codex CLI 0.144.x - [Configuration Reference | ChatGPT Learn \(OpenAI Codex docs\)](#)

- **codex-subagents-sandbox-18** (confirmed/high) **version-sensitive** - Codex MCP servers are configured under `[mcp_servers.<server-name>]` in `~/.codex/config.toml` or a project-scoped `.codex/config.toml`; stdio servers require `command`, streamable-HTTP servers require `url`, and shared ...

Applies to: Codex CLI 0.144.x - [Model Context Protocol | ChatGPT Learn \(OpenAI Codex docs, CLI surface\)](#)

- **codex-subagents-sandbox-19** (confirmed/high) **version-sensitive** - Codex MCP tool approval is controlled by `default_tools_approval_mode`, whose supported values are `auto`, `prompt`, `writes`, and `approve`, with per-tool override via `mcp_servers.<id>.tools.<tool>.approval_mode`.

Applies to: Codex CLI 0.144.x - [Model Context Protocol | ChatGPT Learn \(OpenAI Codex docs, CLI surface\)](#)

- **codex-subagents-sandbox-20** (confirmed/high) **version-sensitive** - Codex manages MCP servers from the terminal via `codex mcp add <name> --env VAR=VALUE -- <command>`, `codex mcp list`, and `codex mcp login <server-name>` for OAuth; a fixed OAuth callback port is set with the top-level ...

Applies to: Codex CLI 0.144.x - [Model Context Protocol | ChatGPT Learn \(OpenAI Codex docs, CLI surface\)](#)

- **codex-subagents-sandbox-21** (confirmed/high) **version-sensitive** - openai/codex issue #16068 ('Setting model_context_window in config.toml breaks auto-compaction (fill_to_context_window resets token counter)'), opened 2026-03-28 against 0.116.0/0.117.0, has been closed as a duplicate of ...

Applies to: Codex CLI 0.116.0–0.117.0 (reported); status verified 2026-07-20 - [Issue #16068 — openai/codex \(GitHub API\)](#)

- **codex-subagents-sandbox-22** (confirmed/high) **version-sensitive** - openai/codex issue #16033 ('Auto-compaction never triggers'), opened 2026-03-27 against codex-cli 0.117.0 and involving `model_auto_compact_token_limit`, remains OPEN as of 2026-07-20 with no released fix identified.

Applies to: Codex CLI 0.117.0 through at least 0.144.6 - [Issue #16033 — Auto-compaction never triggers — openai/codex \(GitHub API\)](#)

- **codex-subagents-sandbox-23** (confirmed/high) **version-sensitive** - Codex CLI 0.144.6, released 2026-07-18, refreshed bundled instructions for GPT-5.6 Sol, Terra and Luna and corrected their context windows to 272,000 tokens.

Applies to: Codex CLI 0.144.6 - [Releases — openai/codex rust-v0.144.6 \(GitHub API, full release body\)](#)

- **codex-subagents-sandbox-24** (confirmed/high) **version-sensitive** - openai/codex issue #32806, opened 2026-07-13 against 0.144.3, reported gpt-5.6-sol's observed raw context window

dropping from 372,000 to 272,000 tokens (effective ~258.4K under a 95% policy) and has been closed.

Applies to: Codex CLI 0.144.3, model gpt-5.6-sol - [Issue #32806 — openai/codex \(GitHub API\)](#)

- **codex-subagents-sandbox-25** (corrected/high) **version-sensitive** - Two open, maintainer-unconfirmed openai/codex issues filed in July 2026 report that Codex Desktop caps concurrent agent threads at 4 regardless of the configured `agents.max_threads`: #33039 (2026-07-14, CLI 0.144.2, mac ...)

Applies to: Codex Desktop 26.707.x with bundled Codex CLI 0.144.2 - [Issue #33039 — openai/codex \(GitHub API\)](#)

- **codex-subagents-sandbox-26** (confirmed/high) - OpenAI Codex documentation URLs under `developers.openai.com/codex/*` now return HTTP 308 permanent redirects to `learn.chatgpt.com/docs/*` (e.g. `/codex/subagents` → `/docs/agent-configuration/subagents`, `/codex/changelog` → ...)

Applies to: OpenAI Codex documentation site, observed 2026-07-20 - [HTTP header check: developers.openai.com/codex/{subagents,changelog,config-reference}](#) (direct curl, 308 responses)

security-sandboxing-injection (16)

- **security-sandboxing-injection-01** (confirmed/high) **version-sensitive** - Claude Code's sandboxed Bash tool enforces isolation using OS primitives: Seatbelt on macOS and bubblewrap on Linux and WSL2. Native Windows and WSL1 are not supported; on Linux/WSL2 the sandbox additionally requires the ...

Applies to: Claude Code v2.1.x CLI, macOS/Linux/WSL2 only - [Configure the sandboxed Bash tool - Claude Code Docs](#)

- **security-sandboxing-injection-02** (confirmed/high) **version-sensitive** - By default the Claude Code Bash sandbox grants read access to the entire computer except certain denied directories. The official documentation states this default still allows reading credential files such as `~/aws/cr ...`

Applies to: Claude Code v2.1.x CLI, sandbox enabled with default filesystem settings - [Configure the sandboxed Bash tool - Claude Code Docs](#)

- **security-sandboxing-injection-03** (confirmed/high) **version-sensitive** - The `sandbox.credentials` settings block declares credential files and environment variables to protect from sandboxed commands and requires Claude Code v2.1.187 or later. File entries support only `"mode": "deny"`. Envi ...

Applies to: Claude Code v2.1.187+ for `deny`; v2.1.199+ for `mask` - [Configure the sandboxed Bash tool - Claude Code Docs](#)

- **security-sandboxing-injection-04** (confirmed/high) **version-sensitive** - Claude Code's built-in sandbox proxy enforces its domain allowlist based on the client-supplied hostname and by

default does not terminate or inspect TLS. The documentation warns that code running inside the sandbox can ...

Applies to: Claude Code v2.1.x CLI, built-in sandbox proxy with default (non-TLS-terminating) configuration - [Configure the sandboxed Bash tool - Claude Code Docs](#)

- **security-sandboxing-injection-05** (confirmed/high) **version-sensitive** - By default, if the Claude Code sandbox cannot start because dependencies are missing or the platform is unsupported, Claude Code shows a warning and runs commands WITHOUT sandboxing. Setting `sandbox.failIfUnavailable` t ...

Applies to: Claude Code v2.1.x CLI, all supported platforms - [Configure the sandboxed Bash tool - Claude Code Docs](#)

- **security-sandboxing-injection-06** (confirmed/high) **version-sensitive** - In Claude Code managed settings, boolean sandbox keys such as `enabled` and `failIfUnavailable` override anything a developer sets locally, but array keys such as `excludedCommands` and `allowRead` are merged across ever ...

Applies to: Claude Code v2.1.x CLI with managed settings (MDM file or server-managed settings) - [Configure the sandboxed Bash tool - Claude Code Docs](#)

- **security-sandboxing-injection-07** (confirmed/high) **version-sensitive** - Claude Code exposes a `dangerouslyDisableSandbox` Bash parameter as an escape hatch: when a command fails because of sandbox restrictions, Claude analyzes the failure and may retry the command outside the sandbox, where ...

Applies to: Claude Code v2.1.x CLI, sandbox enabled - [Configure the sandboxed Bash tool - Claude Code Docs](#)

- **security-sandboxing-injection-08** (confirmed/high) **version-sensitive** - The Claude Code sandbox automatically denies write access to Claude Code's `settings.json` files at every scope and to the managed settings directory, so a sandboxed command cannot modify its own policy. In v2.1.210 and ...

Applies to: Claude Code v2.1.x CLI; symlink resolution requires v2.1.210+ - [Configure the sandboxed Bash tool - Claude Code Docs](#)

- **security-sandboxing-injection-09** (confirmed/high) **version-sensitive** - The Claude Code sandbox isolates Bash subprocesses only. The built-in Read, Edit and Write file tools use the permission system directly rather than running through the sandbox, and sandboxed Bash commands inherit the pa ...

Applies to: Claude Code v2.1.x CLI, sandbox enabled - [Configure the sandboxed Bash tool - Claude Code Docs](#)

- **security-sandboxing-injection-10** (confirmed/high) - Meta's "Agents Rule of Two", published 2025-10-31, states that an agent must satisfy no more than two of the following three properties within a session: [A] processing untrustworthy inputs, [B] access to sensitive syste ...

Applies to: Vendor-agnostic agent architecture guidance; inspired by Chromium security policy and Simon Willison's lethal trifecta f ... - [Agents Rule of Two: A Practical Approach to AI Agent Security - Meta AI](#)

- **security-sandboxing-injection-11** (confirmed/high) **version-sensitive** - OWASP GenAI Security Project published "State of Agentic AI Security and Governance" version 2.01 in June 2026, updating the July 2025 v1.0. The edition ranks prompt injection the leading vulnerability and maps it to six ...

Applies to: Vendor-agnostic; OWASP State of Agentic AI Security and Governance v2.01 (June 2026) - [State of Agentic AI Security and Governance, Version 2.01, June 2026 \(OWASP GenAI Security Project, full PDF\)](#)

- **security-sandboxing-injection-12** (confirmed/high) - The NVIDIA AI Red Team names three mandatory controls for sandboxing agentic workflows against indirect prompt injection: (1) network egress controls via tightly scoped allowlists plus DNS resolution limited to trusted r ...

Applies to: Vendor-agnostic agentic workflow deployment; NVIDIA Technical Blog, published 2026-01-30, author Rich Harang - [Practical Security Guidance for Sandboxing Agentic Workflows and Managing Execution Risk | NVIDIA Technical Blog](#)

- **security-sandboxing-injection-13** (confirmed/high) - Nasr et al., "The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections" (arXiv:2510.09023, submitted 2025-10-10), bypassed 12 recent defenses using gradient descen ...

Applies to: Vendor-agnostic; applies to detection- and training-based prompt-injection defenses evaluated on static benchmarks - [\(2510.09023\) The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections](#)

- **security-sandboxing-injection-14** (confirmed/high) - PromptArmor (arXiv:2507.15219) reports both a false positive rate and a false negative rate below 1% on the AgentDojo benchmark when using off-the-shelf GPT-4o, GPT-4.1 or o4-mini as the guardrail LLM, by prompting the g ...

Applies to: Vendor-agnostic filter-LLM defense pattern; benchmark result specific to AgentDojo and the named guardrail models - [PromptArmor: Simple yet Effective Prompt Injection Defenses \(arXiv:2507.15219v1\)](#)

- **security-sandboxing-injection-15** (confirmed/high) - "Adaptive Evaluation of Out-of-Band Defenses Against Prompt Injection in LLM Agents" (arXiv:2606.26479, submitted 2026-06-25) evaluated out-of-band defenses including CaMeL, FIDES, Progent, RTBAS and FORGE. Testing Proge ...

Applies to: Vendor-agnostic; result specific to Progent on Qwen2.5-7B, not to the other named defenses - [Adaptive Evaluation of Out-of-Band Defenses Against Prompt Injection in LLM Agents \(arXiv:2606.26479\)](#)

- **security-sandboxing-injection-16** (corrected/high) **version-sensitive** - Microsoft's Semantic Kernel had two critical vulnerabilities enabling the prompt-injection-to-code-execution path. CVE-

2026-26030 affects the semantic-kernel Python package prior to version 1.39.4:

InMemoryVectorStore bu ...

Applies to: Microsoft Semantic Kernel: Python <1.39.4, .NET SDK <1.71.0 - [NVD](#) - [CVE-2026-25592](#)

evals-judging (25)

- **evals-judging-01** (confirmed/high) - RuVerBench is a meta-evaluation benchmark of 2,458 rubric-verification instances for measuring LLM-as-a-Judge reliability in agentic scenarios; each instance pairs a model-generated output with a single rubric item and a ...

Applies to: vendor-agnostic; rubric-based scoring of long agentic outputs (deep research and agentic coding) - [Can LLM-as-a-Judge Reliably Verify Rubrics in Agentic Scenarios?](#)

([arXiv:2606.29920](#))

- **evals-judging-02** (confirmed/high) - RuVerBench's 2,458 instances derive from 494 scored cases split as DeepResearch (284 cases, 1,615 rubric points) and AgenticCoding (210 cases, 843 checklist items), where the agentic-coding cases are serialized agent tra ...

Applies to: vendor-agnostic; RuVerBench release as published at THU-KEG/RuVerBench, code under MIT with data under separate terms - [THU-KEG/RuVerBench GitHub repository](#)

- **evals-judging-03** (confirmed/high) - On RuVerBench the authors report that even the most advanced frontier LLMs achieve strong performance on rubric verification but still exhibit substantial noise.

Applies to: vendor-agnostic; frontier LLMs evaluated as of the June 2026 submission - [Can LLM-as-a-Judge Reliably Verify Rubrics in Agentic Scenarios?](#) ([arXiv:2606.29920](#))

- **evals-judging-04** (confirmed/high) - RuVerBench's strategy analysis finds three effects: weaker judge models are more sensitive to prompt-design variation than strong ones; batched rubric verification presents an accuracy-versus-efficiency trade-off; and ma ...

Applies to: vendor-agnostic; rubric verification with LLM judges - [Can LLM-as-a-Judge Reliably Verify Rubrics in Agentic Scenarios?](#) ([arXiv:2606.29920](#))

- **evals-judging-05** (confirmed/high) - A systematic evaluation of 21 judge models from nine providers across MT-Bench, JudgeBench and RewardBench, spanning 118 runs and approximately 541,000 individual judgments, reports that judges can be highly reliable (se ...

Applies to: vendor-agnostic; pairwise LLM-judge protocols as evaluated June 2026 - [Reliability without Validity: A Systematic, Large-Scale Evaluation of LLM-as-a-Judge Models Across Agreement, Consistency, and Bias](#) ([arXiv:2606.19544](#))

- **evals-judging-06** (confirmed/high) - Switching from raw exact-match agreement to Cohen's kappa deflates measured judge-human agreement by 33 to 41 percentage points on MT-Bench, because exact match does not correct for chance agreement.

Applies to: vendor-agnostic; any judge scored against human labels on a small discrete label set - [Reliability without Validity: A Systematic, Large-Scale Evaluation of LLM-as-a-Judge Models Across Agreement, Consistency, and Bias](#) ([arXiv:2606.19544](#))

- **evals-judging-07** (confirmed/high) - Judge model rankings shift by up to 14 positions depending on which benchmark is used to rank them, indicating judge quality measured on one benchmark does not transfer to another.

Applies to: vendor-agnostic; judge selection across MT-Bench, JudgeBench, RewardBench -

[Reliability without Validity: A Systematic, Large-Scale Evaluation of LLM-as-a-Judge Models Across Agreement, Consistency, and Bias \(arXiv:2606.19544\)](#)

- **evals-judging-08** (confirmed/high) - Two production judge models exhibited test-retest reliability above 0.95 while simultaneously showing position bias above 0.10, a consistency-bias pairing showing a judge can be stably wrong in the same direction on ever ...

Applies to: vendor-agnostic; pairwise comparison protocols as measured June 2026 -

[Reliability without Validity: A Systematic, Large-Scale Evaluation of LLM-as-a-Judge Models Across Agreement, Consistency, and Bias \(arXiv:2606.19544\)](#)

- **evals-judging-09** (confirmed/high) - Under a single pairwise rubric, measured verbosity bias was small (effect below 0.011), which is materially weaker than the verbosity-bias effect commonly asserted in practitioner guidance.

Applies to: vendor-agnostic; single pairwise rubric protocol specifically, not multi-criteria or

pointwise scoring - [Reliability without Validity: A Systematic, Large-Scale Evaluation of LLM-as-a-Judge Models Across Agreement, Consistency, and Bias \(arXiv:2606.19544\)](#)

- **evals-judging-10** (confirmed/high) **version-sensitive** - Training a policy against a reference-free LLM judge on GSM8K with Qwen3 policies drove the judge's pass rate from 0.72 to 0.94 across three seeds while true accuracy, measured by a held-out cross-source exact-match audi ...

Applies to: Qwen3 policies on GSM8K, self-play / self-rewarding / reference-free LLM-as-a-judge

training loops; submitted 2026-07-07 - [More Convincing, Not More Correct: Self-Play Reward Hacking of Reference-Free LLM Judges \(arXiv:2607.05904\)](#)

- **evals-judging-11** (confirmed/high) - The self-play judge exploits are not white-box gaming: the induced errors transfer across judge families (Qwen, Llama, Gemma) and across scales, and a strict three-judge ensemble still accepts 55 percent of the exploited ...

Applies to: vendor-agnostic; cross-family judge ensembles, measured July 2026 -

[More Convincing, Not More Correct: Self-Play Reward Hacking of Reference-Free LLM Judges \(arXiv:2607.05904\)](#)

- **evals-judging-12** (confirmed/high) **version-sensitive** - The decisive variable in reference-free judging is whether the judge commits to its own answer before seeing the candidate: committing first drops the false-positive rate from 0.719 to 0.012, blind solving lifts discrimi ...

Applies to: Qwen3/Llama/Gemma judges on GSM8K-style verifiable tasks; reference-free judging

protocols, July 2026 - [More Convincing, Not More Correct: Self-Play Reward Hacking of Reference-Free LLM Judges \(arXiv:2607.05904\)](#)

- **evals-judging-13** (confirmed/medium) - CHERRL is a controlled framework that deliberately injects known judge biases into an LLM-as-a-Judge reward to stably reproduce reward hacking in rubric-based RL, enabling explicit observation of reward divergence and pr ...

Applies to: vendor-agnostic; rubric-based reinforcement learning with LaaJ rewards; submitted 2026-06-03 - [Reproducing, Analyzing, and Detecting Reward Hacking in Rubric-Based Reinforcement Learning \(arXiv:2606.04923\)](#)

- **evals-judging-14** (corrected/high) **version-sensitive** - The Reward Hacking Benchmark (RHB) evaluated 13 frontier models from OpenAI, Anthropic, Google and DeepSeek using multi-step tool-use tasks with naturalistic shortcut opportunities, measuring exploit rates from 0 percent ...

Applies to: model-specific as of the 2026-05-03 submission; exploit rates are tied to those model versions and will move between rel ... - [Reward Hacking Benchmark: Measuring Exploits in LLM Agents with Tool Use \(arXiv:2605.02964\)](#)

- **evals-judging-15** (corrected/high) - In the Reward Hacking Benchmark, 72 percent of reward-hacking episodes included an explicit chain-of-thought rationale for the exploit, which the authors read as models framing exploits as legitimate problem-solving. The ...

Applies to: vendor-agnostic; CoT-monitoring detectors over reasoning-model agent traces, May 2026 - [Reward Hacking Benchmark: Measuring Exploits in LLM Agents with Tool Use \(arXiv:2605.02964\)](#)

- **evals-judging-16** (confirmed/high) - Environment hardening reduced measured exploit rates by 5.7 percentage points, an 87.7 percent relative reduction, and aligned post-training suppresses reward hacking only when honest solutions remain computationally tra ...

Applies to: vendor-agnostic; tool-using agent environments, May 2026 - [Reward Hacking Benchmark: Measuring Exploits in LLM Agents with Tool Use \(arXiv:2605.02964\)](#)

- **evals-judging-17** (corrected/high) **version-sensitive** - CodeScaleBench is a coding-agent benchmark spanning more than forty repositories and nine programming languages, evaluated against approximately 180 GitHub mirrors pinned at fixed commits. Task counts depend on which fro ...

Applies to: CodeScaleBench as published 2026-03-03; Apache 2.0; task counts move as the suite grows - [sourcegraph/CodeScaleBench GitHub repository](#)

- **evals-judging-18** (confirmed/high) **version-sensitive** - CodeScaleBench scores tasks with deterministic verifiers rather than an LLM judge: a patch-based verifier in the SWE-bench style for the SDLC suite, and an artifact verifier comparing an agent-produced answer.json agains ...

Applies to: CodeScaleBench harness (Harbor runner with the Claude Code agent harness), as documented 2026-07-20 - [sourcegraph/CodeScaleBench GitHub repository](#)

- **evals-judging-19** (confirmed/high) **version-sensitive** - CodeScaleBench gates each verifier through a three-check calibration triad before the task is admitted: a null check where an empty answer must score at most 0.1, a golden check where the canonical oracle_answer.json mus ...

Applies to: CodeScaleBench verifier QA pipeline as documented 2026-07-20; thresholds are repo-specific config - [sourcegraph/CodeScaleBench GitHub repository](#)

- **evals-judging-21** (confirmed/high) **version-sensitive** - An audit of three performance-optimization coding-agent benchmarks found their own reference patches fail the benchmarks'

validity rules under cross-machine replay: only 39 of 102 GSO tasks (38 percent), 11 of 140 SWE-Pe ...

Applies to: GSO, SWE-Perf and SWE-fficiency as audited in arXiv:2607.01211 (v1 2026-07-01, v2 2026-07-16) - [Are Performance-Optimization Benchmarks Reliably Measuring Coding Agents? \(arXiv:2607.01211\)](#)

- **evals-judging-22** (confirmed/high) **version-sensitive** - In the same audit, official rankings from GSO and SWE-fficiency disagree on 9 of 28 pairwise comparisons among eight public submissions, and SWE-fficiency assigns its worst ten tasks score weights of 58.5 to 82.8 percent ...

Applies to: GSO and SWE-fficiency leaderboards as audited July 2026 - [Are Performance-Optimization Benchmarks Reliably Measuring Coding Agents? \(arXiv:2607.01211\)](#)

- **evals-judging-23** (confirmed/high) **version-sensitive** - In the same audit, at least one public submission matches or beats the reference patch on 85.3 percent (384 of 450) of replay-valid tasks, and submissions beat the unoptimized baseline on 99.8 percent (449 of 450), a nea ...

Applies to: GSO and SWE-fficiency public submissions as audited July 2026 - [Are Performance-Optimization Benchmarks Reliably Measuring Coding Agents? \(arXiv:2607.01211\)](#)

- **evals-judging-24** (confirmed/high) - Multilingual query-based prompt injection against LLM relevance judges was found highly effective at inflating relevance scores while evading existing prompt-injection defenses, tested across eight languages of differing ...

Applies to: LLM relevance judgment for information retrieval, open-weight models, submitted 2026-07-11; transfer to code-review judg ... - [The Effect of Multi-Lingual and Keyword Adversarial Injection on LLM Relevance Judgment \(arXiv:2607.10080\)](#)

- **evals-judging-25** (confirmed/medium) - A practitioner guide recommends interpreting judge-to-human Cohen's kappa on the bands below 0.4 (rubric too ambiguous, rewrite), 0.4 to 0.6 (weak but tunable), above 0.6 (acceptable for production) and above 0.8 (strong ...

Applies to: vendor-agnostic practitioner heuristic; blog originally published 2025-01-29, updated 2026-05-14 - [LLM-as-a-Judge \(Future AGI blog\)](#)

- **evals-judging-26** (confirmed/high) - Sourcegraph reports that 84 percent of large enterprise accounts saw a steady increase in lines of code after AI rollout.

Applies to: Sourcegraph enterprise customer base, blog published 2026-05-21; vendor-reported, no methodology disclosed - [Agentic Coding in 2026: A Practical Guide for Big Code \(Sourcegraph\)](#)

drift-discovery (29)

- **drift-discovery-01** (confirmed/high) **version-sensitive** - <https://developers.openai.com/codex/changelog> returns HTTP 308 Permanent Redirect to <https://learn.chatgpt.com/docs/changelog>; the redirect rule flattens the path, so /codex/<page> becomes /docs/<page> (e.g. /codex/confi ...

Applies to: OpenAI Codex documentation hosting, observed 2026-07-20; vendor-agnostic impact on any registry pinning developers.opena ... - developers.openai.com/codex/changelog (308 redirect

observed)

- **drift-discovery-02** (confirmed/high) **version-sensitive** - Codex CLI 0.144.6, dated 2026-07-18, updated bundled instructions and corrected the context window to 272,000 tokens for the GPT-5.6 Sol, Terra and Luna models.

Applies to: Codex CLI 0.144.6; GPT-5.6 Sol / Terra / Luna model family - [OpenAI Codex changelog](#), [Codex CLI 0.144.6 entry \(2026-07-18\)](#)

- **drift-discovery-03** (confirmed/high) **version-sensitive** - Codex CLI 0.144.5, dated 2026-07-16, expanded dangerous-command detection to include additional forced 'rm' forms and improved rejection messaging.

Applies to: Codex CLI 0.144.5 and later - [OpenAI Codex changelog](#), [Codex CLI 0.144.5 entry \(2026-07-16\)](#)

- **drift-discovery-04** (confirmed/high) **version-sensitive** - The Codex config reference documents approval_policy allowed values as "untrusted | on-request | never | { granular = { sandbox_approval = bool, rules = bool, mcp_elicitations = bool, request_permissions = bool, skill_ap ...

Applies to: Codex CLI config.toml, documentation as of 2026-07-20 (tracking Codex CLI 0.144.x) - [Codex Configuration Reference \(raw markdown, approval_policy entry\)](#)

- **drift-discovery-05** (corrected/high) **version-sensitive** - The Codex config reference documents sandbox_mode with the type string "read-only | workspace-write | danger-full-access" and the description "Sandbox policy for filesystem and network access during command execution." T ...

Applies to: Codex CLI config.toml, documentation as of 2026-07-20 (tracking Codex CLI 0.144.x) - [Codex Configuration Reference \(raw markdown, sandbox_mode entry\)](#)

- **drift-discovery-06** (confirmed/high) **version-sensitive** - The Codex config reference documents a default_tools_approval_mode key on apps.<id>, apps._default and mcp_servers.<id>, with allowed values "auto | prompt | writes | approve"; the 'writes' app-approval mode was introduced ...

Applies to: Codex CLI 0.144.0 and later; config.toml keys apps.<id>.default_tools_approval_mode, apps._default.default_tools_approva ... - [Codex Configuration Reference \(raw markdown; apps._default, apps.<id>, mcp_servers.<id> entries\)](#)

- **drift-discovery-07** (corrected/high) **version-sensitive** - The Codex config reference documents model_reasoning_effort allowed values as "minimal | low | medium | high | xhigh", with the description "Adjust reasoning effort for supported models (Responses API only; **xhigh** is mo ...

Applies to: Codex CLI config.toml, documentation as of 2026-07-20 (tracking Codex CLI 0.144.x) - [Codex Configuration Reference \(raw markdown; model_reasoning_effort and plan_mode_reasoning_effort entries\)](#)

- **drift-discovery-08** (confirmed/high) **version-sensitive** - Codex documentation states that destructive app/MCP tool calls always require approval when the tool advertises a destructive

annotation, even if it also advertises other hints.

Applies to: Codex CLI approvals model, documentation as of 2026-07-20 - [Codex agent approvals & security](#)

- **drift-discovery-09** (corrected/high) **version-sensitive** - Claude Code 2.1.215 is the current npm 'latest' dist-tag as of 2026-07-20, and its sole changelog bullet reads: "Claude no longer runs the `/verify` and `/code-review` skills on its own; invoke them with `/verify` or `/code-rev` ...

Applies to: Claude Code v2.1.215 - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.215 section read verbatim](#)

- **drift-discovery-10** (confirmed/high) **version-sensitive** - Claude Code 2.1.214 fixed single-segment `'dir/'` **allow rules such as `Edit(src/)`** auto-approving writes to nested `'dir/'` directories anywhere in the tree instead of only `<cwd>/dir`.

Applies to: Claude Code < 2.1.214 affected; fixed in v2.1.214 - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.214 section read verbatim](#)

- **drift-discovery-11** (confirmed/high) **version-sensitive** - Claude Code 2.1.214 fixed a permission-check bypass affecting commands run in Windows PowerShell 5.1 sessions.

Applies to: Claude Code < 2.1.214 on Windows PowerShell 5.1; fixed in v2.1.214 - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.214 section read verbatim](#)

- **drift-discovery-12** (confirmed/high) **version-sensitive** - Claude Code 2.1.214 changed Bash permission checking so that commands over 10,000 characters always prompt instead of running automatically, and made checks fail closed on file-descriptor redirect forms that bash parses ...

Applies to: Claude Code v2.1.214 and later - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.214 section read verbatim](#)

- **drift-discovery-13** (confirmed/high) **version-sensitive** - Claude Code 2.1.214 added the `EndConversation` tool, allowing Claude to end sessions with highly abusive users or jailbreak attempts, described as matching behaviour present on claude.ai since 2025.

Applies to: Claude Code v2.1.214 and later - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.214 section read verbatim](#)

- **drift-discovery-14** (corrected/high) **version-sensitive** - Claude Code 2.1.214 added `message.uuid`, `client_request_id` and `tool_source` attributes to OpenTelemetry log events, and added `CLAUDE_CODE_OTEL_CONTENT_MAX_LENGTH` to configure the 60 KB truncation limit on OpenTelemetry con ...

Applies to: Claude Code v2.1.214 and later; OpenTelemetry export configuration - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.214 section read verbatim](#)

- **drift-discovery-15** (confirmed/high) **version-sensitive** - In Claude Code 2.1.212 the `/fork` command changed to copy the conversation into a new background session with its own row in 'claude agents', while the in-session forked subagent it previously launched became `/subtask`; th ...

Applies to: Claude Code v2.1.212 and later, with documented divergent behaviour on v2.1.161-v2.1.211 and pre-v2.1.161 - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.212 /fork](#)
[bullet read verbatim](#)

- **drift-discovery-16** (confirmed/high) **version-sensitive** - Claude Code 2.1.212 added a session-wide limit on WebSearch tool calls, default 200, tunable via `CLAUDE_CODE_MAX_WEB_SEARCHES_PER_SESSION`, to stop runaway search loops.

Applies to: Claude Code v2.1.212 and later - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.212 section read verbatim](#)

- **drift-discovery-17** (confirmed/high) **version-sensitive** - Claude Code 2.1.212 added a per-session cap on subagent spawns, default 200, overridable with `CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION`, and `/clear` resets the budget.

Applies to: Claude Code v2.1.212 and later - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.212 section read verbatim](#)

- **drift-discovery-18** (corrected/high) **version-sensitive** - Claude Code 2.1.212 made MCP tool calls in the main conversation that are still running after two minutes move to a background task automatically. The env-vars reference documents `CLAUDE_CODE_MCP_AUTO_BACKGROUND_MS` as "E ..."

Applies to: Claude Code v2.1.212 and later; MCP server integrations - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.212 section read verbatim](#)

- **drift-discovery-19** (confirmed/high) **version-sensitive** - Claude Code 2.1.212 deprecated the Task tool's 'mode' parameter, which is now ignored; subagents instead inherit the parent session's permission mode by default.

Applies to: Claude Code v2.1.212 and later; Task tool / subagent invocation - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.212 section read verbatim](#)

- **drift-discovery-20** (confirmed/high) **version-sensitive** - Claude Code 2.1.212 fixed plan mode auto-running file-modifying Bash commands such as `touch` and `rm` without a permission prompt or an SDK `canUseTool` callback.

Applies to: Claude Code < 2.1.212 affected; fixed in v2.1.212 - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.212 section read verbatim](#)

- **drift-discovery-21** (confirmed/high) **version-sensitive** - Claude Code 2.1.210 fixed isolation: 'worktree' subagents being able to run git-mutating commands against the main repo checkout instead of their own isolated worktree.

Applies to: Claude Code < 2.1.210 affected; fixed in v2.1.210; subagent frontmatter key isolation: 'worktree' - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.210 section read verbatim](#)

- **drift-discovery-22** (confirmed/high) **version-sensitive** - Claude Code 2.1.210 added a startup warning for `Write(path)`, `NotebookEdit(path)` and `Glob(path)` permission rules, advising use of `Edit(path)` or `Read(path)` instead.

Applies to: Claude Code v2.1.210 and later; settings.json permission rules - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.210 section read verbatim](#)

- **drift-discovery-23** (confirmed/high) **version-sensitive** - Claude Code 2.1.210 hardened the Agent tool against indirect prompt injection via content a subagent read.

Applies to: Claude Code v2.1.210 and later; Agent tool / subagent delegation - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.210 section read verbatim](#)

- **drift-discovery-24** (confirmed/high) **version-sensitive** - Claude Code 2.1.211 added the --forward-subagent-text flag and the CLAUDE_CODE_FORWARD_SUBAGENT_TEXT environment variable to include subagent text and thinking in stream-json output.

Applies to: Claude Code v2.1.211 and later; stream-json / headless and SDK output - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.211 section read verbatim](#)

- **drift-discovery-25** (confirmed/high) **version-sensitive** - Claude Code 2.1.211 changed "always allow" permission rules to save at the repository root, so approvals granted inside a git worktree persist across sessions and worktrees.

Applies to: Claude Code v2.1.211 and later - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.211 section read verbatim](#)

- **drift-discovery-26** (corrected/high) **version-sensitive** - Claude Code 2.1.211 updated integer environment variables (timeouts, token budgets, retry counts) to accept scientific notation and digit-separator spellings such as 1e6 and 64_000. This is not universal across integer v ...

Applies to: Claude Code v2.1.211 and later; integer-valued environment variables - [anthropics/claude-code CHANGELOG.md \(raw\)](#), [version 2.1.211 section read verbatim](#)

- **drift-discovery-27** (corrected/medium) **version-sensitive** - Anthropic's own support documentation states that starting July 20, 2026, Claude Fable 5 is a standard part of Max plans, premium seats on Team plans and premium seats on seat-based Enterprise plans, where "you can use u ...

Applies to: Anthropic consumer/team subscription plans effective 2026-07-20; Claude Fable 5 - [Anthropic, 'Claude Fable 5 on your plan' \(Claude Help Center\)](#)

- **drift-discovery-28** (corrected/medium) - A practitioner disclosure (published 2026-07-09, circulated more widely from 2026-07-15) reports that Claude.ai's web_fetch tool could be induced to exfiltrate memory contents by following attacker-controlled nested link ...

Applies to: Claude.ai consumer assistant web_fetch tool; reported as fixed. NOT reported against Claude Code - [Ayush Paul, 'The Memory Heist' \(published 2026-07-09\)](#)

- **drift-discovery-29** (confirmed/high) - As of 2026-07-20 the anthropic.com/engineering index shows no post newer than 2026-04-23 ('An update on recent Claude Code quality reports'), while Claude Code engineering content such as 'A harness for every task: dynam ...

Applies to: Source-registry maintenance for Anthropic engineering content, observed 2026-07-20 - [Anthropic Engineering index \(newest post Apr 23, 2026\)](#)

Rejected claims

4 claims failed verification and are excluded from all candidate guidance. Notably, every rejection here was forced by the **tier rule** (a version-sensitive claim lacking a current tier-1 primary source), not by being shown false - the verifiers recorded that distinction explicitly.

- **codex-exec-automation-15** (unsupported) - Because exec mode is non-interactive, approval requests cause the run to fail immediately unless the configured policy auto-approves them.

Reason: Version_sensitive with tier-1 backing absent, so the rule forces unsupported. I went looking specifically: the two tier-1 pages that would state this — the dedicated agent-approvals-security page and the config reference — both decline to describe what happens when a non-interactive run hits an approval request, and the non-interactive page has no approval section at all. Only DeepWiki (tier 5, auto-generated) asserts 'approval requests cause immediate failure'. Useful side-findings for whoever

- **codex-exec-automation-43** (unsupported) - The `codex exec` headless mode is implemented over the same core engine as the TUI and app server via an `InProcessAppServerClient` managing a `ThreadManager`, with two output processors: `EventProcessorWithHumanOutput` and `EventProcessorWithJsonOutput`.

Reason: Version_sensitive claim with tier-5 sole backing, so the rule forces unsupported regardless of plausibility. DeepWiki does assert exactly what the claim says (InProcessAppServerClient managing a ThreadManager, plus EventProcessorWithHumanOutput and EventProcessorWithJsonOutput), and the researcher's own low-confidence flag is appropriate. The externally observable consequence — exec and TUI share the engine and differ in the event renderer — is consistent with tier-1 docs, but internal Rust type

- **security-sandboxing-injection-17** (unsupported) - Two competing vendor comparison pages independently agree on the isolation-mechanism taxonomy for hosted agent code-execution sandboxes: Modal uses gVisor containers, E2B and Vercel Sandbox use Firecracker microVMs, and Northflank uses Kata Containers / Cloud Hypervisor with gVisor. Both pages also

Reason: Marked unsupported on the tier rule, not on falsity — the distinction matters, so recording it precisely. I opened both pages and the mechanism taxonomy is asserted as described.

Northflank's table: Modal 'gVisor containers', E2B 'MicroVM (Firecracker)', Vercel Sandbox 'MicroVM (Firecracker)', Northflank 'MicroVM (Kata/CLH) + gVisor', Daytona 'Docker (Kata optional)'. Modal's page: Modal 'gVisor container isolation', E2B 'Firecracker microVMs', Vercel 'Firecracker microVMs', Northflank 'Kata Con

- **evals-judging-20** (unsupported) - On CodeScaleBench with Claude Code plus Haiku 4.5, adding Sourcegraph MCP retrieval versus a local-tools baseline measured up to 11.2 percent reward improvement on Org incident-debugging tasks and 11.5 percent on the SDLIC Understand suite, alongside roughly 30 percent lower cost per task (0.73 to 0).

Reason: Marked unsupported on the tiering rule, not on accuracy. The claim is flagged version_sensitive and its only backing is the vendor's own tier-3 blog; a version-sensitive claim requires current tier-1 primary support, and I could not confirm these specific deltas in the tier-1

repository. For the record, the blog does state every figure as claimed: Understand +11.5 percent, incident debugging +11.2 percent (also refactor +10.3, security +10.6, fix +9.9), cost 0.73 to 0.51 US dollars at roughly 30

Drift detected

100 observations versus the 2026-07-12 baseline.

- [claude-loops-scheduling] Latest Claude Code version as of today is v2.1.215 (July 19, 2026). Four releases landed after the 2026-07-12 baseline: v2.1.212 (Jul 17), v2.1.214 (Jul 18), and v2.1.215 (Jul 19), plus v2.1.211 (Jul 15).
- [claude-loops-scheduling] NEW GUARDRAILS (post-baseline): a session-wide WebSearch tool-call limit (default 200, `CLAUDE_CODE_MAX_WEB_SEARCHES_PER_SESSION`) and a per-session subagent spawn cap (default 200, `CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION`, reset by `/clear`) were added and are described in the changelog as explicitly intended to stop runaway search and delegation loops. These are the first CLI-side runaway-loop circuit breakers directly relevant to this bucket.
- [claude-loops-scheduling] NEW: MCP tool calls running longer than 2 minutes now move to the background automatically so the session stays usable; threshold configurable or disableable via `CLAUDE_CODE_MCP_AUTO_BACKGROUND_MS`.
- [claude-loops-scheduling] RENAME: `/fork` now copies the conversation into a new background session with its own row in claude agents; the in-session subagent it used to launch is now `/subtask`. Any repo guidance referencing `/fork` as an in-session subagent is stale.
- [claude-loops-scheduling] DEPRECATION: the Task tool's mode parameter is deprecated and now ignored; subagents inherit the parent session's permission mode by default.
- [claude-loops-scheduling] LOOP-RELEVANT FIX (v2.1.214): hooks returning exit code 2 were not blocking as documented when the hook's stdout JSON failed schema validation. Since `/goal` is a prompt-based Stop hook and Stop hooks are the custom termination primitive, any pre-v2.1.214 measurement of hook-based loop control may have been silently non-blocking.
- [claude-loops-scheduling] LOOP-RELEVANT FIX (v2.1.214): scheduled tasks were refusing their own configured prompt as untrusted input; the fired prompt is now delivered as the session's assigned task. Pre-fix, scheduled loops could no-op on their own prompt.
- [claude-loops-scheduling] LOOP-RELEVANT FIX (v2.1.210): a hook callback timeout was being misreported to the model as a user rejection, which made unattended sessions stop and wait. This is a direct silent-stall failure mode for long-running unattended loops before v2.1.210.
- [claude-loops-scheduling] ROUTINES FIX (v2.1.211): routines with no schedule trigger (API- or GitHub-only) reported a next run time in the year 1; the CLI now shows none. Corroborated independently by the routines doc, which states 'Before v2.1.211'.
- [claude-loops-scheduling] BEHAVIOR CHANGE (v2.1.215): Claude no longer invokes the `/verify` and `/code-review` skills on its own. Combined with the v2.1.196 rule that a scheduled fire only runs self-invocable skills, this plausibly changes what `'/loop <interval> /verify'` does. Flagged as derived, not confirmed.

- [claude-loops-scheduling] URL DRIFT: the registered source <https://code.claude.com/docs/en/agent-loop> now 404s. Canonical location is <https://code.claude.com/docs/en/agent-sdk/agent-loop>.
- [claude-loops-scheduling] NEW SOURCE DISCOVERED: <https://claude.com/blog/getting-started-with-loops> is an official Anthropic loop-engineering guide linked directly from the tier-1 agent-loop docs. It supplies the turn-based / goal-based / time-based / proactive taxonomy mapped onto /goal, /loop, and /schedule. Recommend registering it (tier 3, 30-day recheck).
- [claude-loops-scheduling] OTHER CANDIDATE SOURCES SURFACED but not yet fetched in depth: <https://www.anthropic.com/engineering/managed-agents> (Scaling Managed Agents: decoupling the brain from the hands) and <https://claude.com/blog/harnessing-claude-intelligence> (Agent Harness Design: 3 Patterns). Both are on-focus for long-running harness design and worth registering next pass.
- [claude-loops-scheduling] STREAM OBSERVABILITY (v2.1.211): `--forward-subagent-text` and `CLAUDE_CODE_FORWARD_SUBAGENT_TEXT` now include subagent text and thinking in stream-json output, which materially improves the ability to instrument a long-running loop's inner agents.
- [claude-hooks-permissions] Release floor moved: CHANGELOG top entry is v2.1.215, while the docs' inline min-version markers reach v2.1.211. Any baseline pinned at or before v2.1.211 is now behind. The changelog carries no dates, so exact attribution of which releases landed after 2026-07-12 is not possible from this source.
- [claude-hooks-permissions] v2.1.211 materially loosened an auto mode default: pushing to ANY branch of the working repository, including the default branch, is now allowed without a prompt. Previously only the starting branch, Claude-created branches, and routine default-branch pushes were allowed; before v2.1.203 any direct push to the default branch was blocked. Branches whose names mark them as deploy or publication targets (production, release, gh-pages) are explicitly NOT covered and are still judged on their own terms. Guidance written against the older, stricter push default is now wrong.
- [claude-hooks-permissions] v2.1.211 removed the 'Default / protected branches' context slot from `autoMode.environment`, which previously treated main and master as protected until you named others. Configuration examples that set that slot are now obsolete.
- [claude-hooks-permissions] v2.1.211 added a new default block rule: committing or pushing a change that would send secrets or sensitive data outside the repository when it runs, or that widens what a deploy exposes. It applies on any branch, applies even when the repository is public, and fires when the change lands whether or not the landing triggers the pipeline.
- [claude-hooks-permissions] v2.1.212 added `claude auto-mode reset`, which removes the `autoMode` section from `~/.claude/settings.json` (confirmation prompt, `--yes` to skip). Managed-settings and `--settings autoMode` rules survive the reset.
- [claude-hooks-permissions] v2.1.210 changed the classifier model: it now runs on Claude Sonnet 5 by default rather than following the session's /model selection, with a server-side Anthropic override taking precedence. Fallbacks apply when the session model is Sonnet 4.6, when `availableModels` excludes Sonnet 5, or when the session runs on Fable 5. This directly stales third-party write-ups describing the second-stage classifier as Sonnet 4.6.

- [claude-hooks-permissions] v2.1.208 extended the `rm -rf /` and `rm -rf ~` circuit breaker to fire on command substitution (`$(...)` and backticks) and process substitution (`<(...)`), whether the removal sits inside the substitution or elsewhere in the command. Before v2.1.208 those forms went to the classifier instead of prompting. Same release added `claude auto-mode defaults --label`.
- [claude-hooks-permissions] v2.1.207 removed the `CLAUDE_CODE_ENABLE_AUTO_MODE=1` requirement on Amazon Bedrock, Google Cloud Agent Platform, Microsoft Foundry and Claude apps gateway sessions (it had been required from v2.1.158 through v2.1.206; the variable is still accepted but has no effect). The same release stopped the classifier reading `autoMode` from `.claude/settings.local.json`, and fixed hook `if` conditions so file-path patterns like `"Edit(src/**)"` match as documented.
- [claude-hooks-permissions] v2.1.205 added an auto mode block rule covering writes to Claude Code's own session transcripts (the `.jsonl` files under `~/.claude/projects/`), directly or via shell. Reading transcripts is not blocked. Rationale given is defense in depth: a tampered transcript entry reaches every later check once the session resumes.
- [claude-hooks-permissions] v2.1.200 relabeled the `default` permission mode to 'Manual' across the CLI, VS Code, JetBrains and desktop, and added `manual` as an accepted alias for the value (e.g. `--permission-mode manual`). The config value remains `default`, which is what hooks and SDK integrations continue to use - so hook input parsing on `permission_mode` should still expect the string 'default', not 'manual'.
- [claude-hooks-permissions] v2.1.211 added `"fork"` as a `SessionStart` hook source value, alongside `startup/resume/clear/compact`. Matcher configurations that enumerate `SessionStart` sources will not fire on forked sessions unless updated.
- [claude-hooks-permissions] v2.1.215 stopped Claude auto-invoking the `/verify` and `/code-review` skills; they must now be invoked explicitly. Relevant to any workflow that assumed those ran on their own.
- [claude-hooks-permissions] Tier-5 practitioner content in this focus area is measurably stale and in one case wrong. Search surfaced posts asserting a `PreToolUse` field named `modifyInput` (the reference documents `updatedInput`), a fixed count of 12 or 30 hook events (the reference's event table is larger and still growing), and a Sonnet 4.6 second-stage classifier (superseded by the Sonnet 5 default at v2.1.210). None of these were used to support a claim.
- [claude-context-memory] Current Claude Code release is v2.1.214 (2026-07-18). Four releases landed after the 2026-07-12 baseline: 2.1.208 and 2.1.210 (both 2026-07-14), 2.1.212 (2026-07-17), 2.1.214 (2026-07-18).
- [claude-context-memory] NEW in v2.1.214 (2026-07-18): an ISO `modified` timestamp is now added to memory file frontmatter. Directly relevant to this bucket — memory files now carry machine-readable recency, which makes freshness/staleness policies over auto memory implementable for the first time.
- [claude-context-memory] NEW in v2.1.212 (2026-07-17): token usage reduced in inter-agent messaging — `SendMessage` bodies are no longer duplicated into replayed history and tool results. Changes the context cost model for multi-agent setups; any prior measurement of inter-agent token overhead taken before 2026-07-17 is now stale.

- [claude-context-memory] NEW in v2.1.210 (2026-07-14): fixed `/context` reporting 0 tokens for all tool groups on Bedrock. Any Bedrock-based context accounting collected before this date is unreliable.
- [claude-context-memory] NEW in v2.1.208 (2026-07-14): several long-session memory leaks fixed (MCP stdio server stderr accumulating up to 64 MB per server; LSP documents now LRU-capped at 50 docs; async hook output retained after backgrounding; unbounded growth in headless/SDK sessions from large tool-result payloads). Also: file edit read cache now bounded to 16 MB instead of pinning up to 1,000 full files, and session transcript size reduced up to 79x in edit-heavy sessions. These are process-memory changes, not context-window changes — worth recording precisely because 'memory' in these entries does NOT mean MEMORY.md, and the two are easy to conflate in this bucket.
- [claude-context-memory] NEW in v2.1.208: markdown tables over 200 rows now render only the first 200 with a '... N more rows' notice — a new truncation boundary that can silently drop tool output rows from what a reviewer sees.
- [claude-context-memory] Documented behavior predating the baseline but not previously captured in this bucket: v2.1.198 made compaction summarization inherit the session's extended thinking configuration; v2.1.206 added a `/doctor` trim proposal for checked-in CLAUDE.md; v2.1.207 fixed invalid glob bracket patterns in `paths:` frontmatter breaking the Read tool for every evaluated file; v2.1.211 changed path-scoped and nested rules to respect `project` exclusion in `--setting-sources` (before v2.1.211 they loaded even when project was excluded).
- [claude-context-memory] URL drift: www.anthropic.com/news/context-management now issues a 308 permanent redirect to claude.com/blog/context-management. Anthropic content is migrating off anthropic.com onto claude.com; any registered anthropic.com/news or anthropic.com/engineering URLs should be re-resolved, including the assigned source anthropic.com/engineering/effective-context-engineering-for-ai-agents.
- [claude-context-memory] Default-behavior drift worth flagging for guidance rewrites: MCP tool schemas are now deferred by default with only tool names in context. Older repo guidance about pruning MCP servers primarily to save context is weakened by this default.
- [claude-agents-skills] NEW since 2026-07-12 baseline: v2.1.212 (2026-07-17) added a per-session subagent spawn cap, default 200, override `CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION`, reset by `/clear`. First hard first-party ceiling on delegation fan-out.
- [claude-agents-skills] NEW since baseline: v2.1.215 stopped Claude from auto-invoking the `/verify` and `/code-review` skills; they now require explicit `/verify` or `/code-review`. Breaks any workflow assuming spontaneous verification.
- [claude-agents-skills] NEW since baseline: v2.1.211 — subagents (and context compaction) now inherit the session's extended thinking configuration; background subagent results now reach Claude only via a completion notification, so Claude no longer reports results for a subagent that has not finished; `/usage` session totals now reset on `/clear` instead of accumulating for the process lifetime.
- [claude-agents-skills] NEW since baseline: v2.1.210 security fix — isolation: 'worktree' subagents could previously run git-mutating commands against the main repo checkout. Any repo guidance

recommending worktree isolation as a containment boundary must require v2.1.210+.

- [claude-agents-skills] Recent but pre-baseline, newly recorded this pass: v2.1.207 changed agent-team mailbox handling so malformed entries are reported and removed while valid messages still deliver; before v2.1.207 a single malformed entry caused a repeated per-second error and blocked that mailbox until manually deleted.
- [claude-agents-skills] Recent but pre-baseline: v2.1.199 changed idle-teammate row behavior in the agent panel (idle rows now hide only once every agent is idle, and /model or /fast typed while viewing a teammate now shows a notice that the change applies to the lead), and stacked slash-skill invocations now load up to 5 leading skills instead of only the first.
- [claude-agents-skills] Default flip to watch: teammateMode default changed from "auto" to "in-process" at v2.1.179, so sessions upgraded from earlier versions that previously opened split panes now stay in one terminal unless the mode is set explicitly.
- [claude-agents-skills] API removal to watch: v2.1.178 removed the TeamCreate and TeamDelete tools; every session now has one implicit team when `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`. The `team_name` input on the Agent tool and the `team_name` field in TaskCreated / TaskCompleted / TeammateIdle hook payloads are accepted but deprecated and now carry a session-derived name.
- [claude-agents-skills] Default flip already in force: v2.1.198 made subagents run in the background by default; previously Claude chose per task. Also at v2.1.198 the built-in Explore agent began inheriting the main session's model (capped at opus) instead of running on haiku, which raises Explore's cost profile.
- [claude-agents-skills] Registered source staleness: the tembo.io orchestration post still cites 'Claude Code v2.1.32 or later' as the agent-teams minimum and documents 'Shift+Down' for teammate navigation; current tier-1 docs describe post-v2.1.178 setup-free spawning and up/down arrow keys. Recommended for retirement.
- [codex-config-hooks] DOCS HOST MIGRATION: developers.openai.com/codex/ now 308-redirects to learn.chatgpt.com/docs/. Two of the five assigned sources were affected. Old links still resolve, so this is silent drift that would not show up as a fetch failure.
- [codex-config-hooks] PATH CHANGE, not just a host swap: config-advanced moved from /codex/config-advanced to /docs/config-file/config-advanced, gaining a /config-file/ segment. The hooks page did NOT gain an equivalent segment (/docs/hooks), so the two pages migrated under different path conventions -- do not pattern-rewrite registry URLs mechanically.
- [codex-config-hooks] REGISTRY DUPLICATION EXPOSED: after redirect resolution, assigned sources #1 (developers.openai.com/codex/hooks) and #4 (learn.chatgpt.com/docs/hooks) are the same page. The bucket has been spending two fetches per cycle on one document.
- [codex-config-hooks] RELEASE CADENCE IN WINDOW: five stable patch releases shipped between the 2026-07-12 baseline and today -- 0.144.2 and 0.144.3 (2026-07-13), 0.144.4 (2026-07-14), 0.144.5 (2026-07-16), 0.144.6 (2026-07-18). Baseline version anchor should move to 0.144.6.
- [codex-config-hooks] NEXT MINOR IN PRERELEASE: the 0.145.0 alpha line is active and iterating fast (0.145.0-alpha.20 through alpha.24 all within 2026-07-17/18). Config and hook surfaces in this bucket should be re-verified when 0.145.0 reaches stable.

- [codex-config-hooks] NO SUBSTANTIVE CHANGE TO THE BUCKET'S SUBJECT MATTER: no changelog entry dated on or after 2026-07-12 touches hooks, config.toml, AGENTS.md, project_doc, trust, or requirements.toml. The hook event vocabulary, handler types, trust model, and AGENTS.md hierarchy all appear stable across this window. The releases in the window were bug fixes (context-window corrections, dangerous-command detection, TUI rendering).
- [codex-config-hooks] PRIOR-WINDOW HOOK ACTIVITY (context, pre-baseline): 0.143.0 (2026-07-08) relaxed hooks.json top-level metadata validation and 0.144.0 (2026-07-09) emitted canonical hook prompt items. Both predate the 2026-07-12 baseline but indicate the hooks subsystem was being actively changed immediately before this window.
- [codex-exec-automation] VERSION DRIFT: Codex CLI stable advanced from the 0.144.2/0.144.3 line (on/around the 2026-07-12 baseline) to 0.144.6, released 2026-07-18. A 0.145.0-alpha line is active, reaching 0.145.0-alpha.24 on 2026-07-18.
- [codex-exec-automation] 0.144.6 (2026-07-18) refreshed bundled instructions for GPT-5.6 Sol, Terra and Luna and CORRECTED their context windows to 272,000 tokens. Any context-budget arithmetic pinned before this release used a wrong window value.
- [codex-exec-automation] 0.144.5 (2026-07-16) expanded dangerous-command detection to more forced `rm` forms and added clearer rejection reasons. This is a behaviour-narrowing change: unattended automation that previously executed forced deletes may now be denied.
- [codex-exec-automation] 0.144.4 (2026-07-14) and 0.144.3 (2026-07-13) are maintenance/version-only releases with no user-facing changes.
- [codex-exec-automation] 0.144.2 (2026-07-13) ROLLED BACK a prompting regression, restoring the previous Guardian auto-review policy, request format and tool behaviour. Any observation of Guardian behaviour recorded between 0.144.0 and 0.144.2 may reflect the regressed prompt, not intended behaviour.
- [codex-exec-automation] HOST MIGRATION: developers.openai.com/codex/ *now issues 308 Permanent Redirects to learn.chatgpt.com/docs/*. Confirmed for `/codex/cli/features -> /docs/codex/cli`, `/codex/cli/reference -> /docs/developer-commands?surface=cli`, and `/codex/config -> /docs/config`. Every registered developers.openai.com/codex URL in this bucket needs its URL rewritten. Note the blog path ([developers.openai.com/blog/...](https://developers.openai.com/blog/)) did NOT redirect and still resolves directly.
- [codex-exec-automation] FEATURE DEPRECATION: Codex custom prompts (`~/codex/prompts/`, `/prompts:<name>`) are now explicitly deprecated in favour of skills. This changes the recommended answer to 'how do I package a reusable command' from custom prompts to `.agents/skills/SKILL.md`, and makes skills the surface that supports implicit invocation.
- [codex-exec-automation] REGISTRY SCOPE ERROR (not a vendor change): the registered 'developer-commands' source does not document user-authored custom commands at all - it is a built-in slash-command and CLI-subcommand reference. If the bucket's 'developer/custom commands' focus was resting on this URL, that focus was pointed at the wrong page; build-skills is the correct target.
- [codex-exec-automation] BROKEN DOCS LINK: learn.chatgpt.com/docs/config returns HTTP 404 while developers.openai.com/codex/config 308-redirects to it - a redirect into a dead page. The

authoritative approval/sandbox config reference is currently unreachable by URL, likely collateral from the host migration.

- [codex-exec-automation] TIER-5 ACCURACY FLAG: the registered danielvaughan archiving post asserts archived sessions are moved to `~/.codex/archived_sessions/` and that session files are world-readable (mode 0644). The first conflicts with the tier-1 PR (archive is an RPC-driven metadata flag) and with the post's own later text; the second is uncorroborated. Neither was promoted to a claim.
- [codex-subagents-sandbox] DOC DOMAIN MIGRATION: all `developers.openai.com/codex/` URLs now HTTP 308 to `learn.chatgpt.com/docs/`, and the path structure changed too (`/codex/subagents` → `/docs/agent-configuration/subagents`; `/codex/changelog` → `/docs/changelog`; `/codex/config-reference` → `/docs/config-file/config-reference`). Every registered OpenAI Codex doc URL in the manifest needs rewriting, not just a host swap.
- [codex-subagents-sandbox] SANDBOX BACKEND: the current `codex-rs/linux-sandbox` README describes bubblewrap (bwrap) as the primary Linux backend with Landlock demoted to a legacy opt-in behind `features.use_legacy_landlock`. Widely circulated third-party writeups (and search-result summaries) still describe Codex on Linux as 'Landlock plus seccomp'. Any repo guidance carrying the Landlock framing is stale.
- [codex-subagents-sandbox] RELEASES IN WINDOW (since 2026-07-12): 0.144.3 (2026-07-13), 0.144.5 (2026-07-16, expanded dangerous-command detection with clearer rejection feedback), 0.144.6 (2026-07-18, stable). Pre-releases 0.145.0-alpha.22/.23/.24 landed 2026-07-17/18, indicating a 0.145.0 line is in flight.
- [codex-subagents-sandbox] CONTEXT WINDOW CHANGE: 0.144.6 corrected GPT-5.6 Sol / Terra / Luna context windows to 272,000 tokens. A user issue (#32806, filed 2026-07-13 against 0.144.3, now closed) reported this as a drop from a previously observed 372K raw. Any cost/budget figures in the repo keyed to a larger GPT-5.6 window are now wrong.
- [codex-subagents-sandbox] NEW SUBAGENT DEFECT REPORTS IN WINDOW: #33039 (2026-07-14) and #33447 (2026-07-15), both open and both alleging Codex Desktop caps concurrent agent threads at 4 regardless of `agents.max_threads`. Neither has maintainer confirmation. This is the first sign that the documented concurrency knob may not be authoritative on every surface.
- [codex-subagents-sandbox] SUBAGENT ERROR PROPAGATION: recent release notes indicate parent agents now receive terminal subagent errors instead of seeing failed work as an empty successful completion, and subagent MCP startup warnings now stay in the owning thread. Relevant to this repo's `silent_defect_rate` work — a previously silent failure mode in Codex orchestration appears to have been closed. Exact version not pinned; verify before relying on it.
- [codex-subagents-sandbox] MCP AUTH: changelog entry for 0.144.0 (2026-07-09, just before the baseline) states MCP tools can now request authentication interactively without an experimental opt-in. Just outside the window but likely not yet reflected in repo guidance.
- [security-sandboxing-injection] Claude Code sandbox settings surface moved substantially within the v2.1.x line, and the official docs carry explicit min-version markers that make the drift auditable: `sandbox.credentials` requires v2.1.187+; per-session host allow-on-approval (choosing Yes allows the host for the rest of the session rather than re-prompting) landed in v2.1.191; `"mode": "mask"` credential substitution and the experimental `network.tlsTerminate` setting require v2.1.199+;

symlink resolution for protected settings-file deny rules landed in v2.1.210; plan-mode behaviour changed in v2.1.212.

- [security-sandboxing-injection] Behavioural change to flag for any repo guidance written before v2.1.212: a bare `Bash` ask rule (or the `Bash(*)` form) is skipped for commands that run sandboxed, BUT as of v2.1.212 the rule is no longer skipped in plan mode — it now prompts for sandboxed commands including read-only ones. Correspondingly, auto-allow in plan mode now runs only read-only commands without a prompt. Before v2.1.212 both skips applied in plan mode too, so guidance authored against the older behaviour now over-promises silence.
- [security-sandboxing-injection] `network.tlsTerminate` is new and still labelled experimental. It is the first Anthropic-shipped answer to the domain-fronting weakness in the default hostname-only allowlist, but the docs are explicit that it exists to enable credential masking and does NOT add content filtering — so it does not close the exfiltration gap on its own.
- [security-sandboxing-injection] OWASP's flagship agentic document changed edition and character since the 2025 baseline: 'State of Agentic AI Security and Governance' went v1.0 (July 2025) to v2.01 (June 2026), replacing a catalog of plausible threats with disclosed CVEs, vendor advisories and breach reports, and adding a live incidents/exploits tracker, revised agent taxonomy, enterprise adoption maturity model, and new sections on agent identity, AI SBOM, supply-chain provenance and regulatory landscape.
- [security-sandboxing-injection] New primary literature appeared inside the 2026-07-12 baseline window worth tracking: arXiv:2606.26479 (submitted 2026-06-25) is the first adaptive evaluation aimed specifically at out-of-band prompt-injection defenses (CaMeL, FIDES, Progent, RTBAS, FORGE), and reports a preliminary result that runs opposite to the in-band detection collapse documented in arXiv:2510.09023. Direction of the field is shifting from 'can we detect injections' to 'can deterministic enforcement survive adaptive attack'.
- [security-sandboxing-injection] Framework-level CVEs now exist for the prompt-injection-to-RCE path: CVE-2026-26030 (semantic-kernel Python <1.39.4) and CVE-2026-25592 (Semantic Kernel .NET SDK <1.71.0), disclosed 2026-05-07. Any pinned Semantic Kernel dependency in the estate should be checked against these floors.
- [security-sandboxing-injection] Two registered sandbox-comparison sources are being retired t3 to t5 this pass after inspection revealed they are vendor self-comparisons rather than neutral engineering reports. This is a registry-quality drift, not an upstream content change, but it changes what those entries may be cited for.
- [evals-judging] In-window change (post 2026-07-12): arXiv:2607.01211 'Are Performance-Optimization Benchmarks Reliably Measuring Coding Agents?' posted v2 on 2026-07-16 (v1 was 2026-07-01). Claims evals-judging-21 through -23 are drawn from the current version; if a 2026-07-12 baseline captured v1, the replay-validity and score-weight figures should be re-read against v2.
- [evals-judging] In-window change (post 2026-07-12): the BenchLM benchmark registry was last updated 2026-07-14 and now tracks 296 benchmark definitions across 10 categories. Tier 5, no primary citations, recorded as an index only.
- [evals-judging] Boundary item: arXiv:2607.10080 (multilingual injection against LLM relevance judges) was submitted 2026-07-11, one day before the window. Flagged as new material relative to

a 2026-07-12 baseline pass that would likely have missed it.

- [evals-judging] Recent but pre-window, and new relative to this bucket's prior coverage: arXiv:2607.05904 (2026-07-07, self-play judge reward hacking) and arXiv:2607.07189 (2026-07-08, ImagingBench). Neither appears to have been previously registered.
- [evals-judging] Substantive contradiction against practitioner guidance: the assigned tier-5 source treats verbosity bias as a first-order judge failure requiring length penalties or token normalization, but tier-2 measurement (arXiv:2606.19544, 2026-06-17) finds the effect below 0.011 under a single pairwise rubric. Position bias, by contrast, is confirmed as severe (>0.10) even in judges with >0.95 test-retest reliability. Any repo guidance that ranks verbosity bias above position bias should be revised.
- [evals-judging] Assigned source content drift: futureagi.com/blog/llm-as-a-judge was originally published 2025-01-29 and updated 2026-05-14. Its substance is now partly superseded by tier-2 measurement; its registered tier of 5 remains correct.
- [evals-judging] Emerging consensus shift worth tracking: three independent 2026 sources (arXiv:2607.05904, arXiv:2607.01211, sourcegraph/CodeScaleBench) converge on preferring held-out or execution-derived oracles over LLM-judge scoring wherever a deterministic check exists. The CodeScaleBench harness scores with patch-based and artifact verifiers and no LLM judge at all.
- [drift-discovery] ROUTING DRIFT (affects the registry directly): developers.openai.com/codex/ *now 308-redirects to learn.chatgpt.com/docs/*, with the /codex/ path segment DROPPED. Naive retargeting that keeps the segment produces 404s - verified: /docs/codex/config-file/config-reference 404s, /docs/config-file/config-reference resolves. Two registered OpenAI sources are affected.
- [drift-discovery] FETCH-METHOD DRIFT: the registered GitHub blob URL for claude-code CHANGELOG.md returns no changelog body via WebFetch (page chrome only, file is ~450 KB). Must use raw.githubusercontent.com. Any prior pass that 'checked' the changelog via the blob URL likely recorded a false no-change.
- [drift-discovery] SUMMARISATION HAZARD OBSERVED THIS PASS: a first-pass summary of the Claude Code changelog mis-attributed roughly a dozen 2.1.214 bullets to 2.1.215. A verbatim re-fetch showed 2.1.215 contains exactly ONE bullet. All version attributions in this report were re-derived from verbatim quotes. Future passes should quote changelog sections verbatim rather than accept a summarised version mapping.
- [drift-discovery] CLAUDE CODE: latest moved to 2.1.215 (npm dist-tag confirmed 2026-07-20; secondary trackers date it 2026-07-19). Six releases 2.1.210 through 2.1.215 with no 2.1.213.
- [drift-discovery] BEHAVIOURAL: 2.1.215 stopped Claude auto-running the /verify and /code-review skills. Any repo guidance assuming an implicit verification gate is now a silent no-op - highest-impact item in this pass for this repo specifically.
- [drift-discovery] SEMANTIC CHANGE, NOT A RENAME: /fork changed meaning in 2.1.212 (now copies the conversation to a background session); the old in-session forked-subagent behaviour became /subtask. Docs confirm /fork still exists and define three distinct behaviour bands by version ($\geq 2.1.212$, 2.1.161-2.1.211, $<2.1.161$), plus a fourth when agent view is disabled.

- [drift-discovery] DEPRECATION: the Task tool's 'mode' parameter is deprecated and now IGNORED as of 2.1.212; subagents inherit the parent session's permission mode. This fails silently - configs passing 'mode' get no error but a changed permission posture.
- [drift-discovery] NEW STRUCTURAL CAPS in 2.1.212: WebSearch calls capped at 200 per session (CLAUDE_CODE_MAX_WEB_SEARCHES_PER_SESSION) and subagent spawns capped at 200 per session (CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION, reset by /clear). The search cap directly constrains this repo's own weekly-research routine.
- [drift-discovery] SECURITY CLUSTER in 2.1.214 - four independent permission-check fail-closed fixes: single-segment dir/** allow rules over-approving nested dirs tree-wide; a bypass in Windows PowerShell 5.1 sessions; file-descriptor redirect forms; and commands over 10,000 characters. The PowerShell 5.1 bypass is directly relevant to this machine's primary shell. Treat $\geq 2.1.214$ as a floor wherever permission rules are a safety boundary rather than a convenience.
- [drift-discovery] EARLIER SAFETY FIXES worth version-gating in guidance: plan mode auto-ran file-modifying Bash commands without prompting or invoking the SDK canUseTool callback before 2.1.212 (plan mode was NOT read-only); isolation: 'worktree' subagents could git-mutate the main checkout before 2.1.210.
- [drift-discovery] SETTINGS HYGIENE: 2.1.210 added a startup warning for Write(path), NotebookEdit(path) and Glob(path) permission rules, directing authors to Edit(path)/Read(path). Actionable audit item for this org's settings.json files.
- [drift-discovery] DOCS/CHANGELOG CONTRADICTION: CLAUDE_CODE_OTEL_CONTENT_MAX_LENGTH is described as configuring a 60 KB truncation limit by both the changelog and the monitoring-usage page, but the env-vars reference renders a default of 5000 characters. Unresolved - no default was encoded into any claim.
- [drift-discovery] CODEX MODEL CATALOG: 0.144.6 (2026-07-18) corrected the context window to 272,000 tokens for GPT-5.6 Sol, Terra and Luna. The local catalog generated 2026-07-19 should already reflect this; if it carries an older figure it is stale on day one. Separately, the Codex config reference still uses gpt-5.5 and gpt-5.1 as its model-slug examples, which appears to lag the GPT-5.6 family named in the changelog.
- [drift-discovery] CODEX SAFETY: 0.144.5 (2026-07-16) expanded dangerous-command detection to additional forced rm forms, same-day as a tier-5 report of home-directory deletion under full-access mode with sandboxing and auto-review disabled. Pin $\geq 0.144.5$ for destructive-shell workloads.
- [drift-discovery] SOURCE-CHANNEL DRIFT: anthropic.com/engineering has published nothing since 2026-04-23 while Claude Code engineering content (e.g. the 2026-06-02 dynamic-workflows post) is going to claude.com/blog. Registry should add claude.com/blog, and consider code.claude.com/docs/en/changelog as a second tier-1 Claude Code changelog surface discovered this pass.
- [drift-discovery] MODEL AVAILABILITY: Claude Fable 5 reportedly became included in Max and Team Premium plans effective 2026-07-20 (today). Commercial terms are tier-5-sourced only and were deliberately not promoted into cost-ladder guidance.

Proposed repository changes

Full reviewable diff plan is in `proposed-changes.md`. Summary:

1. **Apply the source-registry patch** (52 add / 17 update / 8 retire).
2. **Repair a dead link in** `Claude/OPERATING_CONTRACT.md` - line 32 cites `https://code.claude.com/docs/en/agent-loop`, which now returns **404**; the page moved to `https://code.claude.com/docs/en/agent-sdk/agent-loop`. This was found by the run, not assumed.
3. **Fix five documentation inconsistencies** (README candidate status; missing dated `Research Outputs/` copy in `new-candidate.ps1`; `Research Outputs/README.md` calling the newest report 'the current state of the world'; `weekly-hygiene.md` not integrated into `/weekly-research`; and a stale blocker in `workstreams/current.json` claiming the model catalog expired 2026-07-19 when it was regenerated 2026-07-19 and expires 2026-07-26).
4. **Reconcile the 31 corrected claims** against the two operating contracts before promotion.

Eval and ablation plan

`eval-plan.json` defines 7 cases (6 blocking). **No provider spend is required**: this candidate proposes source-registry and documentation changes only, with no change to Runtime/stable behaviour, Router modules or adaptive tooling, so no arm-based provider campaign is needed to accept it. This respects the standing instruction to keep provider spend disabled.

Case	Name	Blocking
EV-01	Patch applies and is reversible	yes
EV-02	No stable document cites a retired URL	yes
EV-03	Dead agent-loop link is repaired	yes
EV-04	Corrected claims reconciled against existing contract text	yes
EV-05	Version-sensitive sample re-verification	yes
EV-06	Rejected claims stay out of guidance	yes
EV-07	Report source integrity	no

Expected impact

- **Source registry becomes trustworthy.** The 47-entry `pending-review` backlog is triaged for the first time, so future runs start from a reviewed inventory instead of re-litigating unvetted URLs.
- **Eight dead or derivative sources stop being cited**, including one URL that is already referenced by a repo document and now 404s.

- **242 version-sensitive mechanics are pinned to tier-1 evidence with explicit expiry dates**, converting 'someone read this once' into a recheck schedule.
- **No behavioural change to any agent** until promotion; the contracts and Stable are untouched by this run.

Risks

- **Volume.** 282 accepted claims is more than a single promotion should absorb at once; promote in bucket-sized slices so each diff stays reviewable.
- **Version sensitivity.** 242 claims are tied to versions/flags/keys that move between releases. They carry `expires_at`, but a promotion that lands after expiry must re-verify, not assume.
- **Absence claims.** Several claims assert that no flag or setting exists. Documentation silence is not proof; these are the most likely to age badly and are flagged in open questions.
- **Newly discovered sources (28) have not been through a full review cycle** - they enter as `active` on the strength of one pass and should be re-checked at their first due date.
- **Redirect churn at OpenAI.** Several `developers.openai.com/codex/*` paths now 308-redirect to `learn.chatgpt.com`; pinning the pre-redirect URL will rot.

Rollback

This run is inherently reversible: every artifact is confined to `Research/candidate-packages/2026-07-20-weekly-research/`. To abandon it, delete that directory - no other repo state was modified, and no commit or push was performed. If the patch is later promoted, rollback is the pre-patch `Research/sources/registry.json` restored from the promotion rollback manifest (EV-01 requires the restore to reproduce the original file hash before promotion is allowed).

Open questions

84 carried forward as the backlog for the next run.

- [claude-loops-scheduling] Version attribution conflict in the changelog: two independent reads of <https://code.claude.com/docs/en/changelog> disagreed on which release introduced `CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION` (v2.1.199 vs v2.1.212) and which fixed `/loop` hiding the session from `/resume` (v2.1.178 vs v2.1.211). The mechanics and the 200 default are consistent across both reads; only the introducing version is unresolved. Resolve against the anthropic/claude-code CHANGELOG.md in the repo before pinning any version number.
- [claude-loops-scheduling] Model-version conflict on context anxiety: one read of the Anthropic harness-design article attributes the elimination of premature-wrap-up behavior to Opus 4.6, another summary of the same body of work says Opus 4.5. Do not cite a model version for this until the article is re-read directly.
- [claude-loops-scheduling] Publication date of <https://claude.com/blog/getting-started-with-loops> is reported as both June 30, 2026 and July 7, 2026 by different retrievals. Confirm before using it as a

recency marker.

- [claude-loops-scheduling] ScheduleWakeup has no dedicated section in <https://code.claude.com/docs/en/tools-reference>; its stop: true parameter is documented only narratively in the scheduled-tasks page. Its full parameter schema (delay units, whether an explicit next-fire time can be passed) is undocumented in the sources reviewed.
- [claude-loops-scheduling] Does the v2.1.215 change (Claude no longer self-invoking /verify and /code-review) actually cause '/loop <interval> /verify' to arrive as plain text rather than execute? This follows logically from the v2.1.196 scheduled-fire rule but is not stated anywhere. Needs a one-session empirical check.
- [claude-loops-scheduling] /goal has no harness-enforced turn or budget ceiling; the documented bound is a natural-language clause inside the condition, judged by the evaluator. Unknown whether any hard ceiling exists as a settings key or env var. If none exists, that asymmetry versus the SDK's max_turns/max_budget_usd is itself worth recording as a risk.
- [claude-loops-scheduling] Unknown whether the /goal evaluator's small-fast-model choice is configurable independently of the session's general small-fast-model setting, and what the per-turn token cost actually is in practice. The docs say only that it is 'typically negligible' relative to main-turn spend, which is an unquantified vendor claim.
- [claude-loops-scheduling] No tier-1 or tier-2 source found for the token-multiplier figures circulating in tier-5 material (single agents ~4x standard chat, multi-agent ~15x). If the repo wants a cost-ladder number for loop overhead, it likely has to be measured locally rather than cited.
- [claude-loops-scheduling] Routines research-preview caps are not published as numbers: the docs say GitHub webhook events are subject to per-routine and per-account hourly caps and that there is a daily routine-run cap, but the values are only visible in the authenticated UI at claude.ai/code/routines. Any capacity planning guidance needs the user to read their own limits.
- [claude-hooks-permissions] Exact total count of hook events is unresolved. One extraction pass over the reference reported '33 hook events' while a second reported no explicit count and enumerated 31 with one duplicate. No claim asserting a count was emitted. Resolving this needs a direct read of the reference's event table rather than a summarized fetch.
- [claude-hooks-permissions] Whether the reference states a normative firing ORDER between PreToolUse and PermissionRequest within a single tool call, or merely depicts one in the lifecycle diagram. The guide's statement that PreToolUse fires before any permission-mode check is firm and was used; the finer intra-tool-call ordering was not asserted. Worth confirming directly, since a guardrail that assumes PermissionRequest can see a PreToolUse decision would depend on it.
- [claude-hooks-permissions] Whether ConfigChange blocking genuinely exempts the `policy_settings` source. This appeared in one extraction of the reference's blocking table and nowhere else; ConfigChange was included in the block-capable list (corroborated by the guide) but the exemption was deliberately not claimed.
- [claude-hooks-permissions] Blocking semantics for the newer agent-team and task events - TeammateIdle, TaskCreated, TaskCompleted - rest on a single extraction of the reference's blocking table. These look like recently added surfaces and deserve independent confirmation before any guidance leans on them.

- [claude-hooks-permissions] Whether the engineering post's measured rates (0.4% false positive on real traffic after both stages, 17% false negative on real overeager actions, 5.7% on synthetic exfiltration) still describe the current classifier, given the model default moved to Sonnet 5 at v2.1.210. The post does not appear to be versioned, so these numbers may describe an earlier configuration.
- [claude-hooks-permissions] Precise dates for releases v2.1.212 through v2.1.215. The changelog is undated, so nothing in this pass can be firmly attributed to the window since 2026-07-12 versus somewhat earlier. A dated release feed would make future drift detection materially sharper.
- [claude-hooks-permissions] Whether hooks defined in skill or agent frontmatter are subject to the same deny-beats-bypassPermissions precedence as hooks from settings files. The precedence statement in the guide is not explicitly scoped by configuration source, and the answer matters for whether a skill can impose a boundary its invoking session cannot lift.
- [claude-context-memory] Version conflict on the memory index limit check: the memory docs annotate loaded-content-only measurement (frontmatter and HTML comments excluded) as min-version 2.1.211, but the changelog credits 'Improved the memory index over-limit warning to measure only loaded content, excluding frontmatter and HTML comments' to 2.1.193 (2026-06-25). One of the two is wrong. Claim 23 states both; resolve before any guidance depends on the exact version.
- [claude-context-memory] No published auto-compaction percentage exists for the standard 200K case. Docs give only a token figure for Sonnet 5 (~967K on a 1M window) and describe the 200K boundary qualitatively. The widely repeated '92%' / '95%' auto-compact figures from practitioner sources could not be confirmed against any tier-1 page this pass — deliberately omitted rather than asserted.
- [claude-context-memory] 'Microcompact' could not be confirmed as a current, documented Claude Code feature name. The behavior it usually refers to (clearing older tool outputs before summarizing) IS documented on how-claude-code-works, but under no product name, and no DISABLE_MICROCOMPACT env var appears in the env-vars docs. Do not use the term in guidance until a tier-1 source names it.
- [claude-context-memory] A search result referenced a fix for 'the context window (and auto-compact indicator) briefly resetting to 200k after the CLI auto-updates, causing a false 100% context used when resuming long-context sessions.' This entry did not appear in the changelog range fetched (2.1.190+), so the version could not be pinned. Omitted from claims; re-check next pass.
- [claude-context-memory] ENABLE_TOOL_SEARCH is documented on the context-window page (with values auto and false, and a 10%-of-context budget) but did not surface in the env-vars page fetch. Whether it is a true environment variable, a settings key, or documented in only one place is unresolved — the 10% figure in claim 24 rests on a single page.
- [claude-context-memory] MAX_MCP_OUTPUT_TOKENS was searched for and not found in current env-vars docs. If prior repo guidance references it, that reference may be stale; verify before relying on it.
- [claude-context-memory] Anthropic's 29%/39%/84% context-management figures come from an undisclosed internal agentic-search eval with no released task set, baseline definition, or variance.

Given this repo's own measured nulls on scaffolding-buys-correctness claims, these numbers should not be adopted as evidence for context-management lift without an independent held-out replication.

- [claude-context-memory] The '200 lines' CLAUDE.md target and the assertion that longer files 'reduce adherence' are stated by Anthropic without any published measurement. This is a directly testable claim on our existing fixtures and would be a strong candidate for an internal replication rather than a cited guidance line.
- [claude-agents-skills] Version attribution conflict, unresolved: the sub-agents doc annotates background-subagent permission prompts surfacing in the main session as 'As of v2.1.186', while a fetch of the GitHub CHANGELOG mirror attributed the same change to v2.1.208. No claim asserts a version for this behavior; resolve by reading the raw CHANGELOG entries for both versions verbatim next pass.
- [claude-agents-skills] Does the v2.1.212 CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION cap (default 200) count agent-team teammates, or only Agent-tool subagents? The changelog says 'subagent spawns' and teammates are spawned via the Agent tool, which suggests yes, but no source states it. Do not assert either way.
- [claude-agents-skills] Full subagent scope precedence ordering is only partially verified: the docs table shows project .claude/agents/ at priority 3 and user ~/.claude/agents/ at priority 4, and states managed subagents outrank both, but priorities 1 and 2 were not read directly. Confirm whether 1 = managed and 2 = CLI --agents.
- [claude-agents-skills] Publication dates for the two blog sources are fetch-derived and unconfirmed: the Anthropic Skills post and the tembo.io post were both reported as 2026-06-03, which is suspicious enough to warrant direct verification before any date-sensitive use.
- [claude-agents-skills] The ~7x agent-team token multiplier is stated only for teammates running in plan mode. No first-party figure exists for teams running in normal execution mode, so the cost ladder has a gap exactly where most real work happens.
- [claude-agents-skills] No coding-specific measurement of Claude Code agent teams versus a single session was found anywhere. The strongest quantitative evidence (Google Research) uses Finance-Agent, BrowseComp-Plus, PlanCraft and Workbench, none of which is a software-engineering benchmark. Given this repo's track record of nulls when scaffolding claims meet held-out coding tasks, the coding transfer of the +80.9% / -39-70% task-shape result should be treated as an open empirical question and a candidate for a diff-derived-oracle replication rather than promoted as guidance.
- [claude-agents-skills] Anthropic's docs recommend 3-5 teammates and 5-6 tasks per teammate but cite no measurement behind either number. Whether these are tuned values or round-number heuristics is unknown.
- [claude-agents-skills] Agent teams provide no documented equivalent of the subagent isolation: worktree field, so the docs' own 'two teammates editing the same file leads to overwrites' hazard has no mechanical mitigation on the teams path — only prompt-level file-ownership partitioning. Worth checking each pass whether a teammate isolation mechanism ships.

- [codex-config-hooks] CONFLICT BETWEEN TWO OFFICIAL PAGES on project_doc_max_bytes semantics: config-advanced describes it as "Max bytes read per AGENTS.md file" (per-file cap) while the AGENTS.md page describes it as governing the combined accumulated instruction set and says Codex "stops adding files once the combined size reaches the limit". These imply materially different truncation behavior in a monorepo with many nested files. Needs empirical resolution against 0.144.6 before any sizing guidance is promoted.
- [codex-config-hooks] Which exact Codex version marked hooks as stable and introduced inline [hooks] tables in config.toml plus managed hooks via requirements.toml? Multiple tier-5 aggregators assert a July 2026 stabilization, but I could not confirm a version number or date from the official changelog, so no claim was emitted. Needs a targeted changelog search of the 0.140.x-0.143.x range.
- [codex-config-hooks] Is issue #20204 (PreToolUse fires for only shell/unified_exec/apply_patch/mcp) still accurate against 0.144.6? Opened 2026-04-29 and still open with no maintainer response found. If it still reproduces, hook-based audit coverage has a real hole around filesystem reads. Highest-value empirical test in this bucket.
- [codex-config-hooks] The Hooks page states matcher availability "varies by event" but does not enumerate which events accept a matcher and which ignore it. Observed matcher values include tool names (Bash, apply_patch), compaction triggers (manual, auto), and session sources (startup, resume, clear, compact), but the per-event mapping is unspecified.
- [codex-config-hooks] Exact JSON output schema per event is described as having "event-specific shapes" but was not captured field-by-field in this pass. Also unresolved: which output fields are unsupported per event -- there is an indication that continue, stopReason, and suppressOutput are not supported for PreToolUse and PermissionRequest, but this came from a search summary and was not verified against the page, so no claim was emitted.
- [codex-config-hooks] Does the hook trust hash cover only the command string, or the full hook definition including matcher and timeout? Matters for whether editing a matcher silently untrusts a hook.
- [codex-config-hooks] Whether project-scoped .codex/config.toml can define hooks that inherit project trust, or whether each hook still requires separate hash-level trust on top of project trust. Claims 07 and 20 describe two gates but the interaction between them is not documented explicitly.
- [codex-config-hooks] The Configuration Reference does not document experimental_use_rmcp_client (specifically checked, not present). Unclear whether it was removed, renamed, or was never a real key -- no claim emitted.
- [codex-exec-automation] Does `--ask-for-approval` exist as a CLI flag, and what are its valid values (never / on-request / on-failure / untrusted)? Multiple tier-3/tier-5 aggregators assert `--ask-for-approval never` for unattended runs, but I could not confirm the flag name or value set in any tier-1 page this pass - the config reference 404s and the non-interactive page did not list it. Deliberately omitted rather than asserted, to avoid inventing a flag.
- [codex-exec-automation] What is the DEFAULT sandbox mode for `codex exec` when `--sandbox` is not passed? Tier-3 material claims read-only; tier-1 docs list the three modes but did not state the

exec default in the pages reachable this pass. Material for guidance, since it determines whether an unflagged automation can write.

- [codex-exec-automation] What are the canonical valid values of the `approval_policy` config key? Only the value "never" is tier-1 confirmed (via the automations page). The full enum is unverified.
- [codex-exec-automation] Where do archived sessions physically live, and does archiving move files or only set a metadata flag? PR #25021 implies an RPC-driven metadata flag; the tier-5 post claims a move to `~/.codex/archived_sessions/`. Unresolved - resolve in-repo before documenting any path.
- [codex-exec-automation] Are Codex session/rollout files created with world-readable permissions (mode 0644) on Unix? Asserted at tier 5 only. If true this is a real operational security finding worth its own claim; if false it is a fabrication worth flagging against that source. Needs direct filesystem verification, not another web source.
- [codex-exec-automation] Which Codex CLI release shipped `codex archive` / `codex unarchive` to users? The PR merged 2026-05-30 and the tier-5 post says v0.136.0 on 2026-06-01, but neither the official changelog excerpt nor the GitHub releases page I read confirmed the version-to-feature mapping.
- [codex-exec-automation] Do skills load and function inside `codex exec` non-interactive runs, and does implicit invocation apply there? The build-skills documentation does not address non-interactive mode. This materially affects whether skills are a viable automation packaging mechanism or an interactive-only convenience.
- [codex-exec-automation] Is `codex fork` a documented top-level CLI subcommand? It appears in PR #25021's error-handling description and in tier-5 material, but it was not in any tier-1 command list I read.
- [codex-exec-automation] Does the exec JSONL stream define stable exit codes, and are they documented anywhere? Neither the tier-1 non-interactive page nor the DeepWiki entry enumerated exit codes, yet CI wrappers need them.
- [codex-exec-automation] What is the actual review-queue behaviour for runs with NO findings - are they auto-archived? A tier-3 aggregator claims automatic archiving of empty runs and bulk mark-as-read/archive actions; the tier-1 automations page documents the unread indicator and manual mark-all-as-read but did not confirm auto-archiving.
- [codex-subagents-sandbox] Is `features.multi_agent_v2.max_concurrent_threads_per_session` a real config key? It appears only in the body of unconfirmed issue #33447 and does NOT appear in the official config reference. Not asserted as a claim. Needs verification against source or a maintainer statement before it goes anywhere near guidance.
- [codex-subagents-sandbox] What is the DEFAULT value of `default_tools_approval_mode`? The MCP page enumerates auto/prompt/writes/approve but does not state which is default. This matters for the security posture of an unconfigured MCP server.
- [codex-subagents-sandbox] Has the auto-compaction regression (#16033) been fixed in any 0.14x release? The issue is still open with no linked PR, but the changelog does not clearly cover 0.144.5/0.144.6, so a silent fix cannot be ruled out. Needs a direct test or a maintainer comment.

- [codex-subagents-sandbox] Exact semantics of `agents.job_max_runtime_seconds`. The docs gloss it as a per-worker default timeout tied to 'CSV jobs', which reads like a narrow batch-mode feature rather than a general subagent timeout. Whether it bounds ordinary subagent runs is unclear — do not present it as a general watchdog until confirmed.
- [codex-subagents-sandbox] What actually distinguishes the built-in `worker` agent from `default`? Neither the tier-1 docs nor the tier-5 post draw a crisp line; the practitioner source explicitly flags this as uncertain.
- [codex-subagents-sandbox] Sandbox escape claims: tier-5 blog material references a 'CBSE (Configuration-Based Sandbox Escape)' attack class and CVE-2025-59532 against agent sandboxes. NOT asserted — no tier-1/2 corroboration found and no matching OpenAI security advisory located. If the repo wants a Codex threat model, this needs verification against the CVE database and openai/codex security advisories directly.
- [codex-subagents-sandbox] Confirm the release date year for rust-v0.144.6. The releases index reported 2026-07-18 13:51 UTC; a second fetch of the tag page transcribed the year as 2024. Treated as 2026 based on the index and surrounding version numbering, but worth a direct re-check.
- [codex-subagents-sandbox] Does `agents.<name>.config_file` allow pointing at agent definitions outside `.codex/agents/`? If so it is a supply-chain surface worth documenting, since agent files can override `sandbox_mode`.
- [security-sandboxing-injection] Is the current Claude Code release later than v2.1.212, and did further sandbox keys or defaults land after it? The official docs expose min-version markers only up to v2.1.212, so anything newer is invisible from the docs page alone. Confirm against the changelog/releases before treating the settings surface in claims 03-08 as complete.
- [security-sandboxing-injection] The registered tier-5 playbook attributes PromptArmor's sub-1% AgentDojo FPR/FNR to 'ICLR 2026'. The arXiv preprint is 2507.15219 (July 2025) and an OpenReview forum entry exists under a different title ('PromptArmor: An Essential Baseline for Prompt Injection Defenses'), but the publication venue and acceptance status were not confirmed. Do not cite a venue until verified.
- [security-sandboxing-injection] Why does amux.io return 403 — bot/UA filtering, geo-blocking from this host, or removal? Determines whether the retire decision is correct or whether the source should be re-fetched through another path.
- [security-sandboxing-injection] Sophos names several concrete implementations that were not independently verified in this pass: 'OpenClaw's per-session Docker containers', 'Agent Vault (open-source)' for credential isolation, and the 'cedar-for-agents' repository. Confirm these exist and are maintained before any of them enters guidance as a recommended tool.
- [security-sandboxing-injection] Sophos cites a Google April 2026 Common Crawl study reporting a '32% increase in malicious attempts' between Nov 2025 and Feb 2026, and separate secondary sources cite a '340% year-over-year' surge in prompt injection attacks attributed to an OWASP 2026 LLM Security Report. Neither underlying study was located directly. Both numbers are currently uncorroborated and should not be quoted.

- [security-sandboxing-injection] Northflank's '97ms median time-to-interactive with 100% success' is attributed to a ComputeSDK benchmark dated July 2026 on a page otherwise presenting as January 2026. Is ComputeSDK an independent benchmarking effort or vendor-affiliated, and is the methodology published? Determines whether any hosted-sandbox performance figure can ever be cited.
- [security-sandboxing-injection] Does the repo estate actually run Claude Code under WSL2 anywhere? Claims 01-09 describe controls that cannot engage on native Windows, which is this machine's platform. If all agent work is native-Windows, the built-in Bash sandbox is inert and the bucket's practical guidance must pivot to container/VM isolation instead — this materially changes which claims are actionable.
- [evals-judging] RuVerBench per-judge-model accuracy, F1 and agreement figures are not present on the arXiv abstract page or the repo landing page. The paper references a main leaderboard; a full-text PDF pass is needed before any per-model judge-reliability number can be asserted.
- [evals-judging] Majority-voting vote counts in RuVerBench (where the diminishing returns set in: 3, 5, 7 samples?) are unspecified in the abstract. The practical question for a repo policy is the smallest k that captures most of the gain; unanswered.
- [evals-judging] CodeScaleBench task-count discrepancy: the blog states 370 tasks (150 SDLC + 220 Org) while the repo describes a 371-task published analysis set (csb-v1-mixed371) alongside canonical suites of 264-275 tasks. Which count corresponds to which frozen snapshot is unresolved; claim evals-judging-17 is hedged with 'roughly'.
- [evals-judging] CodeScaleBench appears to have been run publicly only with Claude Code plus Haiku 4.5, with five further harnesses (Codex, Cursor, Gemini, Copilot, OpenHands) described as supported but not reported. Whether the measured retrieval deltas hold across harnesses is unknown, which limits how far claim evals-judging-20 generalizes.
- [evals-judging] The self-play de-anchoring mitigation (commit-before-judging, false positives 0.719 to 0.012) was measured on GSM8K, a task with exact-match ground truth. Whether commit-first judging transfers to open-ended coding review, where the judge cannot cheaply produce its own reference solution, is untested and is the single most important follow-up for applying this bucket's findings to a coding-agent repo.
- [evals-judging] Benchmark naming conflict left unresolved: a tier-5 article describes 'Imaging-101' as 57 expert-verified tasks across six domains by Siyi Chen, Jiahe Ying and He Sun, while arXiv:2607.07189 is 'ImagingBench' with 20 tasks across five categories by Ethan Chung and colleagues. These appear to be different artifacts conflated in search results; neither should be cited as a coding-agent eval until disambiguated.
- [evals-judging] Deliberately omitted, needs primary verification: several frontier model identifiers surfaced only in search snippets or tier-5 aggregators (a dated GPT-5.2 build, a dated Claude Opus 4.5 build, a Gemini 3 Pro preview, and the aggregator's 'Claude Mythos 5' / 'Claude Fable 5' / 'GPT-5.6 Sol' names). None were confirmed at a tier 1-2 source, so no claim asserts any of them. Model IDs beyond the January 2026 knowledge cutoff should be resolved against official vendor documentation before entering the registry.

- [evals-judging] LongJudgeBench (arXiv:2606.01629) reportedly evaluates judges on outputs averaging over 9,000 tokens versus under 500 in prior benchmarks, which would bear directly on judging long agent trajectories, but the figure came from a search summary and could not be confirmed on the abstract page. Not asserted as a claim.
- [evals-judging] JudgeBiasBench was referenced in search results as showing frontier models exceeding 50 percent error rates on advanced bias tests. The primary source was not located this pass and the claim is not included; worth a targeted lookup next cycle given how directly it would bear on judge admission policy.
- [evals-judging] No source located this pass measures LLM-judge reliability specifically on code review or code-diff judging with human gold labels. RuVerBench's agentic-coding split (843 checklist items over trajectories) is the closest available, and it judges trajectory checklists rather than code correctness. This remains the principal evidence gap for the bucket.
- [drift-discovery] What is the actual documented default of `CLAUDE_CODE_OTEL_CONTENT_MAX_LENGTH`? The env-vars reference rendered '5000' characters while the changelog and monitoring-usage page both describe a 60 KB limit. The env-vars fetch was truncated mid-table, so the reading may be unreliable. Resolve before encoding any default.
- [drift-discovery] Do `CLAUDE_CODE_MAX_WEB_SEARCHES_PER_SESSION`, `CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION`, `CLAUDE_CODE_FORWARD_SUBAGENT_TEXT` and `CLAUDE_CODE_FORK_SUBAGENT` appear in the env-vars reference page? All four were confirmed from the changelog (and `CLAUDE_CODE_FORK_SUBAGENT` from the /fork docs entry), but the env-vars table fetch truncated before they could be located. Only `CLAUDE_CODE_MCP_AUTO_BACKGROUND_MS` and `CLAUDE_CODE_OTEL_CONTENT_MAX_LENGTH` were positively confirmed on that page.
- [drift-discovery] Exact release dates for Claude Code 2.1.210 through 2.1.214. The npm registry response truncated before the 'time' object for 2.1.x versions; only dist-tags.latest = 2.1.215 was confirmed. The CHANGELOG.md carries no dates at all. A dated source is needed to prove which of these six releases fall inside the 2026-07-12 window - currently only 2.1.215 has a (tier-5) date of 2026-07-19.
- [drift-discovery] What is the canonical post-migration URL for the OpenAI developer blog? `developers.openai.com/blog/` still serves content but returned no in-window posts and rendered implausible dates, while sibling paths on that host 308-redirect to `learn.chatgpt.com`.
- [drift-discovery] Is there a tier-1 Anthropic advisory, changelog entry or security page confirming that `web_fetch` link-following on external pages was disabled? Claim drift-discovery-28 rests on a single tier-5 disclosure with no CVE and no vendor confirmation, and the researcher reports no bounty was awarded.
- [drift-discovery] Is there a tier-1 Anthropic announcement page for the Fable 5 subscription inclusion effective 2026-07-20? Only tier-5 practitioner and press reporting was located; the tier-1 corroboration extends to the existence of the Fable model family, not the commercial terms.
- [drift-discovery] Are the gpt-5.5 and gpt-5.1 model slugs in the Codex config reference stale relative to the GPT-5.6 Sol/Terra/Luna family named in the changelog, or do both generations remain

concurrently selectable? No model IDs were asserted either way. This should be resolved against the local catalog before it expires 2026-07-26.

- [drift-discovery] CANDIDATE TIER-2 SOURCE, just outside the window: arXiv 2607.01087, 'Cheap Code, Costly Judgment: A Case Study on Governable Agentic Software Engineering' (Davis, Amusuo, Singla, Cakar, Davis; v1 2026-07-01, v2 2026-07-04). A 12-week case study of 420 KLOC production code plus 1.16 MLOC of tests/docs, proposing a 'governance conversion' theory in which controls are discovered reactively from failures surfaced during agentic work rather than derived from known requirements. Excluded from claims because it predates the 2026-07-12 boundary, but it is directly on-topic for this repo's governance-and-guardrails positioning and should be assessed next pass.
- [drift-discovery] Does code.claude.com/docs/en/changelog carry release dates that the GitHub CHANGELOG.md lacks? If so it is the better registered tier-1 source for Claude Code drift, since it would resolve the dating gap noted above.

Sources

128 unique sources, each listed once with tier and access date.

Tier 1 (85)

- [Add thread archive CLI commands by etraut-openai · Pull Request #25021 · openai/codex](#) - accessed 2026-07-20
- [Agent approvals & security | ChatGPT Learn](#) - accessed 2026-07-20
- [Agents Rule of Two: A Practical Approach to AI Agent Security - Meta AI](#) - accessed 2026-07-20
- [Anthropic Engineering index \(newest post Apr 23, 2026\)](#) - accessed 2026-07-20
- [Anthropic research, ending a subset of conversations \(linked from the 2.1.214 EndConversation bullet\)](#) - accessed 2026-07-20
- [Anthropic, 'A harness for every task: dynamic workflows in Claude Code' \(June 2, 2026, hosted on claude.com/blog\)](#) - accessed 2026-07-20
- [Anthropic, 'Claude Fable 5 on your plan' \(Claude Help Center\)](#) - accessed 2026-07-20
- [Anthropic, 'Redeploying Claude Fable 5' \(2026-06-30, superseded terms\)](#) - accessed 2026-07-20
- [Anthropic, Web fetch tool docs - exfiltration warning and URL validation section](#) - accessed 2026-07-20
- [anthropics/claude-code CHANGELOG.md](#) - accessed 2026-07-20
- [Automate actions with hooks - Claude Code Docs](#) - accessed 2026-07-20
- [Automate work with routines](#) - accessed 2026-07-20
- [Build skills | ChatGPT Learn](#) - accessed 2026-07-20
- [Built-in memory for Claude Managed Agents](#) - accessed 2026-07-20
- [Choose a permission mode - Claude Code Docs](#) - accessed 2026-07-20
- [Claude Code auto mode - Anthropic Engineering](#) - accessed 2026-07-20
- [Claude Code changelog](#) - accessed 2026-07-20
- [Claude Code changelog](#) - accessed 2026-07-20

- [Claude Code commands reference \(/fork entry, version bands quoted verbatim\)](#) - accessed 2026-07-20
- [Claude Code documentation](#) - accessed 2026-07-20
- [Claude Code MCP docs, 'Automatic backgrounding of long tool calls' section](#) - accessed 2026-07-20
- [Claude Code monitoring and usage docs \(60 KB truncation documented repeatedly\)](#) - accessed 2026-07-20
- [Claude v1 discovery: code.claude.com/docs/en/agent-loop](#) - accessed 2026-07-20
- [Claude v1 discovery: developers.openai.com/codex/cli/features](#) - accessed 2026-07-20
- [Claude v1 discovery: github.com/openai/codex/blob/main/codex-rs/linux-sandbox/README.md](#) - accessed 2026-07-20
- [Claude v1 discovery: github.com/openai/codex/issues/16068](#) - accessed 2026-07-20
- [Claude v1 discovery: www.anthropic.com/engineering/claude-code-auto-mode](#) - accessed 2026-07-20
- [Codex Advanced Configuration](#) - accessed 2026-07-20
- [Codex AGENTS.md](#) - accessed 2026-07-20
- [Codex changelog | ChatGPT Learn](#) - accessed 2026-07-20
- [Codex Configuration Reference](#) - accessed 2026-07-20
- [Codex Hooks](#) - accessed 2026-07-20
- [codex-rs/linux-sandbox/Cargo.toml — openai/codex \(crate name\)](#) - accessed 2026-07-20
- [codex-rs/linux-sandbox/README.md — openai/codex \(raw\)](#) - accessed 2026-07-20
- [Configure auto mode - Claude Code Docs](#) - accessed 2026-07-20
- [Configure the sandboxed Bash tool - Claude Code Docs](#) - accessed 2026-07-20
- [Context editing - Claude Platform Docs](#) - accessed 2026-07-20
- [Create custom subagents - Claude Code Docs](#) - accessed 2026-07-20
- [Custom prompts | ChatGPT Learn](#) - accessed 2026-07-20
- [Developer commands | ChatGPT Learn](#) - accessed 2026-07-20
- [developers.openai.com/codex/agent-approvals-security \(308 redirect observed\)](#) - accessed 2026-07-20
- [developers.openai.com/codex/changelog \(308 redirect observed\)](#) - accessed 2026-07-20
- [developers.openai.com/codex/config-advanced \(308 redirect observed live\)](#) - accessed 2026-07-20
- [developers.openai.com/codex/config-reference \(308 redirect observed\)](#) - accessed 2026-07-20
- [developers.openai.com/codex/hooks \(308 redirect observed live\)](#) - accessed 2026-07-20
- [Environment variables - Claude Code Docs](#) - accessed 2026-07-20
- [Explore the context window - Claude Code Docs](#) - accessed 2026-07-20
- [Extend Claude with skills - Claude Code Docs](#) - accessed 2026-07-20
- [GitHub Advisory Database - semantic-kernel advisories](#) - accessed 2026-07-20
- [Hooks reference - Claude Code Docs](#) - accessed 2026-07-20

- [How Claude Code works - Claude Code Docs](#) - accessed 2026-07-20
- [How Claude remembers your project - Claude Code Docs](#) - accessed 2026-07-20
- [How the agent loop works](#) - accessed 2026-07-20
- [HTTP header check: developers.openai.com/codex/{subagents,changelog,config-reference} \(direct curl, 308 responses\)](#) - accessed 2026-07-20
- [Issue #16033 — Auto-compaction never triggers — openai/codex \(GitHub API\)](#) - accessed 2026-07-20
- [Issue #16068 timeline — openai/codex \(GitHub API\)](#) - accessed 2026-07-20
- [Issue #16068 — openai/codex \(GitHub API\)](#) - accessed 2026-07-20
- [Issue #32806 — openai/codex \(GitHub API\)](#) - accessed 2026-07-20
- [Issue #33039 — openai/codex \(GitHub API\)](#) - accessed 2026-07-20
- [Issue #33447 — openai/codex \(GitHub API\)](#) - accessed 2026-07-20
- [Keep Claude working toward a goal](#) - accessed 2026-07-20
- [learn.chatgpt.com/docs/codex/config-file/config-reference \(HTTP 404 observed\)](#) - accessed 2026-07-20
- [Manage costs effectively - Claude Code Docs](#) - accessed 2026-07-20
- [Managing context on the Claude Developer Platform](#) - accessed 2026-07-20
- [Model configuration - Claude Code Docs](#) - accessed 2026-07-20
- [Model Context Protocol | ChatGPT Learn \(OpenAI Codex docs, CLI surface\)](#) - accessed 2026-07-20
- [Non-interactive mode | ChatGPT Learn](#) - accessed 2026-07-20
- [npm registry metadata for @anthropic-ai/claude-code \(dist-tags.latest and time object\)](#) - accessed 2026-07-20
- [NVD - CVE-2026-25592](#) - accessed 2026-07-20
- [NVD - CVE-2026-26030](#) - accessed 2026-07-20
- [OpenAI Codex documentation](#) - accessed 2026-07-20
- [OpenAI developer blog - Codex](#) - accessed 2026-07-20
- [openai/codex issue #20204: Inconsistent PreToolUse hook coverage across tool handlers \(most tools never emit hook events\)](#) - accessed 2026-07-20
- [openai/codex Releases](#) - accessed 2026-07-20
- [openai/codex-action](#) - accessed 2026-07-20
- [Orchestrate teams of Claude Code sessions - Claude Code Docs](#) - accessed 2026-07-20
- [Releases — openai/codex rust-v0.144.6 \(GitHub API\)](#) - accessed 2026-07-20
- [Run Claude Code programmatically](#) - accessed 2026-07-20
- [Run long-horizon tasks with Codex](#) - accessed 2026-07-20
- [Run prompts on a schedule](#) - accessed 2026-07-20
- [Scheduled tasks | ChatGPT Learn](#) - accessed 2026-07-20
- [sourcegraph/CodeScaleBench GitHub repository](#) - accessed 2026-07-20

- [Subagents | ChatGPT Learn \(OpenAI Codex docs\)](#) - accessed 2026-07-20
- [THU-KEG/RuVerBench GitHub repository](#) - accessed 2026-07-20
- [Tools reference](#) - accessed 2026-07-20

Tier 2 (14)

- [\(2510.09023\) The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections](#) - accessed 2026-07-20
- [Adaptive Evaluation of Out-of-Band Defenses Against Prompt Injection in LLM Agents \(arXiv:2606.26479\)](#) - accessed 2026-07-20
- [Are Performance-Optimization Benchmarks Reliably Measuring Coding Agents? \(arXiv:2607.01211\)](#) - accessed 2026-07-20
- [Can LLM-as-a-Judge Reliably Verify Rubrics in Agentic Scenarios? \(arXiv:2606.29920\)](#) - accessed 2026-07-20
- [More Convincing, Not More Correct: Self-Play Reward Hacking of Reference-Free LLM Judges \(arXiv:2607.05904\)](#) - accessed 2026-07-20
- [PromptArmor: An Essential Baseline for Prompt Injection Defenses | OpenReview](#) - accessed 2026-07-20
- [PromptArmor: Simple yet Effective Prompt Injection Defenses \(arXiv:2507.15219v1\)](#) - accessed 2026-07-20
- [Reliability without Validity: A Systematic, Large-Scale Evaluation of LLM-as-a-Judge Models Across Agreement, Consistency, and Bias \(arXiv:2606.19544\)](#) - accessed 2026-07-20
- [Reproducing, Analyzing, and Detecting Reward Hacking in Rubric-Based Reinforcement Learning \(arXiv:2606.04923\)](#) - accessed 2026-07-20
- [Reward Hacking Benchmark: Measuring Exploits in LLM Agents with Tool Use \(arXiv:2605.02964\)](#) - accessed 2026-07-20
- [State of Agentic AI Security and Governance 2.01 - OWASP Gen AI Security Project](#) - accessed 2026-07-20
- [State of Agentic AI Security and Governance, Version 2.01, June 2026 \(OWASP GenAI Security Project, full PDF\)](#) - accessed 2026-07-20
- [The Effect of Multi-Lingual and Keyword Adversarial Injection on LLM Relevance Judgment \(arXiv:2607.10080\)](#) - accessed 2026-07-20
- [Towards a science of scaling agent systems: When and why agent systems work - Google Research](#) - accessed 2026-07-20

Tier 3 (13)

- [Agentic Coding in 2026: A Practical Guide for Big Code \(Sourcegraph\)](#) - accessed 2026-07-20
- [Claude v1 discovery: www.anthropic.com/engineering/effective-harnesses-for-long-running-agents](#) - accessed 2026-07-20
- [Claude v1 discovery: www.anthropic.com/engineering/harness-design-long-running-apps](#) - accessed 2026-07-20

- [Claude v1 discovery: www.sophos.com/en-us/blog/inside-the-lethal-trifecta-blast-radius-reduction-in-ai-agent-deployments](https://www.sophos.com/en-us/blog/inside-the-lethal-trifecta-blast-radius-reduction-in-ai-agent-deployments) - accessed 2026-07-20
- [CodeScaleBench: Testing coding agents on large codebases and multi-repo software engineering tasks \(Sourcegraph\)](#) - accessed 2026-07-20
- [Don't Build Multi-Agents - Walden Yan, Cognition](#) - accessed 2026-07-20
- [Effective context engineering for AI agents - Anthropic Engineering](#) - accessed 2026-07-20
- [Effective harnesses for long-running agents](#) - accessed 2026-07-20
- [Harness design for long-running application development](#) - accessed 2026-07-20
- [Lessons from building Claude Code: how we use Skills](#) - accessed 2026-07-20
- [Loop engineering: getting started with loops](#) - accessed 2026-07-20
- [Practical Security Guidance for Sandboxing Agentic Workflows and Managing Execution Risk | NVIDIA Technical Blog](#) - accessed 2026-07-20
- [When prompts become shells: RCE vulnerabilities in AI agent frameworks | Microsoft Security Blog](#) - accessed 2026-07-20

Tier 5 (16)

- [Ayush Paul, 'The Memory Heist' \(published 2026-07-09\)](#) - accessed 2026-07-20
- [Best code execution sandbox for AI agents \(Northflank, vendor-authored comparison\)](#) - accessed 2026-07-20
- [Best code execution sandboxes for AI agents \(Modal, vendor-authored comparison\)](#) - accessed 2026-07-20
- [Claude v1 discovery: amux.io/guides/ai-agent-sandboxing/](https://amux.io/guides/ai-agent-sandboxing/) - accessed 2026-07-20
- [Claude v1 discovery: explainx.ai/blog/anthropic-engineer-loops-prompts-ai-coding-harness-engineering-2026](https://explainx.ai/blog/anthropic-engineer-loops-prompts-ai-coding-harness-engineering-2026) - accessed 2026-07-20
- [Claude v1 discovery: simonwillison.net/2026/Mar/16/codex-subagents/](https://simonwillison.net/2026/Mar/16/codex-subagents/) - accessed 2026-07-20
- [Claude v1 discovery: www.helpnetsecurity.com/2026/06/11/owasp-prompt-injection-ai-security-failures/](https://www.helpnetsecurity.com/2026/06/11/owasp-prompt-injection-ai-security-failures/) - accessed 2026-07-20
- [Claude v1 discovery: www.kunalganglani.com/blog/loop-engineering-agent-loops](https://www.kunalganglani.com/blog/loop-engineering-agent-loops) - accessed 2026-07-20
- [Claude v1 discovery: www.tembo.io/blog/claude-code-multi-agent-orchestration](https://www.tembo.io/blog/claude-code-multi-agent-orchestration) - accessed 2026-07-20
- [Headless Execution Mode \(codex exec\) | openai/codex | DeepWiki](#) - accessed 2026-07-20
- [LLM-as-a-Judge \(Future AGI blog\)](#) - accessed 2026-07-20
- [Ralph](#) - accessed 2026-07-20
- [Simon Willison - AI and tooling notes](#) - accessed 2026-07-20
- [Simon Willison, 'Bad Codex bug' \(2026-07-16\)](#) - accessed 2026-07-20
- [Simon Willison, 'Claude make Fable 5 permanent' \(2026-07-18\)](#) - accessed 2026-07-20
- [Simon Willison, 'How I tricked Claude into leaking your deepest, darkest secrets' \(2026-07-15\)](#) - accessed 2026-07-20